



ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA INFORMÁTICA

Trabajo de Fin de Grado

Grado en Ingeniería Informática - Tecnología Informática

Aceleración de código de simulación de difusión de contaminantes mediante optimizaciones y paralelización en CPU

Realizado por
Nicolás García Niebla

Dirigido por
José Luis Guisado Lizar

Departamento
Arquitectura y Tecnología de Computadores

Sevilla, Junio de 2026

Agradecimientos

A José Luis Guisado por guiarme en la realización del proyecto, a Antorio Navas por ayudarme con la implementación original, además de proporcionarla para este trabajo; a Luis Miguel Trinidad por su apoyo tanto en el proyecto como en el transcurso de la carrera, y a mis amigos cercanos y mi familia por el apoyo personal.

Resumen

La investigación de la contaminación y sus efectos es una rama fundamental de estudio en el ámbito científico, dado que esta puede tener consecuencias notorias en todos los ecosistemas existentes del planeta. A raíz de esto, y a partir de los avances realizados por el proyecto de investigación SAINEVRA, se busca realizar en un futuro una herramienta de simulación e investigación completa capaz de modelar entornos complejos de difusión de contaminantes.

Específicamente, como objetivo a cumplir de este proyecto, se pretende acelerar el código realizado por Antonio Navas Orozco para el proyecto SAINEVRA, haciendo uso de lenguajes de alto rendimiento, múltiples optimizaciones y aplicando técnicas de paralelización a la implementación realizada. De esta forma, se pretende acelerar el código original, de manera que permita simulaciones de mayor magnitud.

Posteriormente, se realizará un estudio de los rendimientos de las diferentes versiones, junto a otras métricas de interés (tanto en un sistema local como en el clúster de altas prestaciones I3US); para finalmente sacar conclusiones respecto a la implementación realizada.

Abstract

The study of pollution and its effects is a fundamental branch of research in the scientific field, given that it can have noticeable impacts on all existing ecosystems on the planet. As a result of this, and building upon the advances made by the SAINEVRA research project, the goal in the future is to provide a complete simulation and research tool capable of modeling complex pollutant diffusion environments.

The specific objective of this project is to accelerate the code developed by Antonio Navas Orozco for the SAINEVRA project, by using high-performance programming languages, multiple optimizations, and applying parallelization techniques to the existing code. In this manner, the intention is to accelerate the original code to allow larger-scale simulations.

Subsequently, a comprehensive performance evaluation of the different versions will be carried out, along with other metrics of interest (both on a local system and on the I3US high-performance cluster), in order to draw final conclusions regarding the implemented solution.

Índice general

1	Enfoque y descripción del proyecto	1
1.1.	Introducción sobre sistemas de computación de altas prestaciones	1
1.2.	Antecedentes del proyecto	2
1.2.1.	¿Qué es el proyecto SAINEVRA?	2
1.2.2.	Simulador de contaminantes	2
1.2.3.	Margen de mejora del simulador original	2
1.3.	Objetivos del proyecto	5
2	Diseño y funcionamiento del proyecto	7
2.1.	Funcionamiento del simulador	7
2.1.1.	Estructuras de datos principales	8
2.1.2.	Flujo de trabajo del simulador	10
2.2.	Diseño general implementado	15
2.3.	Diseño de código C	16
2.3.1.	Código auxiliar	16
2.3.2.	Código de simulación	21
2.4.	Diseño de puente entre C y Python	22
2.4.1.	ctypes	22
2.4.2.	API nativa de CPython	23
2.4.3.	cffi	23
2.4.4.	Cython	23
3	Implementación del programa	26
3.1.	Arquitectura y gestión de la memoria en CPU	26
3.1.1.	Arquitectura inicial	26
3.1.2.	Arquitectura final	30
3.2.	Implementación de código C	35
3.2.1.	Función de viento (get_wind)	35
3.2.2.	Función de matriz de difusión (get_difMatrix)	39
3.2.3.	Función de último estado de simulación (get_P_last)	40
3.2.4.	Función de simulación completa (get_P_whole)	44
3.3.	Paralelización del programa mediante OpenMP	48
3.3.1.	Selección de código a paralelizar	48
3.3.2.	Implementación de paralelización	48
3.4.	Implementación de Cython	52
3.4.1.	Estructura de código Cython	53
3.4.2.	Gestión de la memoria	54
3.4.3.	Implementación de transferencia de datos interlenguaje	55
3.5.	Automatización de la implementación	65
4	Pruebas y resultados	66
4.1.	Precisión de la implementación	66

4.1.1.	Imprecisión de la implementación original	66
4.1.2.	Código de pruebas de precisión	70
4.1.3.	Resultados de pruebas de precisión	78
4.1.4.	Imprecisión de IEEE 754	81
4.2.	Rendimiento de la implementación	82
4.2.1.	Entornos de prueba	82
4.2.2.	Rendimiento comparativo entre implementaciones	83
4.2.3.	Rendimiento comparativo entre número de hilos	87
4.3.	Resultados de aceleración	90
5	Planificación	94
5.1.	Planificación temporal	94
5.1.1.	Gestión de paquetes de trabajo (WBS)	94
5.1.2.	Diagrama de Gantt	95
5.2.	Planificación económica	95
6	Conclusiones y trabajo a futuro	97
6.1.	Conclusiones	97
6.2.	Trabajo a futuro	97
	Bibliografía	99

Índice de figuras

1.1. Diagrama de flujo de ejecución del intérprete de Python.	3
1.2. Lenguaje más usado por año según el índice TIOBE [1].	4
2.1. Ejemplo de resultados proporcionados por el simulador.	7
2.2. Diagrama de posibles salidas del simulador.	11
2.3. Diagrama de flujo de trabajo del simulador.	12
2.4. Diagrama de flujo de puente.	16
2.5. Gráfica de crecimientos computacionales de unidades de cálculo (en negro), el ancho de banda de la memoria (en verde), y la interconexión entre elementos (en azul). Fuente de la imagen: [2].	18
2.6. Diagrama de flujo de acceso a memoria. A la izquierda se representa un acceso a memoria con datos alineados, permitiendo acceder a todo ellos simultáneamente. A la derecha se representa lo opuesto, teniendo que acceder la caché a la RAM múltiples veces para cargar varios bloques, ralentizando el acceso.	19
3.1. Diagrama de funcionamiento de memoria. Se aprecia que cada núcleo de la CPU tiene su memoria L1 y L2, además que la caché L1 suele estar segmentada en datos e instrucciones. También se aprecia que L3 es compartida por todos los núcleos.	27
3.2. Diagrama de asignación de memoria en array de punteros, se ve que los punteros no apuntan a direcciones contiguas	28
3.3. Diagrama de asignación de memoria en array aplanado, donde se encadena cada array de manera contigua	29
3.4. Mapeo de registro SIMD. Obtenida de [3].	32
3.5. Diagrama de dependencias entre secciones de "get_P_last". Las dependencias son idénticas en get_P_whole, sólo que cambian las variable entre dependencias. El grafo es aplicado a cada elemento de las claves de los tensores, no habiendo dependencias entre los mismos	50
3.6. Esquema de memoria de un programa C, se aprecia que tanto "heap" como "stack" tienen un tamaño variable	54
3.7. Diferencia entre una variable (referencia) y un objeto físico en memoria según el modelo de datos de Python.	55
4.1. Diferencias máximas entre Python original y modificado, separado por tensores de simulación.	79
4.2. Diferencias máximas entre Python modificado y versiones de C, separado por tensores de simulación.	80
4.3. Resultados de prueba de crecimiento espacial en sistema doméstico.	83
4.4. Resultados de prueba de crecimiento temporal en sistema doméstico.	84
4.5. Resultados de prueba de crecimiento espacial en clúster.	85
4.6. Resultados de prueba de crecimiento temporal en clúster.	86

4.7. Tiempo de ejecución de programa con diferentes cantidades de hilos en el sistema doméstico.	87
4.8. Tiempo de ejecución de programa con diferentes cantidades de hilos en el clúster.	88
4.9. Aceleración de hilos respecto la ideal.	91
5.1. WBS del proyecto.	95
5.2. Diagrama de Gantt del proyecto.	95

Índice de tablas

2.1. Parámetros físicos del simulador de contaminantes.	8
2.2. Parámetros espaciales del simulador de contaminantes.	9
2.3. Comparativa de tecnologías puente entre Python y C.	25
4.1. Tablas de diferencias máximas absolutas.	81
4.2. Especificaciones técnicas de los entornos de prueba.	83
4.3. Tiempos de ejecución según el tamaño de la matriz en sistema doméstico.	84
4.4. Tiempos de ejecución según el tiempo de simulación en sistema doméstico.	85
4.5. Tiempos de ejecución según el tamaño de la matriz en clúster.	86
4.6. Tiempos de ejecución según el tiempo de simulación en clúster.	87
4.7. Tiempos de ejecución según la cantidad de hilos de OpenMP en el sistema doméstico.	88
4.8. Tiempos de ejecución según la cantidad de hilos de OpenMP en el clúster.	90
4.9. Factores de aceleración respecto a la versión en Python para una matriz de tamaño 1000x1000.	90
4.10. Comparativa de aceleración real frente a ideal en el sistema doméstico.	92
4.11. Comparativa de aceleración real frente a ideal en el clúster de altas prestaciones.	92
5.1. Presupuesto detallado del proyecto	96

Índice de códigos

2.1. Inicialización del diccionario de tensores de contaminantes G.	10
2.2. Funciones de inicialización y liberación de memoria para familias de tensores 2D y 3D.	20
3.1. Macro de acceso a matrices y tensores aplanados.	30
3.2. Ejemplo de uso de etiqueta "restrict"	31
3.3. Extracto de código de uso de componentes del viento, donde se aprecia que cada una tiene un índice distinto	32
3.4. Estructura de bucles de implementación (rows=width, cols=height) .	33
3.5. Código de estructuras "SimulationData" y "MapData"	34
3.6. Código de estructura "WindComponents"	34
3.7. Código de estructura "Tensor"	35
3.8. Código de inicio de función "get_wind"	36
3.9. Código de bucle de función "get_wind"	38
3.10. Código de inicio de función "get_difMatrix"	39
3.11. Código de bucle de función "get_difMatrix"	40
3.12. Código de inicio de función "get_P_last"	41
3.13. Código de bucle de función "get_P_last"	42
3.14. Código de inicio de función "get_P_whole"	44
3.15. Código de bucle de función "get_P_whole"	45
3.16. Código de bucles paralelizados de funciones "get_wind" y "get_difMatrix", en ambos aplica misma paralelización	49
3.17. Código de bucles paralelizados de función "get_P_last"	51
3.18. Código de bucles paralelizados de función "get_P_whole"	51
3.19. Código de compilación del programa	53
3.20. Código de clases "SimulationData" y "MapData"	56
3.21. Código de clase auxiliar "TensorDeallocator"	57
3.22. Código de función "dict_to_contiguous_4d_tensor"	58
3.23. Código de función "get_P_last_p"	60
3.24. Código de función "get_P_whole_p"	63
4.1. Extracto de inicio de implementación original.	67
4.2. Inicio código de prueba de precisión, en la que se encuentra la implementación con datos igualados.	67
4.3. Código auxiliar para pruebas.	70
4.4. Código de benchmark de precisión.	74

1. Enfoque y descripción del proyecto

1.1. Introducción sobre sistemas de computación de altas prestaciones

Los sistemas de computación de altas prestaciones han sido y son un pilar fundamental de la historia de la informática y las ciencias modernas, impulsando enormes avances científicos en campos de investigación como la medicina, la física, las matemáticas, entre otros. Estos sistemas se caracterizan por su alta capacidad de cómputo y paralelización, la cual puede ser usada en la aceleración de programas informáticos, siendo los más beneficiados aquellos que tengan las capacidades de aprovechar la infraestructura de este tipo de sistemas.

Para poner en perspectiva la importancia de los sistemas de computación de altas prestaciones y, por ende, de la programación de altas prestaciones, es necesario definir qué es la paralelización. Esta estrategia consiste en dividir una tarea computacional grande y compleja en subtareas más pequeñas que pueden ser ejecutadas de manera simultánea. Históricamente, la computación paralelizada ha sido la clave para resolver problemas intratables con otros métodos, permitiendo hitos como el ensamblaje computacional de miles de millones de pares de bases de ADN durante el Proyecto Genoma Humano mediante arquitecturas de memoria distribuida [4], la predicción de eventos meteorológicos severos con días de antelación utilizando modelos numéricos globales [5], o el estudio del plegamiento de proteínas a nivel atómico en el desarrollo de nuevos fármacos empleando supercomputadores de propósito específico como Anton [6].

En el caso de las simulaciones, que es el campo en el que este proyecto se va a enfocar, la computación de altas prestaciones permite simular entornos dinámicos con un nivel de detalle y escala muy alto. Se consigue, por ello, modelar interacciones masivas de agentes y variables continuas que serían prohibitivas de estudiar empíricamente en el mundo real, tales como el diseño aerodinámico de aeronaves sin depender exclusivamente de túneles de viento físicos mediante la Dinámica de Fluidos Computacional (CFD) [7], la proyección del calentamiento global a décadas vista utilizando los modelos acoplados del CMIP [8], o la optimización de semáforos y reducción de la huella de carbono metropolitana ejecutando simulaciones multiagente de millones de viajes diarios con herramientas como MATSim o SUMO [9, 10].

Actualmente, el hardware empleado en sistemas de computación de altas prestaciones (HPC) se ha diversificado y especializado enormemente. Se componen principalmente de clústeres interconectados mediante redes de baja latencia (como "InfiniBand" [11]). Los nodos de cálculo modernos no solo integran potentes procesadores multinúcleo con avanzadas jerarquías de memoria caché y extensiones vectoriales, sino que dependen cada vez más de aceleradores hardware. Entre estos destacan las Unidades de Procesamiento Gráfico (GPU) y

Unidades de Procesamiento Tensorial (TPU), las cuales ofrecen una capacidad de procesamiento paralelo muy superior a las CPUs tradicionales, siendo ideales para operaciones altamente paralelizables.

1.2. Antecedentes del proyecto

1.2.1. ¿Qué es el proyecto SAINEVRA?

Pasando específicamente al trabajo realizado en este proyecto, primero se han de explicar los antecedentes del mismo. Para ello, se ha de detallar primero en qué consiste el proyecto SAINEVRA [12]. Este estudio, actualmente activo, es la continuación de la investigación SANEVEC (finalizado el 30/09/2025) [13], y consiste de un proyecto financiado por la Agencia Estatal de Investigación, del Ministerio de Ciencia e Innovación; cuyo objetivo es investigar, diseñar e implementar una herramienta de simulación (a partir del simulador preliminar SIMTRAVEL [14]), la cual se usará para realizar estudios predictivos sobre los efectos de la disposición de una red urbana de estaciones de recarga de vehículos eléctricos sobre la congestión de tráfico, la contaminación, el uso de la red eléctrica, entre otros.

1.2.2. Simulador de contaminantes

Entre los elementos clave con los que cuenta el proyecto SAINEVRA, está el simulador de difusión de contaminaciones desarrollado por Antonio Navas Orozco. Más adelante se entrará en profundidad en cuanto a su funcionamiento, pero por ahora basta con conocer que se trata de un programa codificado en Python con integración de la librería NumPy [15, 16], el cual simula de forma discretizada (tanto en tiempo y en espacio) la emisión de contaminantes generados por cada vehículo en una simulación de tráfico y la propagación de dichos contaminantes a través de la atmósfera, teniendo en cuenta los efectos de su difusión, de un viento homogéneo y de las estructuras circundantes (tales como edificios).

El programa recibe los puntos de contaminación iniciales generados por los vehículos, de forma que el simulador de gestión del movimiento y la lógica de los vehículos es completamente independiente de la difusión de los contaminantes. Esto no sólo sirve para poder modularizar el desarrollo, sino que facilita la implementación de optimizaciones más exhaustivas al simplificar la complejidad del código.

1.2.3. Margen de mejora del simulador original

Python

Como ya se ha mencionado, el simulador original se encuentra programado en Python estándar (CPython). Este se trata de un lenguaje de programación interpretado de alto nivel, ampliamente usado en múltiples campos de la informática, cuya implementación se encuentra programada en C. Fue creado por Guido van Rossum y lanzado por primera vez en 1991, con el objetivo de crear una

filosofía de diseño que hiciera hincapié en la legibilidad y simplicidad del código [15]. Las principales características del lenguaje son las siguientes:

- **Claridad y flexibilidad del código:** dada la inherente sencillez que impone la sintaxis y semántica del lenguaje, el código resultante suele ser claro y simple en cuanto a su comprensión, dando una facilidad superior a otros lenguajes al momento de desarrollar en el mismo. Además de eso, el lenguaje es multiparadigma, es decir que permite la programación orientada a objetos, la programación imperativa o la programación funcional.
- **Lenguaje interpretado:** aunque se hable de Python como un lenguaje interpretado y, por ende, no debería existir un tiempo de compilación; se ha de puntualizar que sí que hay un proceso de compilación del código Python. Sin embargo, este no compila a C o a ensamblador, sino que se transforma el código completo a un código intermedio conocido como "bytecode de Python". Este lenguaje no puede ser procesado por una CPU estándar, por lo cual se hace uso de la Máquina Virtual de Python (PVM) programada en C, la cual es la encargada de finalmente interpretar y ejecutar el programa compilado instrucción a instrucción en tiempo de ejecución (es decir, a la par que el programa correspondiente se esté ejecutando) [17]. Este proceso de interpretación en tiempo de ejecución se puede observar detalladamente en el diagrama 1.1. Esto le permite al lenguaje tener características con las que otros lenguajes no cuentan, como el tipado dinámico (es decir, los tipos de las variables no son fijos y se pueden modificar en cualquier momento).

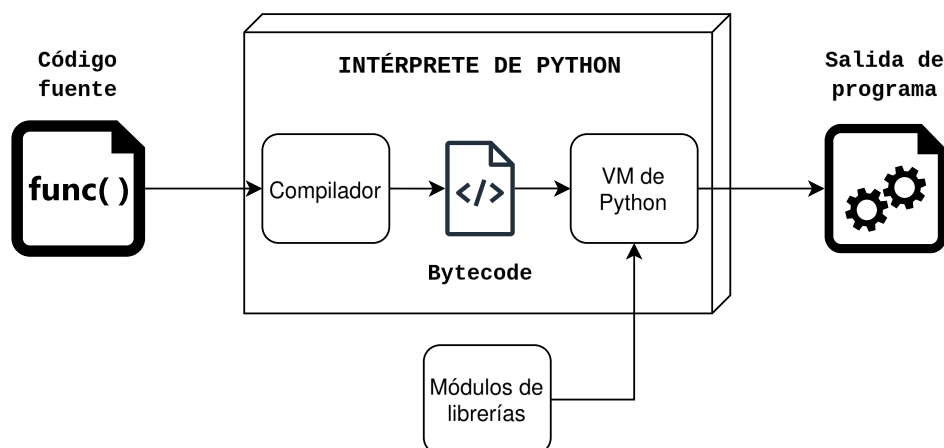


Figura 1.1: Diagrama de flujo de ejecución del intérprete de Python.

- **Popularidad y bibliotecas externas:** debido a que se trata de uno de los lenguajes de programación más conocidos y empleados, este cuenta con extensas librerías externas que permiten al lenguaje ser empleado en múltiples campos de la informática, como por ejemplo el desarrollo de aplicaciones, la ciencia computacional o la inteligencia artificial. Un ejemplo de librería ampliamente usada es NumPy. Esta se ha convertido en un estándar de la industria debido a la introducción de un objeto llamado "ndarray", una estructura implementada internamente en C la cual permite vincular operaciones matemáticas con código nativo de C. Sea prueba de ello la gráfica 1.2

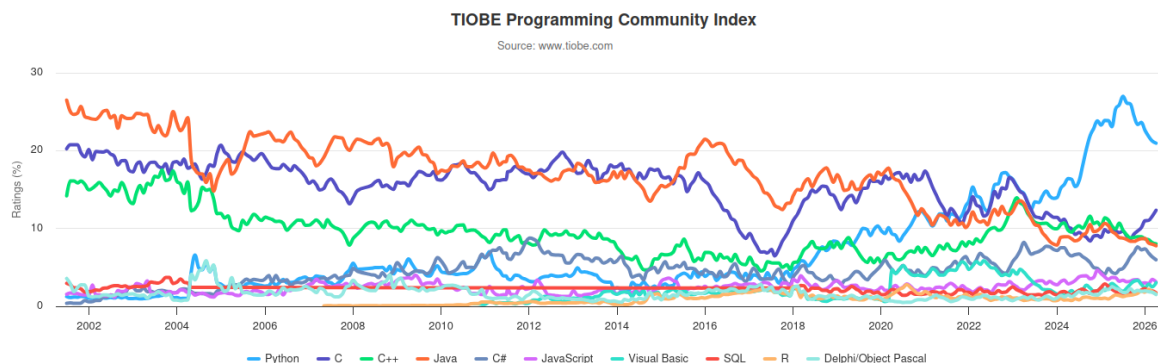


Figura 1.2: Lenguaje más usado por año según el índice TIOBE [1].

Sin embargo, Python cuenta con un problema notorio para la aplicación de este trabajo: su rendimiento. Este no sólo se ve perjudicado por el hecho de ser un lenguaje interpretado, lo cual hace de por sí que su velocidad de ejecución se vea reducida [18]; sino que además este cuenta con un elemento clave para su funcionamiento que afecta a su rendimiento llamado "Global Interpreter Lock" (GIL) [19]. Este mecanismo fuerza a que el programa sólo pueda ser ejecutado por un único hilo, con el objetivo de evitar problemas de referencia de variables entre diferentes hilos (más adelante se explica en detalle el referenciado y conteo de variables), a costa de no poder paralelizar el programa en forma de hilos para procesos exclusivos de la CPU. Más específicamente, se bloquea la posibilidad de paralelización del tratamiento del bytecode; procesos como lectura en disco, descargas por internet o, en general, procesos I/O sí son paralelizables.

C

C es un lenguaje de programación de bajo/medio nivel de propósito general, desarrollado originalmente por Dennis Ritchie entre 1969 y 1973 en los Laboratorios Bell [20]. Es conocido por ser uno de los lenguajes más influyentes de la historia, dado que sirvió como base, tanto teórica como de implementación, de muchos de los lenguajes y tecnologías que se usan hoy en día (como por ejemplo Python). Además, es conocido por su excelente rendimiento, dada su cercanía con el hardware del sistema y la casi nula capa de abstracción (siendo únicamente la compilación del código C a código máquina nativo). Profundizando un poco en las características de C, se pueden destacar los siguientes puntos:

- **Lenguaje compilado:** como se ha mencionado, se trata de un lenguaje compilado; es decir, que el código ha de ser procesado y transformado a código máquina nativo antes de poder entrar en la fase de ejecución. Si bien esto hace que este pierda cierta flexibilidad (como por ejemplo el tipado estático), también permite poder realizar optimizaciones más exhaustivas al código antes de siquiera ejecutarse, permitiendo acelerar notoriamente el código resultante en comparación a si no se realizasen las mejoras correspondientes. A raíz de esto, es necesario hacer uso de un compilador, siendo el estándar de la industria y, por ende el más popular, GCC (el cual será usado en este proyecto) [21].

- **Lenguaje de bajo nivel:** dado que es un lenguaje de bajo/medio nivel, permite un control mayor de los procesos del sistema, llegando a poder asignar secciones de código nativo a instrucciones de ensamblador de manera directa. De esta forma, se permite así una precisión superior a muchos otros lenguajes en cuanto a optimizaciones a nivel de instrucciones del programa; a costa de, evidentemente, complicar el código a nivel técnico y pudiendo reducir la claridad del código.
- **Gestión de memoria manual:** a raíz de lo anterior, la gestión de la memoria también se enfoca desde un nivel bajo de abstracción, llegando a tener que controlar su uso y los recursos de la misma de manera manual (cosa que con Python no sucedía). Si bien esto permite un control absoluto de la capacidad de la misma, pudiendo aprovechar la memoria en su totalidad; también da lugar de manera recurrente a múltiples problemas procedentes de la naturaleza manual de la misma, como por ejemplo las fugas de memoria por no liberar recursos en desuso (memory leaks) o accesos a zonas de la memoria no permitidas o inexistentes (segmentation fault). Por lo tanto, se ha de tratar con sumo cuidado para evitar posibles problemas de memoria en la ejecución del código.

Además de lo anteriormente mencionado, al contrario que Python, C sí permite la paralelización del código en CPU, pudiendo realizarse de manera cómoda mediante el uso de OpenMP. OpenMP es una interfaz de programación de aplicaciones que soporta la programación multihilo de memoria compartida en C, C++ y Fortran. Mediante el uso de las directivas de compilador `"#pragma omp"`, se pueden especificar numerosas formas de proceder con una sección de código a la hora de paralelizarlo [22].

Con esto, se puede llegar a la conclusión de que, si se pudiese implementar en C el simulador de difusión de viento, no sólo mejoraría el rendimiento por el hecho de pasarlo a un lenguaje con mayor eficiencia demostrada [18], sino que además se abriría la posibilidad a que ciertas secciones del código se paralelicen y, por tanto, aumente considerablemente la aceleración del código.

1.3. Objetivos del proyecto

Con todo lo anteriormente comentado, se pueden perfilar los objetivos del proyecto en los siguientes puntos:

- Analizar y comprender en profundidad el simulador base del proyecto SAINEVRA programado en Python con NumPy.
- Acelerar el código con múltiples técnicas de optimización de código en el lenguaje C.
- Paralelizar en código C en CPU mediante herramientas como OpenMP.
- Vincular el código C con Python de forma que se pueda usar las funciones programadas y paralelizadas en C en un entorno Python cualquiera.

- Realizar pruebas y sacar conclusiones en un entorno de alta computación, específicamente el clúster HPC de I3US [\[23\]](#) (más adelante se entrará en detalle del entorno).

2. Diseño y funcionamiento del proyecto

Esta sección se dedicará a la descripción del diseño del simulador, tanto a nivel teórico como a nivel de implementación. Además, se discutirá sobre distintas alternativas para la comunicación entre C y Python.

2.1. Funcionamiento del simulador

Para comprender las decisiones de diseño tomadas y la implementación final realizada, se ha de detallar en profundidad en qué consiste realmente el simulador proporcionado, qué datos recibe, qué resultados retorna, cuáles son las estructuras de datos usadas y cómo es, en general, el flujo del algoritmo. El simulador tiene como objetivo modelar la difusión de gases contaminantes producidos por vehículos dentro de un espacio definido y discretizado. El entorno tiene, además de una modelización de las edificaciones por las cuales no pueden pasar los contaminantes, una serie de variables físicas que pueden afectar a cómo los mismos se distribuyen y se mueven a lo largo del espacio. Un ejemplo visual de las salidas y el entorno generados por este simulador se muestra en la figura 2.1.

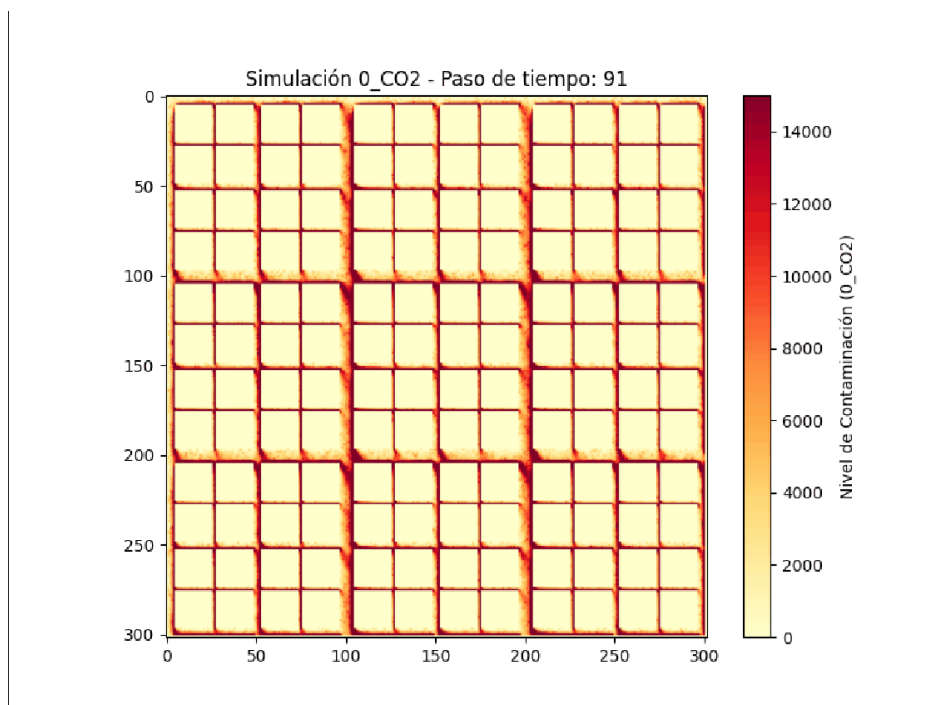


Figura 2.1: Ejemplo de resultados proporcionados por el simulador.

2.1.1. Estructuras de datos principales

Las operaciones correspondientes a los algoritmos del simulador se basan en operaciones matriciales y tensoriales, requiriendo de esta manera estructura afines a estos tipos de datos. Además de eso, como se ha mencionado anteriormente, el simulador cuenta con una serie de parámetros que alteran la física del entorno y determinan cómo se distribuirán los contaminantes a lo largo del tiempo y el espacio.

Parámetros físicos y de entorno

Empezando por los parámetros físicos del simulador, este dispone de los descritos a continuación:

Parámetro	Tipo de dato	Descripción
Tipos de vehículos (carTypes)	int[] (Lista)	Representa los tipos de vehículos: 0 (eléctricos), 1 (gasolina), 2 (diésel).
Tipos de contaminantes (pollutants)	string[] (Lista)	Nombres de los contaminantes de la simulación (por ejemplo "CO2", "NOx", "VOC").
Factor γ (gamma)	float	Tasa de disipación de gases en capas atmosféricas superiores.
Factor δ (delta)	float	Velocidad de difusión base de los contaminantes.
Factor de esquina (corner_factor)	float	Fracción de contaminantes propagados hacia esquinas adyacentes.
Escalar viento norte (WN)	float	Componente de la velocidad del viento en dirección norte (puede ser negativo para ir en dirección opuesta).
Escalar viento este (WE)	float	Componente de la velocidad del viento en dirección este (puede ser negativo para ir en dirección opuesta).

Tabla 2.1: Parámetros físicos del simulador de contaminantes.

El entorno también se rige por una serie de parámetros, los cuales son:

Parámetro	Tipo de dato	Descripción
Altura (height)	int	Se corresponde con el eje X de la simulación
Anchura (width)	int	Se corresponde con el eje Y de la simulación
Tiempo (times)	int	Se corresponde con el tiempo de duración de la simulación

Tabla 2.2: Parámetros espaciales del simulador de contaminantes.

Además de los parámetros espaciales mostrados en la tabla, se cuenta con el parámetro matricial de accesibilidad (acc). Este consiste en una matriz bidimensional que se corresponde con las edificaciones del mapa. Establece cada casilla con un 0, si existe una edificación; o un 1, en el caso opuesto. Ha de tener un margen de 1 casilla (celdas fantasma) en cada uno de los bordes. Esto permite aplicar las operaciones convolucionales en los límites del mapa de manera sencilla, de forma que no sea necesario añadir una lógica condicional de salto de casilla o de trato excepcional que ralentizaría el rendimiento.

Entrada de contaminación inicial (G)

Como ya se mencionó anteriormente, el simulador de difusión de contaminantes está separado del de los vehículos, por lo que se han de recibir como entrada los puntos por los que los vehículos han pasado para cada instante de la simulación y, por ende, por dónde han generado el punto inicial de contaminación. De esta forma, el simulador recibe estos datos en forma de un conjunto de tensores tridimensionales mapeados por pares de claves. Esto se puede expresar matemáticamente de la siguiente manera:

$$G = \{(v, c) \mapsto \mathbf{T}_{v,c} \mid v \in V, c \in C\}$$

Donde V representa el conjunto de tipos de vehículos y C el conjunto de contaminantes. A su vez, cada tensor tridimensional $\mathbf{T}_{v,c}$ se rige por las dimensiones del espacio y tiempo del simulador:

$$\mathbf{T}_{v,c} \in \mathbb{R}^{(width+2) \times (height+2) \times (times+1)}$$

Como se muestra en el código 2.1 de la versión de Python, se implementa mediante un diccionario con claves de tipo string, compuesta por un nombre de contaminante y un tipo de vehículo. A su vez, esto se asocian a arrays de NumPy, con dimensiones (width+2, height+2, times+1) lo cual, como la etiqueta implica por su nombre, significa que cada tipo de contaminante de cada tipo de vehículo tiene asignado un espacio-tiempo correspondiente a los puntos iniciales de contaminación generados por el simulador de los vehículos (cabe destacar que, más adelante, se modificará ligeramente esta estructura por motivos de optimización en la implementación del código en C).

```

1 G = {f"{cartype}_{pollutant}": np.zeros((width+2, height+2, times+1),
    dtype=np.float32)
2     for cartype in carTypes
3     for pollutant in pollutants}

```

Código 2.1: Inicialización del diccionario de tensores de contaminantes G.

Estructuras de datos intermedias

Durante la ejecución del simulador, hay una serie de cálculos que se usan de forma continua, resultando en una carga de cálculo redundante. Para solventar esto, se han incluido 2 estructuras que almacenan valores esenciales para los cálculos de la difusión y propagación, específicamente:

- **Tupla de viento:** consiste en una lista de 9 elementos matriciales. Cada uno se corresponde con una dirección del viento (norte, sur, este, oeste, noroeste...), representando para los 8 primeros elementos los coeficientes de desplazamiento desde cada casilla vecina hacia la casilla central, mientras que el noveno representa la fracción de aire que permanece estática (es decir, la cantidad no desplazada). Las matrices ya asimilan los bloqueos generados por la matriz de accesibilidad.
- **Matriz de difusión:** matriz bidimensional que determina el coeficiente base de retención de cada celda tras aplicar la difusión. Se calcula evaluando cuántos vecinos accesibles tiene cada coordenada, reduciendo la dispersión en zonas cerradas respecto a zonas abiertas.

Resultado del simulador (P)

El simulador retorna una estructura similar a la de entrada, es decir, un diccionario con claves string y tensores representando el resultado en el espacio. Sin embargo, existen 2 maneras de operar con el simulador en cuanto al resultado final: el primero es obtener sólo el estado final de la simulación, en cuyo caso se retorna como tensor una matriz bidimensional espacial, de forma que no tiene dimensión temporal y este representa estado de la simulación en el último instante; el segundo es obtener la simulación completa, retornando así el mismo tipo de tensor que el de entrada, con la diferencia de que este representa el estado de la simulación de difusión de los contaminantes en cada instante de tiempo. Sirva de referencia el diagrama correspondiente a la imagen [2.2](#).

2.1.2. Flujo de trabajo del simulador

Definiendo el flujo de trabajo, este puede ser representado mediante el esquema [2.3](#) y los siguientes pasos:

- Se inicializan los valores de simulación necesarios para las condiciones espaciales y físicas.
- Se calculan las matrices de las componentes del viento.

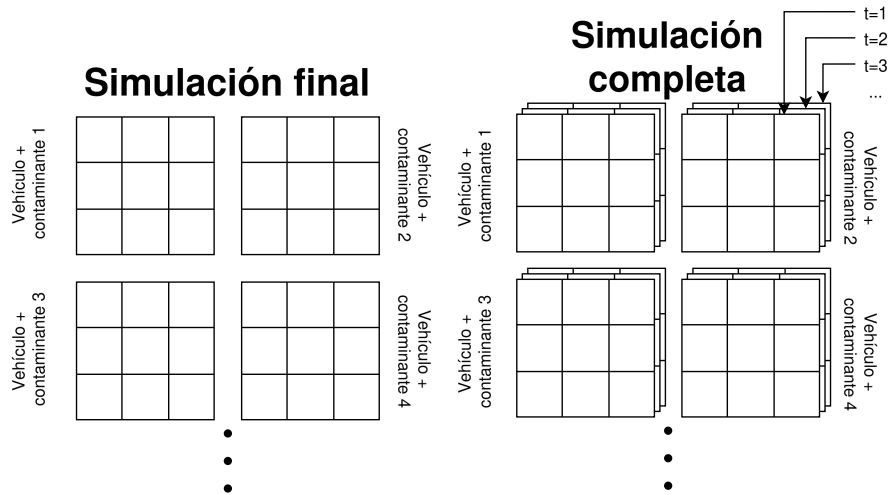


Figura 2.2: Diagrama de posibles salidas del simulador.

- Se calcula la matriz de difusión.
- Se inicializa un bucle que recorre tanto las claves de los elementos tensoriales del diccionario como el tiempo.
- Se realiza un paso de difusión respecto a los datos de la iteración temporal anterior.
- Se aplica un paso de movimiento a raíz del viento de la simulación.
- En función de si es una simulación final o completa, se retorna uno de los tipos de resultados anteriormente explicados.

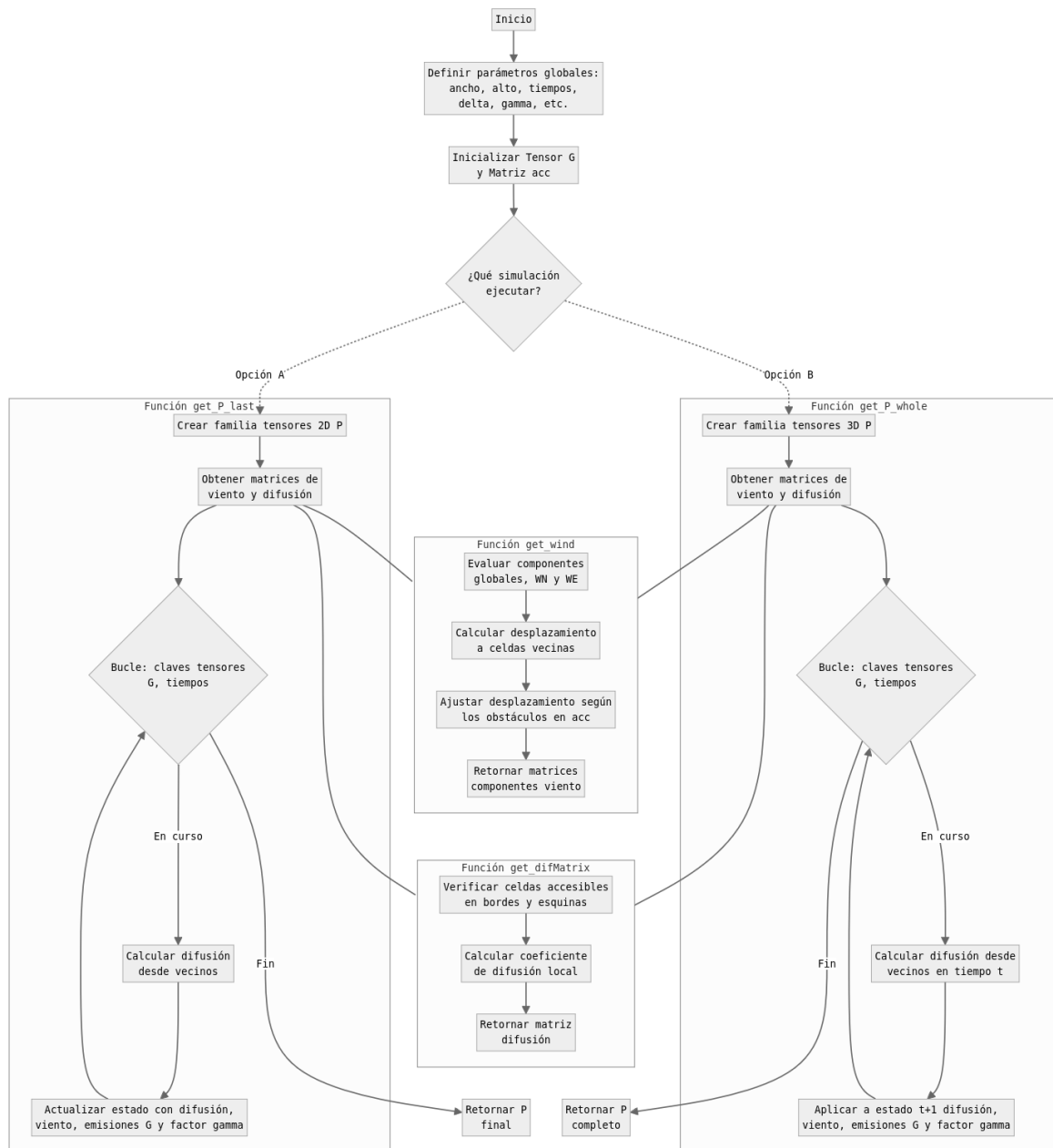


Figura 2.3: Diagrama de flujo de trabajo del simulador.

Función de viento

La función de viento se encarga de retornar las componentes del viento en forma de matrices a partir de los parámetros de los escalares de viento WN y WE y la matriz de accesibilidad. Esta se recorre con el objetivo de determinar si el viento se encuentra con un edificio (ya sea de frente o en una esquina). En caso de haberlo, el coeficiente del viento hacia la celda se anula, de forma que los contaminantes terminan bordeando las edificaciones y desplazándose a zonas abiertas o estancándose en zonas cerradas.

Algoritmo 1 Función de matrices de flujo de viento

```
1: function GET_WIND
2:   Inicializar 8 matrices de desplazamiento direccional
   (N, S, E, O, NE, NO, SE, SO) y 1 matriz central (stays)
3:   for cada celda en el eje del parámetro width + 1 en el mapa do
4:     for cada celda en el eje del parámetro height + 1 en el mapa do
5:       Calcular flujos verticales en función de WN y obstáculos cercanos
6:       Calcular flujos horizontales en función de WE y obstáculos cercanos
7:       Calcular flujos diagonales como combinaciones de WN y WE
8:       Calcular la fracción de viento que permanece en la celda (stays)
9:     end for
10:  end for
11:  Recortar bordes inactivos de las matrices
12:  return las 9 matrices de las componentes del viento
13: end function
```

Función de matriz de difusión

Genera una matriz que establece el comportamiento pasivo de los contaminantes. Se evalúan el número de casillas vecinas sin edificio para luego, en función de las mismas, ajustar los pesos de dispersión mediante los factores delta y de esquina. Las celdas con menos vecinos abiertos, por tanto, contendrán en mayor medida los contaminantes en cada iteración, siendo lo opuesto para zonas sin apenas edificaciones.

Algoritmo 2 Función de matriz de difusión

```
1: function GET_DIFMATRIX
2:   for cada celda en el eje del parámetro width + 1 en el mapa do
3:     for cada celda en el eje del parámetro height + 1 en el mapa do
4:       Contar vecinos ortogonales accesibles
5:       Contar vecinos diagonales accesibles
6:       Calcular factor de difusión de casilla
7:     end for
8:   end for
9:   return matriz de difusión
10: end function
```

Función de difusión final

Aplica el bucle descrito en el flujo general, pero empleando únicamente 2 matrices espaciales: la del paso anterior, que se usará para realizar los cálculos; y la actual, que sobrescribe el paso anterior y se retornará al finalizar la simulación. En cada paso del bucle, se extraen los valores de difusión y advección (movimiento horizontal del viento) de los vecinos en el estado anterior, luego se suma lo obtenido a los datos del instante actual del tensor de entrada G y finalmente se multiplica por el factor de $(1-\gamma)$ para simular la pérdida de contaminantes en capas superiores de la atmósfera. La idea de esta función es minimizar el uso de RAM y evitar cuellos de botella por la misma.

Algoritmo 3 Función de difusión final

```
1: function GET_P_LAST
2:   Inicializar  $P$  como un diccionario de matrices 2D en ceros (solo almacena un
   estado temporal)
3:    $wind$  = llamar función get_wind
4:    $dif_{matrix}$  = llamar función get_difMatrix
5:   for cada estado temporal desde 0 hasta  $times + 1$  do
6:     for cada cada clave de la familia de tensores  $G$  do
7:       for cada celda en el eje del parámetro  $width + 1$  en el mapa do
8:         for cada celda en el eje del parámetro  $height + 1$  en el mapa do
9:           Actualizar  $P$  con aplicación de difusión a estado actual
10:          Actualizar  $P$  con aplicación de viento, fuentes de emisión y
           pérdidas a estado actual
11:         end for
12:       end for
13:     end for
14:   end for
15:   return diccionario  $P$  con los mapas de contaminación finales
16: end function
```

Función de difusión completa

Aplica un procedimiento idéntico al anterior en cuanto al algoritmo implementado. La diferencia con este es el hecho de que se almacenan todos los estados temporales de la simulación, retornando un diccionario de tensores, siendo estos todos los estados espacios-temporales de cada clave. Dada la naturaleza de la función, el consumo de memoria de la misma se puede volver rápidamente un problema en caso de no ser gestionado correctamente.

Algoritmo 4 Función de difusión completa

```
1: function GET_P_WHOLE
2:   Inicializar  $P$  como un diccionario de matrices 3D en ceros
3:    $wind$  = llamar función get_wind
4:    $dif_{matrix}$  = llamar función get_difMatrix
5:   for cada estado temporal desde 0 hasta  $times + 1$  do
6:     for cada cada clave de la familia de tensores  $G$  do
7:       for cada celda en el eje del parámetro  $width + 1$  en el mapa do
8:         for cada celda en el eje del parámetro  $height + 1$  en el mapa do
9:           Actualizar  $P$  aplicación de difusión a estado actual en tiempo  $t$ 
10:          if  $t < times$  then
11:            Actualizar  $P$  aplicación de viento, fuentes de emisión y
            pérdidas a estado actual en tiempo  $t + 1$ 
12:          end if
13:        end for
14:      end for
15:    end for
16:  end for
17:  return diccionario  $P$  con los tensores de contaminación completos
18: end function
```

2.2. Diseño general implementado

La arquitectura del código de la nueva versión se ha definido de la siguiente manera: la sección de código con mayor coste computacional del simulador se ha delegado en un nuevo código C, ya que es uno de los lenguajes con mejor rendimiento en cuanto a computación matemática [18]; mientras que su uso y ejecución desde la parte del usuario se ha dejado en Python, dado que el lenguaje tiene un enfoque a un mayor nivel de abstracción, además de ser compatible con el resto del proyecto de SAINEVRA, el cual se encuentra en este mismo lenguaje. Sin embargo, esto abre la necesidad de implementar un puente comunicativo entre Python y C, de forma que Python se use como interfaz y C como código computacional de altas prestaciones.

En consecuencia, el flujo de funcionamiento correspondiente al diagrama 2.4 será el siguiente:

- Desde Python, se inicializan las variables de la simulación y se llama a una función, estando esta vinculada a un código específico en C. Estas funciones se inician con tipos naturales procedentes de Python, aunque puede tener algún requerimiento de formato específico para las variables de entrada.
- Una vez se llame a la función correspondiente, se usa el puente entre Python y C para traspasar los tipos de datos de entrada entre un lenguaje y otro, de forma que se pueda ejecutar la función programada en C con estructuras y tipos de datos nativos del lenguaje.
- Se ejecuta el código optimizado en C y se retorna el resultado de la función

con un tipo específico de C.

- Nuevamente en el puente, se realiza una transformación del tipo del resultado en C y lo traspasa a un tipo compatible con Python.
- Finalmente, la función de Python retorna el resultado de forma que sea compatible directamente en el lenguaje.

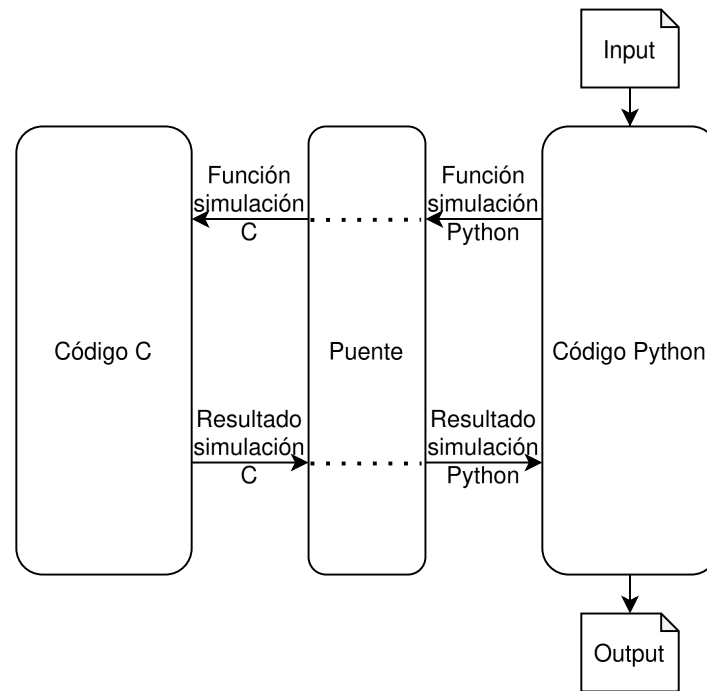


Figura 2.4: Diagrama de flujo de puente.

2.3. Diseño de código C

El código en C se ha segmentado en 4 archivos, siendo 2 de ellos códigos de cabecera (.h) y los otros 2 códigos funcional (.c). Estos tipos de códigos no tienen estrictamente asociados una función específica y definida por el lenguaje (la diferencia reside en que .h expone la interfaz pública del módulo, mientras que .c oculta la implementación), pero en general y por convenio (y por tanto en nuestro caso), la función de estos se describe de la siguiente forma: los códigos de cabecera en C son usados para la declaración de utilidades, macros, tipos y estructuras usadas en el resto del código; mientras que los códigos funcionales son los que suelen contener la lógica y funcionamiento del código con mayor carga computacional.

Junto con lo anterior, el código se segmenta en 2 partes principales: código auxiliar ("simulatorModule") y código de simulación ("contaminationSeparated").

2.3.1. Código auxiliar

El código de cabecera y funcional definido como auxiliar es aquel en el que se definen las estructuras de datos, las macros y las funciones de apoyo usadas en el

programa. Estos elementos se usarán a lo largo del programa de manera análoga, como método para reducir la longitud y complejidad del código.

Estructuras de datos

El lenguaje C permite la creación de tipos de datos compuestos definidos mediante la etiqueta "struct", los cuales permiten almacenar distintos tipos de datos y asignarles a cada uno un nombre en clave para referirse a ellos. Los detalles de su funcionamiento se explicarán más adelante, pero lo crucial es comprender la idea de que es un tipo de dato que permite relacionar otros tipos de datos (en el caso de esta implementación se usa de manera equivalente al tipo de dato diccionario definido en Python u otros lenguajes de programación, pero a nivel técnico difieren en su implementación y funcionamiento interno). Con la definición ya dada, se procede a detallar las estructuras definidas en el programa:

- **Datos de la simulación:** datos de entrada correspondientes a tipos de vehículos, número de contaminantes y parámetros físicos.
- **Datos del mapa:** datos relacionados con la estructura y forma del mapa de simulación, junto con las casillas anuladas y el tiempo de simulación (este último se ha añadido aquí dado que se ha planteado como una dimensión extra del mapa bidimensional).
- **Componentes del viento:** datos auxiliares usados para la definición de las componentes del viento en el simulador.
- **Tensor:** datos de un tensor de emisiones. Este se compone por un identificador (tipo de vehículo y tipo de contaminante) y los datos relacionados a ese identificador.

Aplanamiento de datos y acceso espacial

En primer lugar, es necesario mencionar el problema conocido como "memory wall" [24]. Este establece que la velocidad de procesamiento de las CPU ha crecido a un ritmo mucho mayor que la velocidad de acceso a la memoria. Como resultado, se ha creado un cuello de botella sistémico, donde el procesador pasa una gran cantidad de tiempo inactivo esperando a que los datos sean transferidos. Esta brecha de rendimiento sigue siendo un desafío crítico hoy en día, limitando drásticamente el rendimiento de cargas de trabajo modernas que manejan grandes volúmenes de datos, tal y como puede apreciarse en la gráfica 2.5.

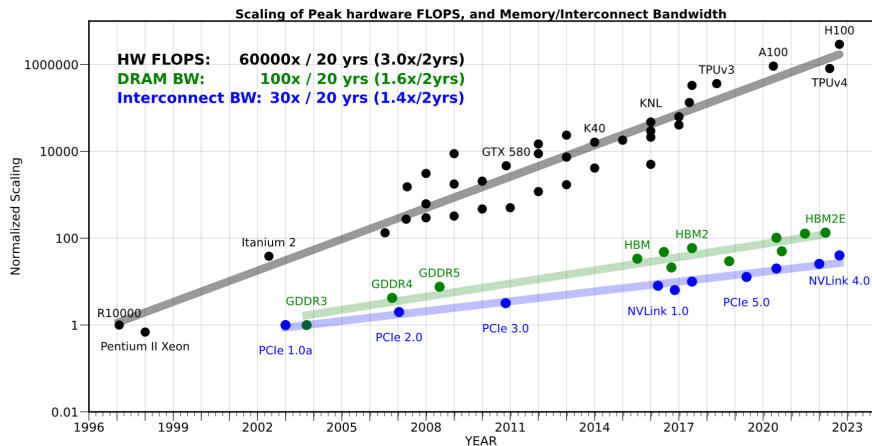


Figura 2.5: Gráfica de crecimientos computacionales de unidades de cálculo (en negro), el ancho de banda de la memoria (en verde), y la interconexión entre elementos (en azul). Fuente de la imagen: [2].

A raíz de esto, uno de los puntos claves en la optimización del código es el acceso a los datos en memoria. Específicamente, se ha tenido en cuenta para el diseño los posibles cuellos de botella generados por la localidad espacial, es decir, el hecho de que elementos relacionados o de acceso continuo no se encuentren disponibles en la caché. Esto implica que, como se ve en el diagrama 2.6, el programa debe cargar un bloque entero en caché sólo para acceder al elemento que no se encuentra contiguo en memoria (fallo de cache) lo cual provoca retrasos en la ejecución del programa.

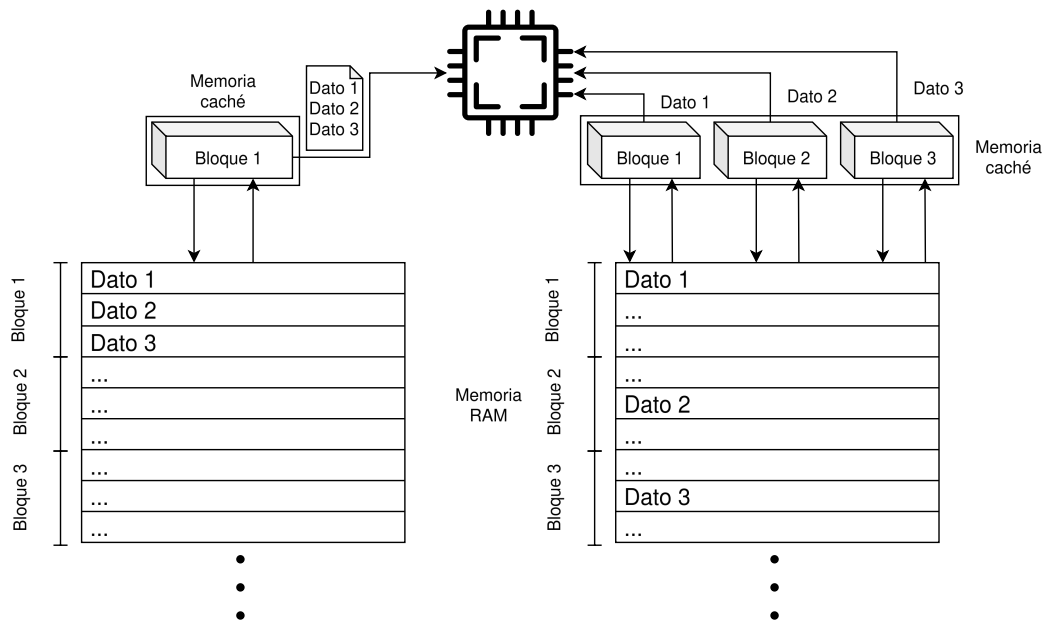


Figura 2.6: Diagrama de flujo de acceso a memoria. A la izquierda se representa un acceso a memoria con datos alineados, permitiendo acceder a todos ellos simultáneamente. A la derecha se representa lo opuesto, teniendo que acceder la caché a la RAM múltiples veces para cargar varios bloques, ralentizando el acceso.

Para evitar este problema con datos contiguos, todas las matrices 2D y tensores 3D se han diseñado en C como arrays unidimensionales contiguos (aplanados). Para acceder a los elementos de estas estructuras se debe usar aritmética de punteros, precisando de realizar una serie de cálculos en relación a las dimensiones de los arrays.

Gestión de la memoria

Debido a la naturaleza de bajo nivel de C, la gestión de memoria de los programas en este lenguaje es completamente manual. Esto quiere decir que, por ejemplo, para usar una sección de la memoria dinámica, se debe reservar manualmente la misma con la cantidad exacta que se prevé usar y, de la misma forma, cuando se deje de usar esa memoria reservada, se debe liberar manualmente para que pueda ser usada nuevamente. Esto suele dar lugar a numerosos problemas, ya que es muy común no liberar las secciones reservadas de la memoria de manera no intencionada, dando lugar así al problema conocido como fuga de memoria (o Memory Leak); o no asignar correctamente la memoria necesaria para un conjunto de datos y que se termine usando direcciones no definidas que no contengan información, generándose un error de segmentación (o Segmentation Fault).

Para evitar estos problemas, como se muestra en el código 2.2, se han definido funciones de inicializado y liberado de memoria de diferentes estructuras, específicamente: estructura de datos de simulador, estructura de datos de mapa, estructura de datos de viento, lista de estructura de datos de tensores (2D y 3D) y matrices aplanadas. Además, se inicializa la memoria a 0 mediante "calloc", de

forma que no se lea información indeseada de la memoria.

```
1 //Funcion que inicializa familia de tensores 3D
2 Tensor* init_T_3D(SimulationData* sim_data, MapData* map_data, int*
   cartypes, char** pollutants) {
3
4     int num_cartypes = sim_data->num_cartypes;
5     int num_pollutants = sim_data->num_pollutants;
6     int rows = map_data -> rows + 2;
7     int cols = map_data -> cols + 2;
8     int times = map_data -> times + 1;
9
10    int index = 0;
11    Tensor* T = (Tensor*) calloc((num_cartypes * num_pollutants), sizeof(
Tensor));
12    for (int ct = 0; ct < num_cartypes; ct++) {
13        for (int p = 0; p < num_pollutants; p++) {
14            //Como cartypes es un array de enteros, se convierte a string
para formar el nombre del tensor
15            char car_char[30];
16            snprintf(car_char, sizeof(car_char), "%d", cartypes[ct]);
17
18            int name_size = strlen(car_char) + strlen(pollutants[p]) + 2;
19            T[index].name = malloc(name_size * sizeof(char));
20            //Union de strings para formar el nombre del tensor
21            snprintf(T[index].name, name_size, "%s_%s", car_char,
pollutants[p]);
22
23            T[index].data = (float*) calloc((size_t) rows * cols * times,
sizeof(float));
24            index++;
25        }
26    }
27    return T;
28 }
29
30 //Funcion que inicializa familia de tensores 2D
31 Tensor* init_T_2D(SimulationData* sim_data, MapData* map_data, int*
   cartypes, char** pollutants) {
32     int num_cartypes = sim_data -> num_cartypes;
33     int num_pollutants = sim_data -> num_pollutants;
34     int rows = map_data -> rows + 2;
35     int cols = map_data -> cols + 2;
36
37     int index = 0;
38     Tensor* T = (Tensor*) calloc((num_cartypes * num_pollutants), sizeof(
Tensor));
39     for (int ct = 0; ct < num_cartypes; ct++) {
```

```

40     for (int p = 0; p < num_pollutants; p++) {
41         //Como cartypes es un array de enteros, se convierte a string
para formar el nombre del tensor
42         char car_char[30];
43         snprintf(car_char, sizeof(car_char), "%d", cartypes[ct]);
44
45         int name_size = strlen(car_char) + strlen(pollutants[p]) + 2;
46         T[index].name = malloc(name_size * sizeof(char));
47         //Union de strings para formar el nombre del tensor
48         snprintf(T[index].name, name_size, "%s_%s", car_char,
pollutants[p]);
49
50         T[index].data = (float*) calloc((size_t) rows * cols, sizeof(
float));
51         index++;
52     }
53 }
54 return T;
55 }
56
57 //Funcion que libera familia de tensores de cualquier tipo
58 void free_T(Tensor* T, SimulationData* sim_data) {
59     int num_cartypes = sim_data->num_cartypes;
60     int num_pollutants = sim_data->num_pollutants;
61     for (int i = 0; i < num_cartypes * num_pollutants; i++) {
62         free(T[i].name);
63         free(T[i].data);
64     }
65     free(T);
66 }

```

Código 2.2: Funciones de inicialización y liberación de memoria para familias de tensores 2D y 3D.

2.3.2. Código de simulación

Siguiendo el esquema lógico del simulador, el código se ha segmentado en 4 funcionalidades, las cuales se corresponden de forma muy similar a las funciones descritas en la sección de funciones del simulador, sólo que con las estructuras de datos descritas para el código en C:

- **Función de matriz de viento:** recibe la estructura de datos del simulador y la estructura de datos de mapa; y retorna una estructura de datos del viento.
- **Función de matriz de difusión:** recibe la estructura de datos del simulador y la estructura de datos del mapa; y retorna una matriz aplanada con los datos de difusión.
- **Función de generación de último estado de la simulación:** recibe la estructura

de datos del simulador, la estructura de datos del mapa, los datos de polución de entrada en forma de lista de estructura de datos de tensores 3D, la lista de tipos de vehículos y la lista de contaminantes; y retorna el resultado final de la simulación en forma de lista de estructura de datos de tensores 2D.

- **Función de generación de estado completo de la simulación:** recibe la estructura de datos del simulador, la estructura de datos del mapa, los datos de polución de entrada en forma de lista de estructura de datos de tensores 3D, la lista de tipos de vehículos y la lista de contaminantes; y retorna el resultado completo y temporal de la simulación en forma de lista de estructura de datos de tensores 3D.

2.4. Diseño de puente entre C y Python

Dado que el código se ha planteado para ser usado desde Python, es necesario que C se comunique de alguna forma con este, para que así se puedan realizar las operaciones de alto rendimiento en C y, posteriormente, se puedan obtener los resultados en el entorno de Python, de forma como se muestra en el diagrama 2.4. Sin embargo, no hay una forma trivial de realizar tal comunicación minimizando la latencia por transferencia, por lo que se ha de determinar cuál es la mejor opción entre las distintas posibilidades que se encuentran disponibles.

Principalmente, se han valorado las siguientes opciones: ctypes, API nativa de CPython, cffi y Cython.

2.4.1. ctypes

La biblioteca ctypes es una interfaz de funciones foráneas (es decir, funciones externas al lenguaje) que permite llamar a funciones y tipos de datos de C.

ctypes es una biblioteca de Python estándar la cual proporciona tipos compatibles con C y permite llamar a funciones de bibliotecas compartidas (.so para el caso de Linux). Al ser parte de la biblioteca estándar, su principal ventaja es la portabilidad y la ausencia de una etapa de compilación adicional [25].

De hecho, inicialmente, cuando el trabajo inició como un proyecto de alumno interno, fue la opción seleccionada para este trabajo dada su facilidad de vinculación con Python, ya que no requería nada más que el código compilado de C, por lo que inicialmente parecía ser una alternativa simple en cuanto a su implementación. Sin embargo, conforme se fue profundizando, se vieron varios problemas respecto a su uso:

- **Complejidad sintáctica:** la complejidad del código con ctypes se disparó al momento de empezar a usar punteros y direcciones de memoria, dando lugar a una sintaxis muy densa y poco intuitiva que dificultaba el desarrollo del código.
- **Complejidad de depuración de errores:** el intérprete de Python no da detalles de los errores causados por ctypes, por lo que la depuración de problemas se vuelve especialmente compleja en cuanto el código se refina.

- **Transferencia de datos:** este es el problema más grave y el que descartó su uso pese a tener gran parte de su implementación completada. Al necesitar realizar una conversión de tipo en la transferencia de datos entre C y Python, se genera un cuello de botella que, no sólo afecta al rendimiento, sino que se maneja de manera ineficiente la memoria, causando así muchas veces un uso excesivo de la misma. Si bien esto no es algo estrictamente causado por ctypes, este no facilita una manera cómoda de manejar el problema si los elementos no están alineados en memoria.

2.4.2. API nativa de CPython

El uso de la API de CPython implica escribir código en C que interactúa directamente con las estructuras internas del intérprete de Python. Es el método que ofrece el control más absoluto sobre el comportamiento del programa y la gestión de recursos [26].

Pese a que sea la manera más eficiente de crear una conexión entre C y Python, este método cuenta con varios problemas. No solo es que sea muy complejo en su uso, haciendo que en la mayoría de casos sea mejor el código compilado automáticamente por herramientas como cffi o Cython que el escrito manual (si no se conoce en profundidad su funcionamiento); sino que su uso supone manejar manualmente las referencias de los objetos de Python (más específicamente, se deben gestionar los conteos de referencia, más adelante se profundizará en el tema), lo cual suele dar lugar a numerosos problemas de fugas de memoria.

2.4.3. cffi

cffi (de las siglas "C Foreign Function Interface") se presenta como una alternativa más actual a ctypes. Consiste en una biblioteca externa que permite interactuar por medio de una API con los elementos de C desde Python mediante declaraciones de cabeceras estándar. Su principal enfoque es poder separar claramente la lógica correspondiente a C de la de Python [27].

Su principal ventaja respecto a ctypes es que cffi dispone de un modo llamado "API", el cual compila un módulo en C cuya función es la validación de tipos en tiempo de compilación, reduciendo así la sobrecarga de transferencia de datos entre lenguajes. Sin embargo, pese a arreglar el principal problema de ctypes, fue finalmente descartado debido a que no tiene capacidad de acelerar código más cercano a Python por su enfoque de lenguajes separados, cosa que en el siguiente candidato es cubierto satisfactoriamente.

2.4.4. Cython

Cython se trata de un lenguaje de programación superconjunto de Python (es decir, que todo código válido en Python también lo es en Cython), el cual añade soporte a características de C y C++ como el tipado estático y las llamadas a funciones nativas [28]. Esto da lugar a que se pueda compilar el código Cython en un código C con comunicación con Python altamente eficiente.

Finalmente se ha seleccionado Cython como la solución definitiva por su simplicidad al momento de ser implementado en un proyecto y, sobretodo, por su capacidad de realizar mapeado de memoria manual con NumPy. Esto permite que tanto C como Python operen sobre la mismas direcciones de memoria física, eliminando la necesidad de realizar copias de los elementos traspasados de un lenguaje a otro. Esto se verá más adelante que supone un punto clave en la optimización de los accesos a memoria y la carga espacial de la memoria.

Funcionamiento

Tal y como se detalló anteriormente, el código Cython es tratado sintácticamente de manera similar a Python, con código simple, tabulado estricto, etcétera. La principal diferencia viene al momento de usar el código, ya que realmente este se compila a código C para su uso, el cual sirve de intermediario entre C y Python al manejar la API de CPython (es decir, la API nativa) de manera optimizada. Una vez se tiene el código en C, se puede unir al resto de código del programa en el mismo lenguaje en formato .so, de forma que se pueda llamar y usar desde Python como librería compartida.

Ventajas

Además de la ventaja principal descrita al inicio, Cython cuenta con una serie de ventajas más que se describen a continuación:

- **Código Python optimizado:** algunas partes del código de Python pueden ser optimizadas, por ejemplo, mediante los tipados estáticos de Cython y su compenetración con NumPy, el cual acelera notoriamente el código correspondiente.
- **Módulo de intercomunicación unificado:** toda la capa de comunicación se puede simplificar en un único código independiente a los otros lenguajes, permitiendo de esta manera una clara separación entre los códigos de C, Python y la capa de comunicación (es decir, Cython).
- **Conversiones de tipos:** para tipos simples, Cython realiza conversión de tipos directamente mediante la API nativa de CPython, llevándose a cabo de manera eficiente. Sin embargo, para tipos más complejos o elementos de NumPy, Cython realmente no realiza conversiones de tipo, sino que transfiere las direcciones de memoria haciendo que ambos lenguajes compartan la misma sección de memoria. Eso sí, es importante aclarar que en ciertas situaciones se deben hacer los traspasos de memoria manualmente en Cython en caso de ser muy complejos, y en casos extremos (donde se busque una optimización mayor que la proporcionada por el lenguaje, como se verá en la implementación), se deberá tratar con el conteo de referencias de Python en caso de querer minimizar el uso de memoria o maximizar la velocidad de sincronización de gestión de memoria entre lenguajes. A pesar de esto, por los demás puntos que tiene Cython a favor, sigue siendo la mejor alternativa de todas las planteadas.

Tecnología	Rendimiento	Curva aprendizaje	Eficiencia memoria	Principal desventaja
ctypes	Bajo	Baja	Alta (<i>overhead</i> grave)	Sintaxis densa con punteros y nula optimización de arrays.
API CPython	Muy alto	Muy Alta	Muy alta (Control Manual Total)	Gestión de refcounts propensa a memory leaks.
cffi	Medio	Media	Media	No permite optimizar código Python.
Cython	Muy alto	Media	Muy alta (Punteros compartidos)	Requiere compilación explícita (archivos .pyx).

Tabla 2.3: Comparativa de tecnologías puente entre Python y C.

3. Implementación del programa

En esta sección se detallará con profundidad la implementación de las estructuras de datos, las optimizaciones aplicadas, el código de simulación y la comunicación interlenguaje. Todo el código introducido respecto a la implementación del simulador puede ser encontrado en "https://github.com/nical1706/sim_propagacion_contaminantes_optimizado".

3.1. Arquitectura y gestión de la memoria en CPU

Como se mencionó anteriormente, la gestión de los datos de la simulación es una parte crucial del programa, dada la gran cantidad de información manejada durante la ejecución del mismo. Esto da lugar a que los cuellos de botella generados por la memoria y el ancho de banda de la misma se conviertan en una prioridad en cuanto a la optimización del programa para obtener el rendimiento deseado. Por tanto, se debe plantear cuál es la mejor manera de almacenar estos datos en memoria para su uso en el código de C.

3.1.1. Arquitectura inicial

En versiones iniciales del programa, esto fue abordado con una estructura que almacenaba multipunteros (es decir, un elemento de punteros que apuntaban a otros punteros). Se eligió ese enfoque debido a que fue el acercamiento claro en ese momento, al poderse abstraer de manera más directa como un conjunto etiquetado de tensores tridimensionales, justo lo que recibe de entrada el programa en Python. Además, la sintaxis de manejo de los distintos punteros resultaba bastante clara al momento de programar con ella, dado que funcionaba mediante la indexación de elementos de manera numérica, pudiendo acceder a un elemento cualquiera con una serie de índices correspondientes al mismo (funcionando de manera similar a Python).

Sin embargo, al realizar pruebas con este enfoque se vieron varios problemas que hicieron impráctica esta implementación, los cuales se explican a continuación.

Fallos de caché

Para entender los problemas asociados a la caché, se debe detallar primero el funcionamiento de la misma. Partiendo de lo que se muestra en el diagrama 3.1, la memoria caché de una CPU se divide comúnmente en tres niveles: L1, L2 y L3. Estas se encuentran ordenadas de menor a mayor latencia de transmisión y también de menor a mayor capacidad de almacenamiento, de forma que la L1 es la más rápida pero con menor almacenamiento y la L3 su opuesta. Cuando la CPU requiere un dato, este se busca de forma escalonada en los niveles existentes de la caché, realizando primero una búsqueda en la L1, y si el dato no se encuentra en el nivel se

produce un fallo de caché (o "caché miss"), provocando que se inicie nuevamente la búsqueda en la L2, luego la L3 y finalmente, si el dato no está en caché, se realiza la búsqueda en RAM, la cual induce una penalización grave en el rendimiento debido a su latencia. Debido a esto, es de suma importancia realizar una gestión óptima del uso de memoria en el programa.

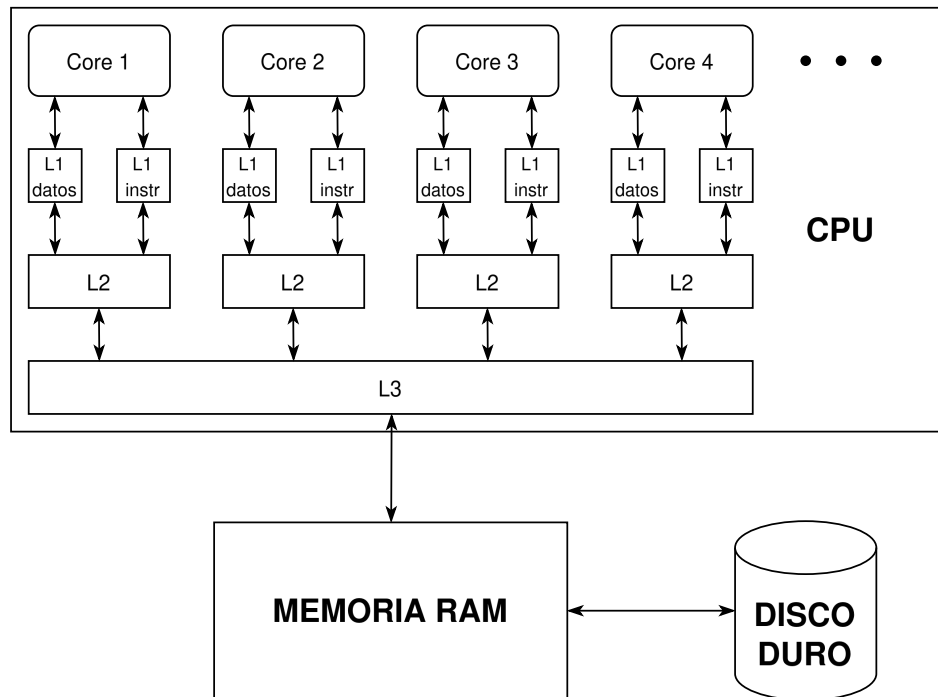


Figura 3.1: Diagrama de funcionamiento de memoria. Se aprecia que cada núcleo de la CPU tiene su memoria L1 y L2, además que la caché L1 suele estar segmentada en datos e instrucciones. También se aprecia que L3 es compartida por todos los núcleos.

Como se ilustró en la imagen 2.6, al momento de cargar los datos en la caché de la CPU, estos no se almacenan de uno en uno, debido a que se generarían enormes cuellos de botella por la velocidad de transferencia. En su lugar, los datos se cargan de forma simultánea en forma de bloques contiguos denominados "batches" (generalmente con un tamaño de 64 bytes). De esta manera, se minimiza el impacto del retardo causado por la transmisión de los datos desde la RAM hasta la caché si se transmiten datos relacionados o de acceso contiguo en un mismo bloque.

Para aprovechar aún más los bloques de transmisión, los procesadores modernos cuentan con un componente hardware denominado "Prefetcher". El "Prefetcher" analiza en tiempo de ejecución los patrones de acceso a memoria, de manera que si detecta que un programa lee datos de forma secuencial y predecible, solicitará de manera anticipada el siguiente batch a la memoria antes de que la CPU lo demande de forma explícita. Sin embargo, para que el "Prefetcher" sea efectivo, los datos relacionados deben estar almacenados de manera contigua o alineada en la memoria.

Sin embargo, este planteamiento implica que los datos a almacenar se deben ordenar de forma lógica y eficiente, ya que en caso de ordenar los datos de forma

desalineada, se generarían fallos de acceso a caché al no estar el elemento solicitado en la misma, obligándolo a bloquearse y generando así un efecto cascada de instrucciones pendientes por dependencia de datos.

Para poner el problema en perspectiva de manera más práctica respecto al enfoque que se propuso al inicio, se propone la siguiente comparación de enfoques de gestión de datos junto a sus diagramas 3.2 y 3.3:

- Supongamos que se tiene un conjunto de datos almacenados en una estructura matricial (es decir, un array multidimensional de punteros o multipuntero). Tal y como funcionan los punteros en C, estos consisten en una variable (es decir una sección de la memoria) que almacena la dirección de otra sección. Estas direcciones de memoria, ajenas a la ubicación del puntero en sí, se asignan por medio de funciones como "malloc". Como tal, la función sí que reserva memoria de forma contigua, pero para implementar este tipo de estructuras se necesitan de múltiples llamadas a la función. Es aquí donde surge el problema, ya que cada vez que se ejecuta, el bloque completo es asignado mediante el gestor de memoria dinámica (también referenciado como allocator) del sistema operativo. El allocator asigna la dirección en función de algoritmos de búsqueda sobre una estructura de datos que rastrea los bloques libres en el "heap", lo cual no garantiza la contigüidad espacial predecible para programas que realizan múltiples asignaciones aisladas [29]. En el caso de un puntero doble, el primer puntero almacena la dirección del segundo puntero, el cual a su vez almacena una segunda dirección, la cuál se corresponde con una dirección de memoria final. Con esto explicado, se puede ver que, pese a que un conjunto de punteros estén alineados en memoria, la realidad es que los espacios de la memoria a los que apuntan no lo están, dando lugar a una memoria desalineada. Sirva de referencia la imagen 3.2.

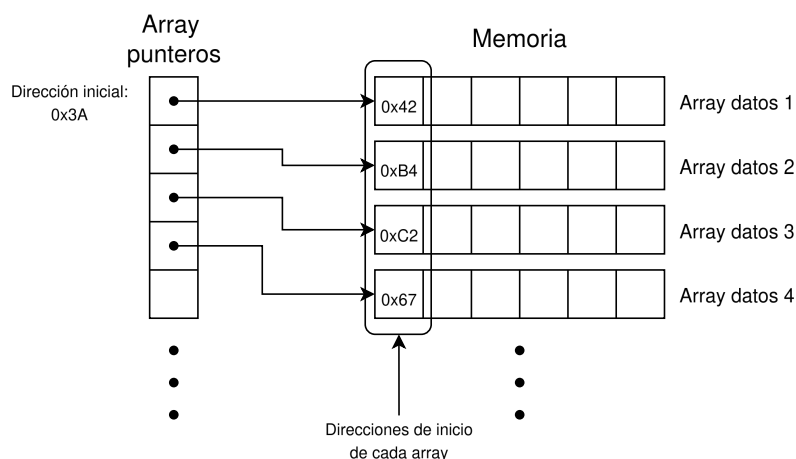


Figura 3.2: Diagrama de asignación de memoria en array de punteros, se ve que los punteros no apuntan a direcciones contiguas

- Por otra parte, los arrays unidimensionales en C no son más que un elemento puntero con elementos ordenados en memoria. Como se explicó en la parte de diseño, estos pueden ser usados para almacenar datos de múltiples

dimensiones por medio de una aritmética de punteros establecida. En la práctica, esto implica que, si se elige la aritmética de acceso adecuada, se estaría ordenando eficientemente los datos en memoria, pudiendo aprovechar el tamaño de los bloques de transmisión de datos y reduciendo así los fallos de caché. Sirva de referencia la imagen 3.3.

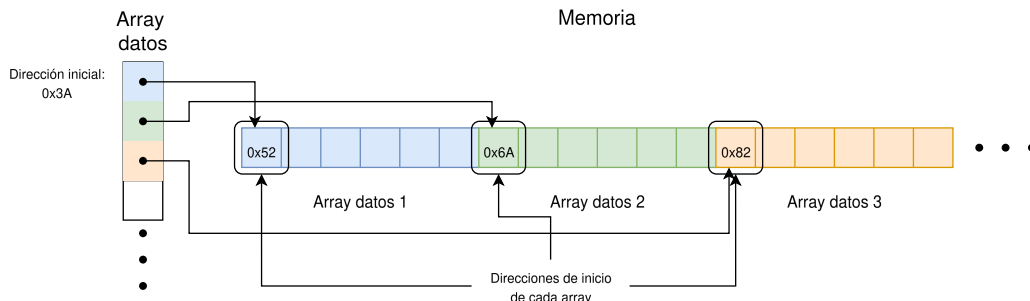


Figura 3.3: Diagrama de asignación de memoria en array aplanado, donde se encadena cada array de manera contigua

De esta manera, se puede comprender el problema referente al enfoque inicialmente planteado, ya que este no sólo rompe la simetría una vez, sino que lo hace una segunda vez, al añadir un puntero más correspondiente al tiempo del dato. Esto hace que la memoria esté completamente desalineada, causando que se den múltiples fallos de caché al momento de acceder a los datos de manera continua, perjudicando así al rendimiento del simulador.

Desreferenciación

El acceso a los elementos almacenados en estructuras multipuntero son en sí mismo otro cuello de botella para el programa, ya que la obtención de las direcciones debe hacerse de forma secuencial al orden de las direcciones almacenadas en los subpunteros. Esto da lugar al caso conocido como encadenamiento de punteros (pointer chasing) al momento de desreferenciar un elemento que sólo es accesible por punteros, reduciendo por tanto el rendimiento del programa.

Pointer aliasing

Cuando se trabajan con datos almacenados mediante punteros, el compilador no puede asegurar si dos de estos no almacenan información de la misma dirección de memoria. Este escenario es delicado, ya que puede dar pie a que, cuando una función reciba múltiples punteros como argumentos, el compilador no pueda garantizar que dos punteros distintos no estén apuntando a la misma dirección de la memoria o a una superpuesta. Esto se hace así porque, si el compilador asumiese que las memorias son independientes y aplicase técnicas como el desenrollado de bucles o la vectorización SIMD (por aclarar, SIMD, de procesamiento de múltiples datos con una sola instrucción, son un tipo de instrucciones de CPU que permiten realizar una operación con múltiples datos a la vez, mejorando así el rendimiento del sistema, con bloques de datos de 256 bits para AVX2), una escritura en el

primer puntero podría sobrescribir accidentalmente los datos que el segundo puntero iba a leer, resultando en una solución del simulador incorrecta.

A raíz de esto, el compilador desactiva este tipo de optimizaciones, reduciendo la capacidad de optimizar el código de manera autónoma. En el caso específico de los elementos multipunteros, los elementos de diferentes filas (o columnas o similar) se tratan de la forma anteriormente mencionada, perdiendo así posibles optimizaciones que el compilador podría aplicar.

3.1.2. Arquitectura final

Debido a los problemas encontrados en la arquitectura anterior, se decidió cambiar el enfoque de cómo se manejaban los datos a uno que priorizase el rendimiento del simulador. Para ello, se decidió aplicar la técnica ya mencionada de aplanamiento de datos (descrita también como "Flattening").

Implementación de aritmética de punteros

Para usar en el simulador las estructuras aplanadas, se ha de definir la aritmética de acceso a los datos de las mismas. En este caso, se ha hecho uso del estándar conocida como ordenamiento principal por filas (Row-major order). De esta forma, para acceder a unas coordenadas (x, y) en una matriz de dimensiones $W \times H$, la operación matemática de acceso queda definida como:

$$\text{Índice}_{2D}(x, y) = y \times W + x$$

Para los tensores tridimensionales de la simulación, compuestos por (x, y, t) y dimensiones (W, H, T) , la operación se expande para preservar la contigüidad espacial de las matrices en cada salto de tiempo:

$$\text{Índice}_{3D}(x, y, t) = t \times (W \times H) + y \times W + x$$

Con el objetivo de simplificar el código correspondiente de esta metodología, se han implementado una serie de etiquetas que abstraen el cálculo denominadas "macros". Las macros son secciones de códigos definidas mediante la directiva "#define", las cuales asignan un identificador al código para que, cuando aparezca tal identificador, se sustituya por el código asociado al mismo antes de compilar el programa, permitiendo así una implementación optimizada en la compilación y eliminando en consecuencia el overhead de una función convencional[30]. De esta forma, se pueden definir estructuras de código simple que afectan en menor medida al tiempo de ejecución que una función cualquiera. En el caso del código 3.1, se han implementado 2 macros: acceso a datos de una matriz 2D (IDX_2D) y acceso a datos de una matriz 3D (IDX_3D).

```

1 //Macro para acceder a un elemento de una matriz 2D
2 //Se usa (size_t) para forzar 64 bits, evitando desbordamientos
3 #define IDX_2D(a, b, cols) ((size_t) a * cols + (size_t) b)
4
```

```

5 //Macro para acceder a un elemento de un tensor 3D
6 #define IDX_3D(a, b, c, dim2, dim3) ((size_t) a * dim2 * dim3 + (size_t)
   b * dim3 + (size_t) c)

```

Código 3.1: Macro de acceso a matrices y tensores aplanados.

Cabe resaltar que esta implementación de las macros no reduce el problema derivado de tener que realizar cálculos matemáticos para cada elemento que se quiera obtener (por ejemplo, la multiplicación suele tener un tiempo de ejecución que ronda los 3 o 4 ciclos de reloj en procesadores modernos con arquitectura x86 [31]), teniendo así que tratar de minimizar el uso de las macros de acceso y buscar formas óptimas de obtener los elementos deseados.

Evasión del pointer aliasing

Los problemas asociados al pointer aliasing no se solventan con este enfoque (aunque sí que se reducen considerablemente), dado que el hecho de usar punteros en C ya implica que existe la posibilidad de que se dé esta situación. Para erradicarlo por completo, durante el código se verá continuamente el uso de la etiqueta "restrict" (ejemplo de uso en código 3.2). Esta tiene como función contextualizar al compilador para especificarle que la variable tipo puntero que contenga tal etiqueta tiene la exclusividad de la sección de la memoria a la que esté apuntando [32]. Entre otras cosas, esto permite al compilador realizar paralelización a nivel de datos mediante instrucciones SIMD, de forma que se pueda optimizar en mayor medida el código. Cabe destacar que, si no se hace un uso adecuado de la etiqueta, podría dar lugar a errores bastante graves, dado que el compilador asume la exclusividad de la dirección.

```

1 //Uso de restrict para indicar que no hay aliasing con otras variables,
   lo que permite optimizaciones por parte del compilador
2 float* restrict acc = map_data -> acc;

```

Código 3.2: Ejemplo de uso de etiqueta "restrict"

AoS y SoA

Al momento de implementar la estructura "WindComponents", se abrió el debate de si debía de ser una estructura de tipo AoS (Array of Structures) o de SoA (Structure of Arrays).

- **AoS:** una estructura AoS almacena elementos unitarios, de forma que se concatenarían mediante un array de las estructuras correspondientes. En el caso de WindComponents, por ejemplo, almacenaría valores tipo float planos (es decir, sin punteros), y posteriormente se formaría un array de WindComponents de longitud width * height, cubriendo así la totalidad de las celdas en todas las componentes. Esta estructura está pensada para cuando se precisan todos los elementos de la estructura de manera simultánea. Sin embargo, este no es el caso del código del simulador, dado que si se observa el código 3.3, donde se aplica el movimiento del viento, se

aprecia que las posiciones usadas para acceder a cada array de cada componente del viento difieren entre sí. Lo cual implica que se tendrían que cargar varios elementos WindComponents en caso de ser AoS; y como los datos de una misma componente no estarían alineados se generarían fallos de caché y se ralentizaría el programa.

```

1 float transport = N[id + 1] * P_diff[id + 1] +
2   S[id - 1] * P_diff[id - 1] +
3   E[id - cols_l] * P_diff[id - cols_l] +
4   W[id + cols_l] * P_diff[id + cols_l] +
5   NE[id - cols_l + 1] * P_diff[id - cols_l + 1] +
6   NW[id + cols_l + 1] * P_diff[id + cols_l + 1] +
7   SE[id - cols_l - 1] * P_diff[id - cols_l - 1] +
8   SW[id + cols_l - 1] * P_diff[id + cols_l - 1] +
9   stays[id] * P_diff[id];
10
11 P_next[id] = (m_gamma) * acc[id] * (transport + G_slice[id]);
12

```

Código 3.3: Extracto de código de uso de componentes del viento, donde se aprecia que cada una tiene un índice distinto

- **SoA:** una estructura SoA almacena arrays contiguos en memoria del mismo tipo, almacenando todo en una única estructura con array para cada variable. En el caso del simulador, este es el mejor enfoque, ya que no sólo soluciona el problema descrito en AoS, sino que se pueden aprovechar de forma eficiente las instrucciones SIMD. En el caso de AVX2 (instrucciones vectoriales disponibles en el clúster), estas permiten hasta 256 bits de información, lo cual implica que se podrían cargar hasta 8 elementos de una misma componente a la vez sin tener que acceder múltiples veces a memoria (se adjunta mapeo de registros SIMD como referencia en 3.4).

Mapeo de Registro i16 (Formato Q15 con Guarda)

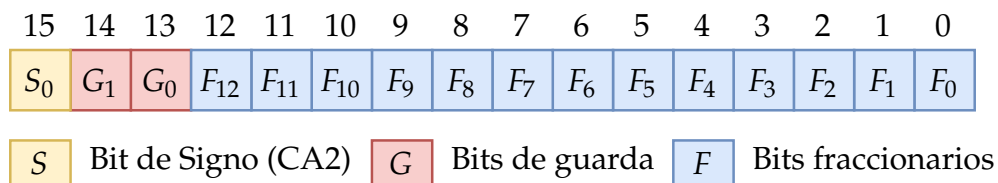


Figura 3.4: Mapeo de registro SIMD. Obtenida de [3].

Debido a lo anteriormente mencionado, se ha decidido implementar "WindComponents" como una estructura SoA.

Cambio en el orden de las dimensiones de los datos

Un cambio clave que se ha realizado respecto a la gestión de los datos en comparación con la implementación original en Python, es la reasignación y

modificación del orden de las dimensiones los datos y de sus correspondientes accesos.

Específicamente, se ha cambiado en el orden de acceso a los elementos espacio-temporales de la simulación, pasando de estar ordenados como (width, height, times) a estar ordenados como (times, width, height). Esto se justifica por cómo se acceden a los datos en el simulador, viéndose en el código 3.4

```
1 for (int t = 0; t < times; t++) {
2     for (int k = 0; k < num_tensors; k++) {
3         (...)
4         //Bucle difusion
5         for (int i = 1; i <= rows; i++) {
6             for (int j = 1; j <= cols; j++) {
7                 size_t id = IDX_2D(i, j, cols_l);
8                 (...)
9             }
10        }
11        //Bucle viento
12        for (int i = 1; i <= rows; i++) {
13            for (int j = 1; j <= cols; j++) {
14                size_t id = IDX_2D(i, j, cols_l);
15                (...)
16            }
17        }
18    }
19 }
```

Código 3.4: Estructura de bucles de implementación (rows=width, cols=height)

Al acceder a la variable "times" al inicio, si esta se encuentra como la última dimensión de los datos, resultará contraproducente en cuanto a la eficiencia del uso de acceso a memoria, ya que implicaría que los datos se encontraría alineada en memoria en función de "times" pese a ser una dimensión que se accede de forma mucho menos frecuente que las otras dos, las cuales, debido a que los datos estarían ordenados según el tiempo, no lo estarían en relación a "width" y "height". Esto ocasionaría que, constantemente, se accediesen a elementos lógicamente continuo, los cuales están desalineados en memoria, causando así múltiples fallos de caché.

En cambio, si estos datos se ordenasen en función de "width" y "height", se realizaría un uso más óptimo de la memoria, reduciendo en su mayoría los fallos de acceso a memoria.

Cabe mencionar que el simulador de coches retorna los datos del diccionario de contaminantes de entrada con la estructura inicial, es decir, que no concuerda con la que usa el simulador del proyecto. Para solventar esto, se ha desarrollado un código auxiliar y temporal que realiza el cambio de dimensiones, requiriéndose en un futuro implementar el cambio directamente en el simulador que genera los datos iniciales (en la sección de Cython se darán más detalles de esta función).

Implementación de estructuras de datos

En lo que se refiere a la gestión de los datos, sólo quedaría detallar la implementación de las estructuras usadas a lo largo del programa:

- **SimulationData y MapData:** como se mencionó, estas dos estructuras almacenan diversos parámetros usados concurrentemente en el programa. Resalta del código 3.5 el aplanamiento aplicado a la variable "acc", la cual se aprecia que consiste en un puntero unitario (si fuera multipuntero y, por ende, 2D, se escribiría como "float** acc"). En el código del simulador (3.5), estas estructuras sólo se pedirán una vez para, posteriormente, extraer los valores almacenado en variables definidas y usar los datos desde ahí. Esto se hace con el objetivo de reducir el overhead ocasionado al momento de acceder a los datos almacenados en una estructura.

```
1 //Estructura para almacenar datos de simulacion, como numero de
   tipos de vehiculos, numero de contaminantes y parametros fisicos
2 typedef struct {
3     int num_cartypes;
4     int num_pollutants;
5     float gamma;
6     float delta;
7     float corner_factor;
8     float WN;
9     float WE;
10 } SimulationData;
11
12 //Estructura que almacena el tamano temporal y espacial de la
   simulacion y la matriz de accesibilidad
13 typedef struct {
14     int rows;
15     int cols;
16     int times;
17     float* acc;
18 } MapData;
19
```

Código 3.5: Código de estructuras "SimulationData" y "MapData"

- **WindComponents:** de código 3.6 destaca por implementar una estructura tipo SoA y por tener aplanadas todas las matrices de todas las componentes. Las variables de la estructura, al igual que las anteriores, se almacenan en variables locales para reducir el overhead de acceso a los datos.

```
1 //Estructura para almacenar las 9 componentes del viento
2 typedef struct {
3     float* N;
4     float* S;
5     float* E;
6     float* W;
```

```

7   float* NE;
8   float* NW;
9   float* SE;
10  float* SW;
11  float* stays;
12 } WindComponents;
13

```

Código 3.6: Código de estructura "WindComponents"

- **Tensor:** estructura que se usará tanto para el tensor de emisiones de entrada como para el de salida de tiempo completo en forma de array de estas mismas estructuras. Si bien la implementación 3.7 puede contrastar con lo explicado respecto a la alineación en memoria, dado que se trataría en consecuencia de un puntero doble (el de tensores y el de datos); debido a gran cantidad de datos almacenados en cada tensor, los posibles cuellos de botella causados por ello resultan irrelevantes en la práctica al afectar sólo en el cambio de referencia de tensores.

```

1 //Estructura para representar un tensor generico, la cual esta
   compuesta por un identificador y la informacion correspondiente
2 typedef struct {
3     char* name;
4     float* data;
5 } Tensor;
6

```

Código 3.7: Código de estructura "Tensor"

3.2. Implementación de código C

A continuación, se procede a exponer las funciones implementadas con todas las optimizaciones aplicadas a nivel de código, junto con su justificación y explicación del funcionamiento. Se abordarán las funciones específicas del funcionamiento del simulador, ya que las funciones auxiliares tienen como único objetivo la correcta reserva y liberación de los recursos de la memoria, por lo que no se han realizado optimizaciones específicas salvo las explicadas anteriormente respecto al aplanamiento de los datos.

3.2.1. Función de viento (get_wind)

Inicializado de función y variables

Inicialmente se recibe la estructura "SimulationData", de datos de la simulación (como número de tipos de vehículos, número de contaminantes y parámetros físicos); y la estructura "MapData", de datos del mapa (la cual contiene el tamaño temporal y espacial de la simulación y la matriz de accesibilidad). Al finalizar la función, se retornará "wind", un puntero a un tipo de estructura

personalizada llamada "WindComponents", la cual contiene todas las componentes del viento para cada dirección (norte, sur, este, oeste, noreste, noroeste, sureste y suroeste) junto con la matriz de coeficiente de la contaminación que permanece en la misma casilla. La razón por la que se retorna en forma de puntero es por motivos de optimizar el uso de memoria, ya que si se pasasen todos los datos como resultado en vez de un puntero, rápidamente se podría sobrecargar la memoria y colapsar en medio de la ejecución.

Entrando en la función, en el código 3.8 se ve el uso de la etiqueta "restrict" y el almacenamiento en variables locales de los datos de las estructuras, tal y como se mencionó anteriormente. Tras eso, se inicializa la variable "wind", que es la que contendrá el puntero al resultado; y una serie de variables auxiliares usadas para el cálculo de las componentes del viento como por ejemplo "sign_WN" o "max_m_WE".

```
1 //Funcion que obtiene las componentes del viento para cada celda
2 //Se retorna un puntero por eficiencia, ya que en caso contrario se
  precisaria retornar la estructura completa
3 WindComponents* get_wind(SimulationData* sim_data, MapData* map_data) {
4     int rows = map_data -> rows;
5     int cols = map_data -> cols;
6     //Uso de restrict para indicar que no hay aliasing con otras
  variables, lo que permite optimizaciones por parte del compilador
7     float* restrict acc = map_data -> acc;
8
9     float WN = sim_data -> WN;
10    float WE = sim_data -> WE;
11
12    WindComponents* wind = (WindComponents*) malloc(sizeof(WindComponents
  ));
13
14    //Variables auxiliares para acelerar calculos y legibilidad del
  codigo
15    int rows_l = rows + 2;
16    int cols_l = cols + 2;
17
18    wind -> N = init_matrix(rows_l, cols_l);
19    wind -> S = init_matrix(rows_l, cols_l);
20    wind -> E = init_matrix(rows_l, cols_l);
21    wind -> W = init_matrix(rows_l, cols_l);
22    wind -> NE = init_matrix(rows_l, cols_l);
23    wind -> NW = init_matrix(rows_l, cols_l);
24    wind -> SE = init_matrix(rows_l, cols_l);
25    wind -> SW = init_matrix(rows_l, cols_l);
26    wind -> stays = init_matrix(rows_l, cols_l);
27
28    //Variables locales para evitar aliasing
29    float* restrict w_N = wind -> N;
```

```

30  float* restrict w_S = wind -> S;
31  float* restrict w_E = wind -> E;
32  float* restrict w_W = wind -> W;
33  float* restrict w_NE = wind -> NE;
34  float* restrict w_NW = wind -> NW;
35  float* restrict w_SE = wind -> SE;
36  float* restrict w_SW = wind -> SW;
37  float* restrict w_stays = wind -> stays;
38
39  //Signos de los componentes del viento
40  int sign_WN = (WN > 0) ? 1 : ((WN < 0) ? -1 : 0);
41  int sign_WE = (WE > 0) ? 1 : ((WE < 0) ? -1 : 0);
42
43  float max_WN = (WN > 0.0f) ? WN : 0.0f;
44  float max_m_WN = (WN < 0.0f) ? -WN : 0.0f;
45  float max_WE = (WE > 0.0f) ? WE : 0.0f;
46  float max_m_WE = (WE < 0.0f) ? -WE : 0.0f;
47
48  (Bucle funcion)
49 }

```

Código 3.8: Código de inicio de función "get_wind"

Bucle de función

Entrando en el bucle de recorrido del mapa (código 3.9), se calcula para cada celda su id de acceso correspondiente. Sin embargo, tal y como se habló anteriormente, esto resulta en una carga computacional extra cada vez que se tenga que acceder a una nueva id, lo cual se daría múltiples veces en el código presentado al tener que crear un id para cada vecino de la celda actual. Para solventar esto, se ha aprovechado el hecho de que los datos están aplanados y, por ende, el acceso a los elementos se realiza de manera aritmética; se ha sacado del bucle la multiplicación correspondiente al salto y se ha almacenado en la variable "jmp_WN". De esta forma, las operaciones de acceso a los elementos del array sólo necesitan realizar una serie de sumas para obtener el id equivalente que si se hiciera mediante la macro correspondiente. Eso sí, se debe mencionar que, en general, los compiladores modernos realizan tal operación de forma automática (optimización conocida como "Loop Invariant Code Motion"), pero con el objetivo de asegurar por completo su implementación y facilitar la implementación automática de las instrucciones SIMD, se ha decidido dejar explicitado.

Otro punto que se ha observado ha sido la posibilidad de evitar cálculos innecesarios mediante un condicional. Esto debido a que, cuando la casilla correspondiente que se está calculando equivale a 0 en la matriz de acceso, todos los cálculos se pueden reducir a establecer la componente de la contaminación estática a 1 (en el resto no hay que hacer nada, ya que se han establecido en memoria con el valor 0). Sin embargo, al introducir un salto en esta sección del código, el compilador no puede asegurar la correcta vectorización de las

instrucciones, por lo que se pierde la paralelización introducida por SIMD. Para el proyecto, dado que no se cuentan con datos reales de simulación, se ha decidido mantener el condicional, pero para llevar a cabo una implementación optimizada para una serie de entradas concretas sería conveniente realizar pruebas para establecer si beneficia o perjudica.

```

1 //Calculo de desplazamientos para cada direccion
2 int jmp_WE = sign_WE * cols_l;
3 //Este se deja por claridad de codigo
4 int jmp_WN = sign_WN;
5
6 for (int i = 1; i <= rows; i++) {
7     for (int j = 1; j <= cols; j++) {
8         size_t id = IDX_2D(i, j, cols_l);
9         float acc_ij = acc[id];
10
11         //Solo calcula desplazamientos para celdas con acc > 0
12         if (acc_ij != 0) {
13             //Se calculan los indices aparte en vez de usar la macro,
14             //dado que asi se reducen el
15             //numero de multiplicaciones realizadas en el bucle al
16             //momento de obtener los indices
17             float acc1 = acc[id + jmp_WE];
18             float acc2 = acc[id - jmp_WN];
19
20             float max_N = (acc[id + jmp_WE - 1] > acc1) ? acc[id + jmp_WE
21             - 1] : acc1;
22             float max_S = (acc[id + jmp_WE + 1] > acc1) ? acc[id + jmp_WE
23             + 1] : acc1;
24             float max_E = (acc[id + cols_l - jmp_WN] > acc2) ? acc[id +
25             cols_l - jmp_WN] : acc2;
26             float max_W = (acc[id - cols_l - jmp_WN] > acc2) ? acc[id -
27             cols_l - jmp_WN] : acc2;
28
29             w_N[id] = acc_ij * max_WN * (1.0f - max_N * fabsf(WE)) * acc[
30             id - 1];
31             w_S[id] = acc_ij * max_m_WN * (1.0f - max_S * fabsf(WE)) *
32             acc[id + 1];
33             w_E[id] = acc_ij * max_WE * (1.0f - max_E * fabsf(WN)) * acc[
34             id + cols_l];
35             w_W[id] = acc_ij * max_m_WE * (1.0f - max_W * fabsf(WN)) *
36             acc[id - cols_l];
37
38             w_NE[id] = acc_ij * max_WN * max_WE * acc[id + cols_l - 1];
39             w_NW[id] = acc_ij * max_WN * max_m_WE * acc[id - cols_l - 1];
40             w_SE[id] = acc_ij * max_m_WN * max_WE * acc[id + cols_l + 1];
41             w_SW[id] = acc_ij * max_m_WN * max_m_WE * acc[id - cols_l + 1
42             ];
43         }
44     }
45 }

```

```

32
33         w_stays[id] = 1.0f - (w_N[id] + w_S[id] + w_E[id] + w_W[id] +
34             w_NE[id] + w_NW[id] + w_SE[id] + w_SW[id]);
35     } else {
36         w_stays[id] = 1.0f;
37     }
38 }
39 }
40
41 return wind;

```

Código 3.9: Código de bucle de función "get_wind"

3.2.2. Función de matriz de difusión (get_difMatrix)

Inicializado de función y variables

Como se ve en la función 3.10, la función recibe los mismos elementos que la función anterior, siendo la estructura "SimulationData" y la estructura "MapData". Al realizarse satisfactoriamente la función, se retornará un puntero a una matriz aplanada como un array unidimensional de tipo float.

El inicio de la función se resume en: iniciado de variables con las mismas consideraciones que la función anterior para la evasión del pointer aliasing y la declaración de la variable que se retornará denominada "dif_matrix".

```

1 //Retorna matriz de difusion
2 float* get_difMatrix(SimulationData* sim_data, MapData* map_data) {
3     int rows = map_data -> rows;
4     int cols = map_data -> cols;
5     float* restrict acc = map_data -> acc;
6
7     float delta = sim_data -> delta;
8     float corner_factor = sim_data -> corner_factor;
9
10    //Variables auxiliares para acelerar calculos y legibilidad del
    codigo
11    int rows_l = rows + 2;
12    int cols_l = cols + 2;
13
14    float* restrict dif_matrix = init_matrix(rows_l, cols_l);
15
16    (Bucle funcion)
17 }

```

Código 3.10: Código de inicio de función "get_difMatrix"

Bucle de función

Además de las consideraciones tomadas en la función 3.9, en el código 3.11 se suma la extracción del bucle de un factor estático, el cual se almacena en la variable "f_factor" y reduce los cálculos realizados en el bucle. Por lo demás, se calcula el id correspondiente, se usan sumas simples para acceder a elementos vecinos y se realizan los cálculos correspondientes a la matriz de difusión.

```
1 //Para reducir las divisiones y multiplicaciones realizadas, se calcula
   fuera del bucle la parte fija de la funcion y se anade
2 //a posteriori al calculo de la difusion
3 float f_factor = delta / (4.0f + 4.0f * corner_factor);
4
5 for (int i = 1; i <= rows; i++) {
6     for (int j = 1; j <= cols; j++) {
7         size_t id = IDX_2D(i, j, cols_l);
8         //Se calculan los indices aparte en vez de usar la macro, dado
           que asi se reducen el
9         //numero de multiplicaciones realizadas en el bucle al momento de
           obtener los indices
10
11         //Conteo vecinos cruz accesibles
12         float acc_neig_edge = acc[id - cols_l] + acc[id + cols_l] + acc[
id + 1] + acc[id - 1];
13         //Conteo vecinos diagonales accesibles
14         float acc_neig_corner = acc[id - cols_l - 1] + acc[id - cols_l +
1] + acc[id + cols_l + 1] + acc[id + cols_l - 1];
15
16         dif_matrix[id] = 1.0f - (acc_neig_edge + acc_neig_corner *
corner_factor) * f_factor;
17     }
18 }
19
20 return dif_matrix;
```

Código 3.11: Código de bucle de función "get_difMatrix"

3.2.3. Función de último estado de simulación (get_P_last)

Inicializado de función y variables

En el código 3.12, al llamar la función, esta recibe aparte de las estructuras anteriormente mencionadas, la familia de tensores G, la cual contiene los datos iniciales de contaminación del simulador y son pasados desde Python en forma de diccionario (más adelante se indaga en la transformación de tipos de Python a C); el array de tipos de vehículos y el array de tipos de contaminantes (en C no hay un tipo String nativo, por lo que se asigna un array de caracteres a cada palabra).

Simétricamente a las anteriores funciones expuestas, al inicio se inicializan

variables de datos y auxiliares usadas durante la ejecución del programa, con la diferencia de la inclusión del llamado a las funciones "get.wind", para el inicializado de las componentes del viento; "get_difMatrix", para el inicializado de la matriz de difusión; e "init.T_2D", el cual inicializa una familia de matrices que contendrán el resultado final de la simulación.

```
1 //Retorna familia de tensores 2D que representan el ultimo instante de la
  simulacion
2 //Recibe las estructuras de datos auxiliares, la matriz de acceso, la
  familia de tensores de datos G,
3 //la lista de coches y la lista de contaminantes
4 Tensor* get_P_last(SimulationData* sim_data, MapData* map_data, Tensor* G
  , int* cartypes, char** pollutants) {
5   int rows = map_data -> rows;
6   int cols = map_data -> cols;
7   int times = map_data -> times;
8   float* restrict acc = map_data -> acc;
9
10  float delta = sim_data -> delta;
11  float corner_factor = sim_data -> corner_factor;
12  float gamma = sim_data -> gamma;
13  int num_cartypes = sim_data -> num_cartypes;
14  int num_pollutants = sim_data -> num_pollutants;
15
16  int num_tensors = num_cartypes * num_pollutants;
17
18  //Variables auxiliares para acelerar calculos y legibilidad del
  codigo
19  int rows_l = rows + 2;
20  int cols_l = cols + 2;
21
22  Tensor* P = init_T_2D(sim_data, map_data, cartypes, pollutants);
23
24  WindComponents* wind = get_wind(sim_data, map_data);
25  float* restrict dif_matrix = get_difMatrix(sim_data, map_data);
26
27  //Puesta en variables de datos de viento para reduccion de tiempos de
  acceso a memoria
28  float* restrict N = wind -> N;
29  float* restrict S = wind -> S;
30  float* restrict E = wind -> E;
31  float* restrict W = wind -> W;
32  float* restrict NE = wind -> NE;
33  float* restrict NW = wind -> NW;
34  float* restrict SE = wind -> SE;
35  float* restrict SW = wind -> SW;
36  float* restrict stays = wind -> stays;
37
```

```

38 //Para reducir las divisiones y multiplicaciones realizadas, se
    calcula fuera del bucle la parte fija de la funcion y se anade
39 //a posteriori al calculo de la difusion
40 float f_factor = delta / (4.0f + 4.0f * corner_factor);
41
42 float m_gamma = 1.0f - gamma;
43
44 (Bucle funcion)
45 }

```

Código 3.12: Código de inicio de función "get_P_last"

Bucle de función

Antes de inicializar el bucle, en el código 3.13 declara una variable llamada "P_diff", la cual consiste en una matriz aplanada cuya información contenida será la de la contaminación del estado anterior tras aplicarle la difusión.

Entrando en el bucle de la función, este se encuentra inicialmente formado por el bucle correspondiente al tiempo de la simulación y por el bucle del número de tensores de la simulación, tal y como se mencionó y vio cuando se comentó el cambio de dimensiones. Una vez dentro del doble bucle, se inicializan las variables "P_slice" y "G_slice", los cuales almacenan un array correspondiente al instante de tiempo actual, se usarán como variables auxiliares para evitar accesos innecesarios a la memoria debido al Pointer Aliasing, usándose así como "registros" intermedios de los datos de entrada y de salida de la simulación respectivamente.

A continuación, se aprecian dos bucles principales: el de difusión, el cual se corresponde a la aplicación de la matriz de difusión en los datos almacenados de esa iteración; y el de viento, que realiza los correspondientes cálculos relacionados con las operaciones de viento. Como se puede apreciar, en cada iteración es necesario calcular el índice para cada iteración de ambos bucles, esto junto a que las separación de bucles genera accesos extra a la memoria puede dar a pensar que se podría optimizar uniendo ambos bucles. Sin embargo, esto no es posible, ya que el flujo de datos del programa se encuentra limitado por las restricciones de dependencias RAW, las cuales se dan al querer acceder a distintas posiciones no lineales de los elementos de "P_diff". Esto junto a la alta complejidad de unificar los bucles por la misma razón, hace que se haya decidido mantener los bucles separados.

Finalmente, cuando el programa ha realizado todas las iteraciones temporales sobre todos los tensores, se descargan de memoria tanto la estructura de componentes del viento, como la matriz de difusión como el elemento auxiliar "P_diff" (pudiendo estos dos últimos liberarlos de memoria sin una función personalizada, ya que en el programa funcionan como un array unidimensional el cual es descargable directamente con la función base "free" de C).

```

1 //Matriz intermedia que almacena estado de P tras bucle difusion
2 float* restrict P_diff = init_matrix(rows_l, cols_l);
3

```

```

4  for (int t = 0; t <= times; t++) {
5      for (int k = 0; k < num_tensors; k++) {
6          //Matriz auxiliar para reducir accesos a memoria
7          //Tambien sirve para evitar Pointer Aliasing al momento de
guardar el resultado
8          float* restrict P_slice = P[k].data;
9
10         //Seccion de tiempo de G, para ahorrar accesos a memoria
11         float* restrict G_slice = &G[k].data[(size_t)t * rows_l * cols_l
];
12
13         //Bucle difusion
14         for (int i = 1; i <= rows; i++) {
15             for (int j = 1; j <= cols; j++) {
16                 size_t id = IDX_2D(i, j, cols_l);
17
18                 float diffusion_edge = P_slice[id - cols_l] + P_slice[id
+ cols_l] + P_slice[id - 1] + P_slice[id + 1];
19                 float diffusion_corner = P_slice[id - cols_l - 1] +
P_slice[id - cols_l + 1] + P_slice[id + cols_l + 1] + P_slice[id +
cols_l - 1];
20
21                 P_diff[id] = acc[id] * (dif_matrix[id] * P_slice[id] + (
diffusion_edge + diffusion_corner * corner_factor) * f_factor);
22             }
23         }
24
25         //Bucle viento
26         for (int i = 1; i <= rows; i++) {
27             for (int j = 1; j <= cols; j++) {
28                 size_t id = IDX_2D(i, j, cols_l);
29
30                 float transport = N[id + 1] * P_diff[id + 1] +
31                 S[id - 1] * P_diff[id - 1] +
32                 E[id - cols_l] * P_diff[id - cols_l] +
33                 W[id + cols_l] * P_diff[id + cols_l] +
34                 NE[id - cols_l + 1] * P_diff[id - cols_l + 1] +
35                 NW[id + cols_l + 1] * P_diff[id + cols_l + 1] +
36                 SE[id - cols_l - 1] * P_diff[id - cols_l - 1] +
37                 SW[id + cols_l - 1] * P_diff[id + cols_l - 1] +
38                 stays[id] * P_diff[id];
39
40                 P_slice[id] = (m_gamma) * acc[id] * (transport + G_slice[
id]);
41             }
42         }
43     }

```

```

44 }
45
46 free_wind(wind);
47 free(dif_matrix);
48 free(P_diff);
49
50 return P;

```

Código 3.13: Código de bucle de función "get_P.last"

3.2.4. Función de simulación completa (get_P_whole)

Inicializado de función y variables

De forma similar al código 3.12, la función con código de inicio 3.14 recibe las estructuras de datos, la familia de tensores inicial, el array de tipos de vehículos y el array de tipos contaminantes.

El inicializado, a su vez, es prácticamente igual al anterior, con el añadido de llamar a la función "init_T_3D"; la cual, como su nombre indica, inicializa una familia de tensores 3D. Esto debido a que el objetivo de la función no es retornar un único resultado, sino realizar un registro de la progresión de la simulación al completo.

```

1 //Retorna la progresion del tensor P durante un tiempo establecido
2 Tensor* get_P_whole(SimulationData* sim_data, MapData* map_data, Tensor*
  G, int* cartypes, char** pollutants) {
3     int rows = map_data -> rows;
4     int cols = map_data -> cols;
5     int times = map_data -> times;
6     float* restrict acc = map_data -> acc;
7
8     float delta = sim_data -> delta;
9     float corner_factor = sim_data -> corner_factor;
10    float gamma = sim_data -> gamma;
11    int num_cartypes = sim_data -> num_cartypes;
12    int num_pollutants = sim_data -> num_pollutants;
13
14    int num_tensors = num_cartypes * num_pollutants;
15
16    //Variables auxiliares para acelerar calculos y legibilidad del
    codigo
17    int rows_l = rows + 2;
18    int cols_l = cols + 2;
19
20    Tensor* P = init_T_3D(sim_data, map_data, cartypes, pollutants);
21
22    WindComponents* wind = get_wind(sim_data, map_data);
23    float* restrict dif_matrix = get_difMatrix(sim_data, map_data);

```

```

24
25 //Puesta en variables de datos de viento para reduccion de tiempos de
    acceso a memoria
26 float* restrict N = wind -> N;
27 float* restrict S = wind -> S;
28 float* restrict E = wind -> E;
29 float* restrict W = wind -> W;
30 float* restrict NE = wind -> NE;
31 float* restrict NW = wind -> NW;
32 float* restrict SE = wind -> SE;
33 float* restrict SW = wind -> SW;
34 float* restrict stays = wind -> stays;
35
36 //Para reducir las divisiones y multiplicaciones realizadas, se
    calcula fuera del bucle la parte fija de la funcion y se anade
37 //a posteriori al calculo de la difusion
38 float f_factor = delta / (4.0f + 4.0f * corner_factor);
39
40 float m_gamma = 1.0f - gamma;
41
42 (Bucle funcion)
43 }

```

Código 3.14: Código de inicio de función "get_P_whole"

Bucle de función

El general, el funcionamiento de 3.15 es similar a 3.13, pero dado que en el programa original se trataba de manera diferente el estado temporal final y no es conveniente incluir bucles en el código (por la posibilidad de perder la optimización SIMD), se ha separado este último del resto del bucle temporal. Además de eso, se han de tratar por separado el estado actual ("t") y siguiente ("t+1") de la simulación, utilizando la variable "P_next" para almacenar el estado temporal "t+1", evaluándose tras la aplicación de la difusión y el viento en el estado actual; y "P_slice" para almacenar el estado temporal "t", actualizándose en el bucle de viento para evitar modificar la matriz del estado a la par que se usan (ya que se acceden a los vecinos en el bucle de difusión).

```

1 //Matriz intermedia que almacena estado de P tras bucle difusion
2 float* restrict P_diff = init_matrix(rows_l, cols_l);
3
4 for (int t = 0; t < times; t++) {
5     for (int k = 0; k < num_tensors; k++) {
6         //Matriz auxiliar para reducir accesos a memoria
7         float* restrict P_slice = &P[k].data[(size_t)t * rows_l * cols_l
8         ];
9
10        //Matriz auxiliar para reducir accesos a memoria

```

```

10     //Corresponde al siguiente instante de tiempo, guardado en el
    mismo tensor para mejorar la localidad temporal y evitar accesos a
    memoria a estructuras distintas
11     float* restrict P_next = &P[k].data[(size_t)(t + 1) * rows_l *
    cols_l];
12
13     //Seccion de tiempo de G, para ahorrar accesos a memoria
14     float* restrict G_slice = &G[k].data[(size_t)t * rows_l * cols_l
    ];
15
16     //unsigned long long t_0 = __rdtsc();
17     //Bucle difusion
18     for (int i = 1; i <= rows; i++) {
19         for (int j = 1; j <= cols; j++) {
20             size_t id = IDX_2D(i, j, cols_l);
21
22             float diffusion_edge = P_slice[id - cols_l] + P_slice[id
    + cols_l] + P_slice[id - 1] + P_slice[id + 1];
23             float diffusion_corner = P_slice[id - cols_l - 1] +
    P_slice[id - cols_l + 1] + P_slice[id + cols_l + 1] + P_slice[id +
    cols_l - 1];
24
25             P_diff[id] = acc[id] * (dif_matrix[id] * P_slice[id] + (
    diffusion_edge + diffusion_corner * corner_factor) * f_factor);
26         }
27     }
28     //Bucle viento
29     for (int i = 1; i <= rows; i++) {
30         for (int j = 1; j <= cols; j++) {
31             size_t id = IDX_2D(i, j, cols_l);
32
33             float transport = N[id + 1] * P_diff[id + 1] +
34                 S[id - 1] * P_diff[id - 1] +
35                 E[id - cols_l] * P_diff[id - cols_l] +
36                 W[id + cols_l] * P_diff[id + cols_l] +
37                 NE[id - cols_l + 1] * P_diff[id - cols_l + 1] +
38                 NW[id + cols_l + 1] * P_diff[id + cols_l + 1] +
39                 SE[id - cols_l - 1] * P_diff[id - cols_l - 1] +
40                 SW[id + cols_l - 1] * P_diff[id + cols_l - 1] +
41                 stays[id] * P_diff[id];
42
43             //Actualizado estado t+1
44             P_next[id] = (m_gamma) * acc[id] * (transport + G_slice[
    id]);
45
46             //Actualizado estado t
47             P_slice[id] = P_diff[id];

```

```

48     }
49 }
50 }
51 }
52
53 //Para evadir el uso de condicionales en el codigo que impidan la
    implementacion de SIMD se ha anadido la iteracion t=times para que
54 //corresponda con el simulador original, modificando unicamente el estado
    t con un bucle de difusion
55 for (int k = 0; k < num_tensors; k++) {
56     //Matriz auxiliar para reducir accesos a memoria
57     float* restrict P_slice = &P[k].data[(size_t)times * rows_l * cols_l
    ];
58
59     #pragma omp for simd collapse(2) schedule(static)
60     //Bucle difusion
61     for (int i = 1; i <= rows; i++) {
62         for (int j = 1; j <= cols; j++) {
63             size_t id = IDX_2D(i, j, cols_l);
64
65             float diffusion_edge = P_slice[id - cols_l] + P_slice[id +
    cols_l] + P_slice[id - 1] + P_slice[id + 1];
66             float diffusion_corner = P_slice[id - cols_l - 1] + P_slice[
    id - cols_l + 1] + P_slice[id + cols_l + 1] + P_slice[id + cols_l - 1
    ];
67
68             P_diff[id] = acc[id] * (dif_matrix[id] * P_slice[id] + (
    diffusion_edge + diffusion_corner * corner_factor) * f_factor);
69         }
70     }
71
72     //Actualizado estado t
73     #pragma omp for simd collapse(2) schedule(static)
74     for (int i = 1; i <= rows; i++) {
75         for (int j = 1; j <= cols; j++) {
76             size_t id = IDX_2D(i, j, cols_l);
77             P_slice[id] = P_diff[id];
78         }
79     }
80 }
81
82 free_wind(wind);
83 free(dif_matrix);
84 free(P_diff);
85
86 return P;

```

Código 3.15: Código de bucle de función "get_P_whole"

3.3. Paralelización del programa mediante OpenMP

Para la optimización mediante paralelización a nivel de hilos en el procesador, se ha hecho uso de OpenMP (Open Multi-Processing) [22]. Este consiste en una interfaz de programación que permite paralelizar secciones del programa mediante la directiva de compilación "pragma". Su modelo de ejecución se basa en "Fork-Join", donde un hilo principal (maestro) genera un grupo de hilos secundarios para dividir la carga de trabajo (fork), sincronizándose una vez finalizar todos los hilos.

A continuación, se explica en detalle qué código se ha decidido paralelizar y cómo se ha llevado a cabo.

3.3.1. Selección de código a paralelizar

Pasando al código que se ha decidido paralelizar, se ha determinado que todas las funciones auxiliares (es decir, las de los archivos "simulatorModule") no sean paralelizadas. A partir de la ley de Amdahl, se puede concluir que las mejoras de rendimiento más efectivas se obtienen al acelerar la parte del programa que concentra la mayor fracción del tiempo de ejecución, cosa que además va en línea con el principio de "hacer el caso más común más rápido" formulado por Patterson y Hennessy [33].

Para ilustrar este límite teórico de aceleración, la Ley de Amdahl se define mediante la siguiente ecuación:

$$S_{\text{latency}}(s) = \frac{1}{(1 - p) + \frac{p}{s}} \quad (3.1)$$

Donde S_{latency} representa la mejora teórica máxima en latencia (aceleración general), p la proporción del tiempo de ejecución del programa que es estrictamente paralelizable y s la aceleración de dicha parte paralelizable.

3.3.2. Implementación de paralelización

Pasando al código paralelizado mediante OpenMP, primero se tiene que evaluar en cada función cuál es la sección o secciones a las que se les puede aplicar una paralelización para, posteriormente, aplicar OpenMP mediante una directiva de compilador "pragma".

- **get.wind y get.difMatrix:** para el código paralelizado referente a estas funciones (resumido en 3.16), se ha decidido aplicar OpenMP a los bucles dobles de los resultados, que constituyen el grueso del coste computacional en ambas funciones. Específicamente, se ha aplicado un colapso del bucle doble mediante la directiva "collapse(2)", para así paralelizarlo en su totalidad y que, de esta forma, se aproveche el mayor espacio de iteración posible para cada hilo. Además de eso, se ha especificado mediante la directiva "schedule(static)", la cual divide el bucle en secciones similares y se les ha asignado a cada hilo de manera estática, ya que, por cómo son los

algoritmos, los hilos realizan las mismas operaciones, lo cual permite evitar el overhead que se daría en una gestión dinámica. También se puede apreciar el uso de la directiva "simd", la cual sirve para forzar al compilador a aplicar SIMD en cualquier caso que sea posible.

```
1 <get_wind/get_difMatrix>(SimulationData* sim_data, MapData* map_data
  ) {
2   (...)
3   #pragma omp parallel for simd collapse(2) schedule(static)
4   for (int i = 1; i <= rows; i++) {
5       for (int j = 1; j <= cols; j++) {
6           (...)
7       }
8   }
9
10  return <wind/dif_matrix>;
11 }
12
```

Código 3.16: Código de bucles paralelizados de funciones "get_wind" y "get_difMatrix", en ambos aplica misma paralelización

- **get_P_last y get_P_whole:** dada la mayor complejidad de las siguiente dos funciones, es necesario determinar primero qué sección es realmente paralelizable (eso sí, dada su similitud, se hablará en general de ambas por igual, ya que el flujo de datos y dependencias en la práctica es muy similar; con la única excepción de la última iteración temporal de "get_P_whole", que se detallará de manera separada). En los códigos 3.17 y 3.18 se pueden apreciar 4 bucles anidados: iteraciones temporales ("t"), nombre de tensores ("k"), y las dimensiones espaciales de la simulación ("i" y "j"). Se aporta a continuación en la figura 3.5 el diagrama de dependencias entre secciones.

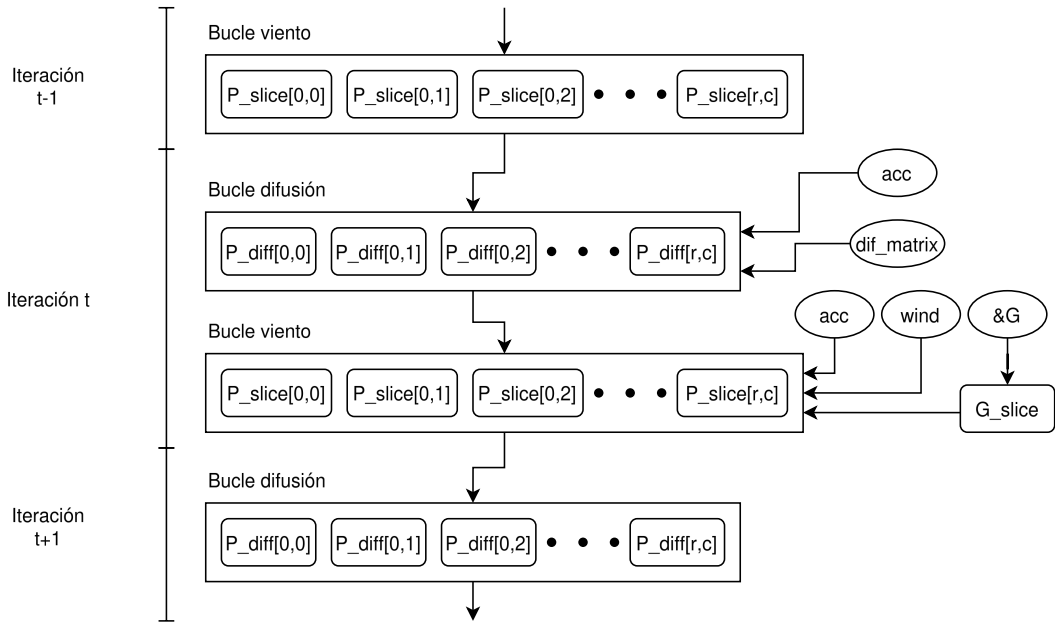


Figura 3.5: Diagrama de dependencias entre secciones de "get_P_last". Las dependencias son idénticas en get_P_whole, sólo que cambian las variable entre dependencias. El grafo es aplicado a cada elemento de las claves de los tensores, no habiendo dependencias entre los mismos

El bucle "t" contiene dependencias de datos (loop-carried dependencies): no se pueden calcular las iteraciones sucesoras de "t" sin resolver las predecesoras, por lo que se debe ejecutar de manera secuencial forzosamente. Por otro lado, el bucle de tensores "k" carece de dependencias, pero el tamaño del mismo es muy bajo, siendo muy influenciado por la longitud de las listas de los tipos de coches y contaminantes, lo cual puede hacer que no se garantice suficiente trabajo para todos los hilos. Esto deja como únicas opciones a los bucles espaciales interiores 'i' y 'j'. Estos se dividen en 2 bucles (difusión y viento), y en ambos los resultados de las casillas no afectan a las vecinas, ya que los resultados se almacenan en variables aparte ("P_slice" o "P_diff"), de manera que no existen dependencias entre iteraciones.

Finalmente, para evitar de crear y destruir los hilos de forma innecesaria ("overhead" de "de Fork-Join"), se ha definido una región de paralelización que coge todo los bucles anidados (#pragma omp parallel que coge todo el bucle "t"). Con esto, los hilos se inician una única vez, distribuyéndose el trabajo entre los mismo al entrar en las zonas con "#pragma omp for". Esto puede parecer que podría ocasionar condiciones de carrera al no cerrar la zona de paralelización en los bucles espaciales, pero OpenMP impone una barrera implícita en cada bloque del bucle de manera automática, de forma que se previene este escenario sin tener que modificar nada del código. Cabe aclarar que, dado que todos los hilos requieren acceder al elemento "P_diff", este se ha de quedar fuera del bloque paralelizado, de manera que sea compartido por todos los hilos en funcionamiento.

```

1 get_P_last(SimulationData* sim_data, MapData* map_data, Tensor* G,
  int* cartypes, char** pollutants) {
2     (...)
3     #pragma omp parallel
4     {
5         for (int t = 0; t <= times; t++) {
6             for (int k = 0; k < num_tensors; k++) {
7                 (...)
8                 #pragma omp for simd collapse(2) schedule(static)
9                 //Bucle difusion
10                for (int i = 1; i <= rows; i++) {
11                    for (int j = 1; j <= cols; j++) {
12                        (...)
13                    }
14                }
15
16                #pragma omp for simd collapse(2) schedule(static)
17                //Bucle viento
18                for (int i = 1; i <= rows; i++) {
19                    for (int j = 1; j <= cols; j++) {
20                        (...)
21                    }
22                }
23            }
24        }
25    }
26    (...)
27    return P;
28 }
29

```

Código 3.17: Código de bucles paralelizados de función "get_P_last"

Respecto a la parte final del código 3.18, se ha mantenido dentro del bloque paralelo y se les ha aplicado una directriz de paralelización idéntica al resto de dobles bucles del programa.

```

1 get_P_whole(SimulationData* sim_data, MapData* map_data, Tensor* G,
  int* cartypes, char** pollutants) {
2     (...)
3     #pragma omp parallel
4     {
5         for (int t = 0; t <= times; t++) {
6             for (int k = 0; k < num_tensors; k++) {
7                 (...)
8                 #pragma omp for simd collapse(2) schedule(static)
9                 //Bucle difusion
10                for (int i = 1; i <= rows; i++) {

```

```

11         for (int j = 1; j <= cols; j++) {
12             (...)
13         }
14     }
15
16     #pragma omp for simd collapse(2) schedule(static)
17     //Bucle viento
18     for (int i = 1; i <= rows; i++) {
19         for (int j = 1; j <= cols; j++) {
20             (...)
21         }
22     }
23 }
24
25
26 for (int k = 0; k < num_tensors; k++) {
27     (...)
28     #pragma omp for simd collapse(2) schedule(static)
29     //Bucle difusion
30     for (int i = 1; i <= rows; i++) {
31         for (int j = 1; j <= cols; j++) {
32             (...)
33         }
34     }
35
36     #pragma omp for simd collapse(2) schedule(static)
37     //Actualizado estado t
38     for (int i = 1; i <= rows; i++) {
39         for (int j = 1; j <= cols; j++) {
40             (...)
41         }
42     }
43 }
44 }
45 (...)
46 return P;
47 }
48

```

Código 3.18: Código de bucles paralelizados de función "get_P_whole"

3.4. Implementación de Cython

Una vez optimizada y paralelizada la implementación del código del simulador en C, se procede a vincularlo con Python y adaptarlo para su uso en el mismo. Se recuerda que el objetivo es realizar una implementación de altas prestaciones del código original que sea cómoda de usar. Para ello, como se mencionó antes, se hará

uso del lenguaje Cython y de su capacidad de poder vincular ambos lenguajes de forma eficiente.

3.4.1. Estructura de código Cython

Como se ha explicado, el código Cython es compilado a un código C, el cual tiene las llamadas a la API nativa de CPython. Este código es finalmente unificado junto al código del simulador y, de esta forma, se obtiene una librería compartida que puede ser importada directamente de forma nativa desde Python.

Pues bien, para realizar tal compilación se hace uso principalmente de dos códigos: el código de Cython "cython_module.pyx" (distribuido en los códigos 3.20, 3.21, 3.22, 3.24 y 3.24), el cual tiene toda la lógica relacionada con la conversión, transferencia y gestión de datos de C a Python y viceversa; y el código de compilación "setup.py" (3.19), el cual contiene las instrucciones y especificaciones de compilación del archivo Cython junto con el resto de código C, retornando un módulo de extensión tipo librería compartida compatible con Python nativo.

```
1 from setuptools import setup, Extension
2 from Cython.Build import cythonize
3 import numpy as np
4
5 ext_modules = [
6     Extension(
7         name="simulator_core",
8         sources=[
9             "cython_module.pyx",
10            "contaminationSeparated.c",
11            "simulatorModule.c",
12        ],
13        #Obligatorio para que encuentre numpy/arrayobject.h
14        include_dirs=[np.get_include()],
15        extra_compile_args=['-O3', '-march=native', '-flto', '-fopenmp'],
16        extra_link_args=['-O3', '-flto', '-fopenmp']
17    )
18 ]
19
20 setup(
21     name="SimuladorContaminacion",
22     ext_modules=cythonize(ext_modules, compiler_directives={'
23         language_level': "3"})
24 )
```

Código 3.19: Código de compilación del programa

3.4.2. Gestión de la memoria

La razón fundamental por la que es requerida esta implementación es por la distanciada gestión que realizan los lenguajes de la memoria, y por ende su complejidad para unificarla e integrarlas.

Gestión de memoria en C

Como se muestra en la figura 3.6, la memoria RAM en C asignada se organiza tradicionalmente en 5 secciones: la memoria de instrucciones (conocida como "text"), que se destina a gestionar las instrucciones binarias (sólo lectura); la memoria de los datos inicializados (conocida como "data"), la cual contiene las variables globales y estáticas del programa con un valor distinto a 0; la memoria de los datos no inicializados (conocida como "BSS"), la cual contiene las variables globales y estáticas del programa con un valor de 0; la pila (o también conocida como "stack"), que se encarga de conservar temporalmente las variables locales encontradas en las funciones; y el "heap", la cual se emplea para la reserva de memoria dinámica del programa en tiempo de ejecución (mediante las funciones "malloc", "calloc"...)[34]. Específicamente, el "heap" se plantea como un bloque de memoria dinámica en tamaño, la cual ha de ser reservada manualmente para su uso y liberada de la misma forma para evitar fugas de memoria.

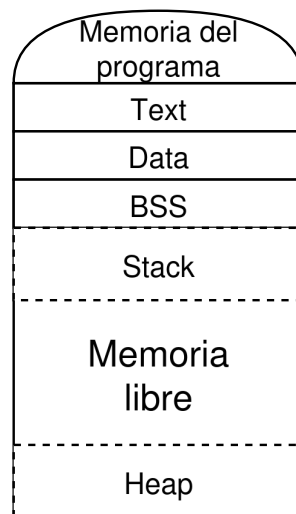


Figura 3.6: Esquema de memoria de un programa C, se aprecia que tanto "heap" como "stack" tienen un tamaño variable

Esto hace que se tenga que exponer explícitamente el uso de memoria del programa que se va a realizar en el código en sí, lo cual, como ya se ha comentado, suele dar a numerosos problemas de "memory leaks" en caso de no ser lo suficientemente riguroso.

Gestión de memoria en Python

En Python, el manejo de la memoria es abstraído mediante la máquina virtual de Python (PVM). A diferencia de C, donde la gestión y liberación de recursos de la memoria es manual y explícita; en Python, la liberación de recursos se delega a

un proceso interno de la PVM llamada "Garbage Collector" (o recolector de basura) [35]. Este sistema se encarga gestionar los recursos por medio de la temporalidad de los mismos durante la ejecución, de forma que si detecta que un objeto no está siendo utilizado en ningún sitio, es liberado de forma autónoma de la memoria de forma instantánea.

Antes de explicar la gestión de la temporalidad de los datos, es necesario explicar la diferencia entre un objeto y una variable o estructura de datos en Python. Se entiende por objeto al dato alojado en memoria, el cual tiene asociada información útil de su identidad, tipo y comportamiento; tal y como se define en el Modelo de Datos estándar del lenguaje [36] (por definición, esto implica que el lenguaje, pese a que en un inicio parece ser mayormente imperativo, fundamenta toda su arquitectura en la Programación Orientada a Objetos). A partir del objeto, se construyen el resto de estructura del lenguaje como las variables, funcionando como un envoltorio que recoge los datos del objeto más algunos nuevos como la etiqueta de referencia o, en estructuras de datos, relaciones entre objetos como la posición tal y como se muestra en la figura 3.7.

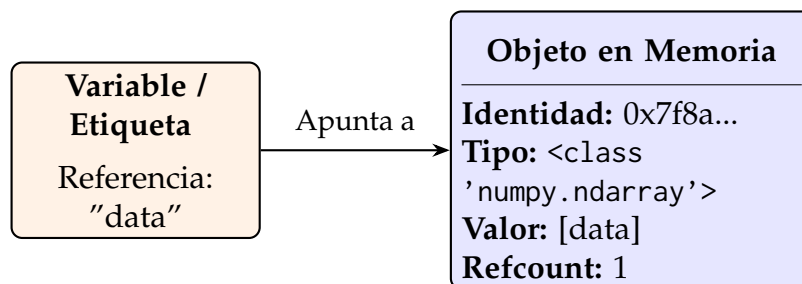


Figura 3.7: Diferencia entre una variable (referencia) y un objeto físico en memoria según el modelo de datos de Python.

Ahora sí, para poder gestionar la temporalidad de los datos en Python, existe un sistema integrado llamado "conteo de referencia". Este consiste en un valor asignado a cada objeto de Python, el cual varía en función de cuántas veces se asigna a una variable o estructura de datos del sistema (aumentando el conteo) o al eliminar las variables referenciadas (disminuyendo el conteo). Cuando el contador llega a 0, el Garbage Collector determina que el dato no es necesario, liberándolo en consecuencia de la memoria.

Si bien esto simplifica el desarrollo al evadir posibles fugas de memoria, también introduce una gran complejidad a la hora de comunicar las variables con C, ya que este espera que los datos en memoria permanezcan estáticos en memoria.

3.4.3. Implementación de transferencia de datos interlenguaje

Entrando en detalle en el código Cython, uno de los temas más importantes a profundizar es la sincronización entre los distintos lenguajes usados, a la par que se mantiene una consistencia en los datos transferidos. Además, se ha de tener en consideración el evitar el uso innecesario de memoria al traspasar la información entre los distintos entornos.

Clases de transferencia de datos

Primero, para el paso de parámetros y datos de Python a C, se han implementado una serie de clases puente en Cython mediante la directiva "cdef class" (código 3.20). Ejemplos de esto son las clases "SimulationData" y "MapData", las cuales actúan como envoltorios (o "wrappers") orientados a objetos en Python, pero que internamente contienen y rellenan las estructuras nativas de C (específicamente "SimulationData_c" y "MapData_c").

- **SimulationData:** almacena los parámetros físicos de la simulación y los transfiere a la estructura SimulationData_c nativa.
- **MapData:** gestiona las dimensiones del mapa y los tiempos de simulación. El caso de "MapData" refleja el problema del Garbage Collector y cómo se ha solventado para este caso. Al recibir una matriz de NumPy (acc) como un puntero a la estructura (para poder ser tratado en C), no se modifica el conteo de referencia de los objetos de acc, pudiendo ser eliminados automáticamente sin quererlo. Para evitar esto, se ha implementado en la clase una referencia artificial como un atributo (específicamente, "self.acc_ndarray"), de forma que los objetos de acc no serán eliminados hasta que la clase "MapData" sea eliminada.

```
1 #Clases puente para creacion de structs
2 cdef class SimulationData:
3     cdef SimulationData_c c_struct
4
5     def __init__(self, float delta, float corner_factor, float gamma, int
        num_cartypes, int num_pollutants, float WN, float WE):
6         self.c_struct.num_cartypes = num_cartypes
7         self.c_struct.num_pollutants = num_pollutants
8         self.c_struct.gamma = gamma
9         self.c_struct.delta = delta
10        self.c_struct.corner_factor = corner_factor
11        self.c_struct.WN = WN
12        self.c_struct.WE = WE
13
14 cdef class MapData:
15     cdef MapData_c c_struct
16     cdef cnp.ndarray acc_ndarray
17
18     def __init__(self, int rows, int cols, int times, cnp.ndarray[cnp.
        float32_t, ndim=2, mode="c"] acc):
19         self.c_struct.rows = rows
20         self.c_struct.cols = cols
21         self.c_struct.times = times
22         #Se toma directamente el puntero de los datos del array numpy acc
23         self.c_struct.acc = <float*> acc.data
24         #Para evitar que python elimine automaticamente el dato acc, se
        asigna a un atributo auxiliar de la clase
```

Código 3.20: Código de clases "SimulationData" y "MapData"

Problema de la desasignación (TensorDeallocator)

Dado que los resultados del simulador son retornados por el código en C, esto se encuentran gestionados por el mismo lenguaje, de forma que la gestión de la memoria ha de ser manual. El Garbage Collector, al estar gestionado por Python, no tiene conocimiento de la memoria asignada con "malloc" en C, por lo que existe el riesgo de colapsar la memoria con datos no deseados.

Para evitar esto, es necesario indicar a Python cómo liberar los datos para no generar memory leaks. Para ello, en el código 3.21, se muestra la creación de una clase específica para la gestión de memoria, cuya función será la de limpiar la memoria correspondiente a c_P y a sim_data de la manera especificada en el método "__dealloc__", el cual es un método exclusivo de Cython que permite ejecutar una función cuando el conteo de referencia del objeto llega a 0. De este modo, la función de limpieza se aplicará de forma segura cuando el Garbage Collector de Python decida eliminar la instancia correspondiente de TensorDeallocator.

```

1 #Dado que el resultado obtenido de la simulacion se encuentra gestionado
  en memoria por C, es necesario indicar
2 #a python como se debe liberar para no generar memory leaks
3
4 #Para ello, se ha creado la siguiente clase, cuya funcion sera la de
  limpiar la memoria correspondiente a c_P y a sim_data de la manera
5 #especificada en __dealloc__, la cual se aplicara cuando el garbage
  collector de python decida eliminar la instancia correspondiente
6 #de TensorDeallocator
7 cdef class TensorDeallocator:
8     cdef Tensor_c* c_P
9     cdef SimulationData_c* sim_data
10
11     def __cinit__(self):
12         self.c_P = NULL
13         self.sim_data = NULL
14
15     def __dealloc__(self):
16         if self.c_P != NULL and self.sim_data != NULL:
17             free_T(self.c_P, self.sim_data)

```

Código 3.21: Código de clase auxiliar "TensorDeallocator"

Funciones de transferencia

El proceso de mover información hacia C y empaquetar los resultados de vuelta hacia Python requiere de un control preciso sobre cómo se manipulan los datos y

sus referencias.

- **dict_to_contiguous_4d_tensor:** para la conversión inicial de las dimensiones de los datos comentada anteriormente se tiene la función 3.22. Su principal cometido es iterar sobre un diccionario de tensores 3D y aplanarlos en un único bloque de memoria contiguo, lo cual es requerido para que C pueda iterar de la manera más eficiente posible.

Inicialmente se calculan las dimensiones totales de la memoria a reservar basándose en el número de tipos de vehículos, contaminantes y las dimensiones del mapa. Además, se emplea "np.empty" en lugar de "np.zeros" para reducir el tiempo de ejecución al no tener que inicializar los valores a 0.

Tras eso, se emplea una sintaxis exclusiva de Cython (::1) para garantizar que el tensor de salida es continua en memoria, lo que permite un acceso muy eficiente a la memoria.

Finalmente, se realiza un triple bucle para transponer los datos. En este, se cambia el orden de los datos del formato de entrada (i, j, t) (filas, columnas, tiempo) al formato que espera el motor en C (k, t, i, j) (donde k es el índice del tensor con etiqueta "vehículo/contaminante").

```
1 def dict_to_contiguous_4d_tensor(dict G_dict, MapData map_data, list
    cartypes_list, list pollutants_list):
2     cdef int num_cartypes = len(cartypes_list)
3     cdef int num_pollutants = len(pollutants_list)
4     cdef int num_tensors = num_cartypes * num_pollutants
5
6     cdef int rows_l = map_data.c_struct.rows + 2
7     cdef int cols_l = map_data.c_struct.cols + 2
8     cdef int times_l = map_data.c_struct.times + 1
9
10    #np.empty en lugar de np.zeros para no reducir tiempo de
    ejecucion al no tener que inicializar valores a 0
11    cdef cnp.ndarray[cnp.float32_t, ndim=4, mode="c"] G_out = np.
    empty((num_tensors, times_l, rows_l, cols_l), dtype=np.float32)
12
13    #Sintaxis exclusiva de Cython, la cual crea un struct con
    elementos float de 4 dimensiones a la cual se le asigna el
    puntero de G_out
14    #::1 para determinar que la dimension asignada es continua en
    memoria
15    cdef float[:, :, :, ::1] G_out_view = G_out
16    cdef float[:, :, :] G_in_view
17
18    cdef int k = 0
19    cdef int t, i, j
20    cdef str key
21
```

```

22     for cartype in cartypes_list:
23         for pollutant in pollutants_list:
24             key = f"{cartype}_{pollutant}"
25
26             #pop() extrae el valor y elimina la clave del
diccionario
27             G_in_view = G_dict.pop(key)
28
29             for i in range(rows_l):
30                 for j in range(cols_l):
31                     for t in range(times_l):
32                         G_out_view[k, t, i, j] = G_in_view[i, j, t]
33
34             k += 1
35
36     return G_out
37

```

Código 3.22: Código de función "dict.to_contiguous_4d.tensor"

- **get_P_last.p:** esta función sirve como envoltorio (wrapper) para la función de C "get_P_last". Se utiliza cuando solo se requiere el estado final de la contaminación. Implementación en código [3.23](#)

Al empezar, se asigna una sección de memoria dinámica mediante "malloc" para convertir las listas de tipos de vehículos y nombres de contaminantes en arrays de C. Para evitar que el Garbage Collector elimine estos datos sin querer, se inicializa una lista llamada "protected_names" para evitar que los strings convertidos a bytes se eliminen antes de que C termine de usarlos.

Tras eso, se inicializa un array de estructuras de "Tensor.c", asignando para el campo de datos un puntero a las direcciones controladas por Python (específicamente, la dirección de memoria dentro del tensor 4D de NumPy creado previamente), de forma que no se copien o se traspasen datos de forma innecesaria, sino que directamente se use esa región de la memoria.

Con todo esto, se puede llamar a la función nativa de C "get_P_last", y obtener de la misma el puntero a los resultados del simulador.

Para traspasar los datos a Python de vuelta, se inicializa una instancia de "TensorDeallocator" para gestionar la memoria de C.

Tras ello, se inicializa el diccionario de salida de la función y se define las dimensiones de los datos mediante la variable "shape", para posteriormente inicializar el array de datos mediante la función específica de la API de NumPy "PyArray_SimpleNewFromData". Esta función, al contrario de la inicialización de un array normal, no reserva nueva memoria para los datos ni copia los valores, sino que crea una cabecera de objeto NumPy que apunta directamente a la dirección de memoria preexistente en "c_P[k].data" la cual contiene los resultados del código C [37].

Para completar la sincronización de los dos lenguajes, se hace uso de la función de la API de NumPy "PyArray_SetBaseObject". Esta función toma posesión de la referencia indicada, estableciendo además las propiedades base del objeto (en este caso, las propiedades de destrucción del mismo). Sin embargo, esta implementación acarrea un problema grave y es que, dado que se van a crear múltiples arrays (uno por cada tipo de vehículo y contaminante) con el mismo objeto "owner", se debe gestionar manualmente el contador de referencias de CPython. Esto se debe a cómo gestiona la referencia "PyArray_SetBaseObject", ya que esta toma posesión y control de la referencia al objeto indicado, forzando al desarrollador a incrementar el conteo manualmente. Esto sumado a que se usa numerosas veces con el mismo objeto, se llega a la situación en la que, si se borra algún elemento de la lista, el contador de referencia llegue a 0 de manera anticipada, eliminando al completo los datos de todos los tensores. Para solventar esto, se incrementa en 1 el contador de referencia manualmente mediante la función "Py_INCREF".

Para terminar, aprovechando que se tienen los nombres almacenados de cada tensor en C, se hace una comprobación para determinar que el orden de las salidas concuerda con la entrada proporcionada, añadiéndolo al diccionario de salida el resultado en caso de ser correcto.

```

1 def get_P_last_p(SimulationData sim_data, MapData map_data, list
  cartypes_list, list pollutants_list, cnp.ndarray[cnp.float32_t,
  ndim=4, mode="c"] G):
2     cdef int num_cartypes = len(cartypes_list)
3     cdef int num_pollutants = len(pollutants_list)
4     cdef int num_tensors = num_cartypes * num_pollutants
5
6
7     #Conversion de listas python a arrays C
8     cdef int* c_cartypes = <int*> malloc(num_cartypes * sizeof(int))
9     cdef int i
10    for i in range(num_cartypes):
11        c_cartypes[i] = cartypes_list[i]
12
13    #Para evitar que python elimine automaticamente los nombres (al
    momento de sobrescribirlos), se crea una lista auxiliar la cual
    va a aumentar automaticamente
14    #el numero de referencia, ademas de que se eliminara
    automaticamente al finalizar la funcion
15    cdef list protected_names = []
16    cdef char** c_pollutants = <char**> malloc(num_pollutants *
    sizeof(char*))
17    for i in range(num_pollutants):
18        name = pollutants_list[i].encode('utf-8')
19        protected_names.append(name)
20        c_pollutants[i] = name

```

```

21
22
23     #Creacion de familia de tensores G
24     cdef Tensor_c* c_G = <Tensor_c*> malloc(num_tensors * sizeof(
Tensor_c))
25     cdef int k = 0
26     cdef str key
27     for cartype in cartypes_list:
28         for pollutant in pollutants_list:
29             key = f"{cartype}_{pollutant}"
30             name = key.encode('utf-8')
31             protected_names.append(name)
32
33             c_G[k].name = name
34             c_G[k].data = <float*> &G[k, 0, 0, 0]
35             k += 1
36
37
38
39     cdef double t_inicio_c = time.perf_counter()
40
41     #Funcion principal
42     cdef Tensor_c* c_P = get_P_last(&sim_data.c_struct, &map_data.
c_struct, c_G, c_cartypes, c_pollutants)
43
44     cdef double t_fin_c = time.perf_counter()
45
46
47
48     #Inicializado de gestor de memoria
49     cdef TensorDeallocator owner = TensorDeallocator()
50     owner.c_P = c_P
51     owner.sim_data = &sim_data.c_struct
52
53     #Dimensiones de array np
54     cdef cnp.npy_intp shape[2]
55     shape[0] = map_data.c_struct.rows + 2
56     shape[1] = map_data.c_struct.cols + 2
57
58     cdef dict P_dict = {}
59     cdef cnp.ndarray P_arr
60     k = 0
61
62     for cartype in cartypes_list:
63         for pollutant in pollutants_list:
64             #Creacion de array np a partir de la direccion de c_P[k
].data

```

```

65         P_arr = cnp.PyArray_SimpleNewFromData(2, shape, cnp.
NPY_FLOAT32, c_P[k].data)
66
67         #Incremento del numero de referencias de owner manual
68         Py_INCREF(owner)
69
70         #Establecimiento de las propiedades base del objeto (de
destruccion en este caso)
71
72         #Dado que lo que hace la funcion es guardar owner como
un atributo de P_arr, no se incrementa el numero de referencia
del mismo, pudiendo hacer que se
73         #elimine antes de tiempo si se empiezan a eliminar los
arrays pese a pertenecer a otros arrays que si lo usen, de ahi
que se incremente manualmente
74         #el numero de referencia de owner
75         cnp.PyArray_SetBaseObject(P_arr, owner)
76
77         key = f"{cartype}_{pollutant}"
78
79         #Comprobacion nombres de tensores respecto a orden
esperado
80         if key != c_P[k].name.decode('utf-8'):
81             raise ValueError(f"Inconsistencia encontrada de
tensores en C respecto a orden establecido: "
82                             f"se esperaba '{key}', pero se recibio '{c_P
[k].name.decode('utf-8')}'")
83
84         P_dict[key] = P_arr
85         k += 1
86
87
88         free(c_G)
89         free(c_cartypes)
90         free(c_pollutants)
91
92         return P_dict
93

```

Código 3.23: Código de función "get_P.last.p"

- **get_P.whole.p**: esta función sirve como envoltorio (wrapper) para la función de C "get_P.whole". Se utiliza cuando se requiere el estado completo de la contaminación. Implementación en código [3.23](#)

El proceso de preparación de datos es idéntico al de "get_P.last.p": se realiza la reserva de memoria mediante "malloc" para los arrays de tipos de vehículos y contaminantes, y se utiliza la lista "protected_names" para asegurar que las

referencias de los strings de Python convertidos a "char*" no sean recolectadas por el Garbage Collector mientras el motor de C las procesa. Posteriormente, se mapean las direcciones de memoria del tensor 4D de entrada al array de estructuras "Tensor_c" para evitar duplicidad de datos.

La diferencia fundamental radica en la reconstrucción de los resultados, ya que tras ejecutar la simulación en C, se configura el objeto "TensorDeallocator" y se define una variable "shape" de 3 dimensiones en vez de 2. Al iterar para crear los objetos de salida, se utiliza nuevamente "PyArray_SimpleNewFromData", pero especificando que los nuevos arrays de NumPy son tridimensionales. Al igual que en el caso anterior, es crítico el uso de "Py_INCREF"(owner) antes de cada llamada a "PyArray_SetBaseObject".

```

1 def get_P_whole_p(SimulationData sim_data, MapData map_data, list
    cartypes_list, list pollutants_list, cnp.ndarray[cnp.float32_t,
    ndim=4, mode="c"] G):
2
3     cdef int num_cartypes = len(cartypes_list)
4     cdef int num_pollutants = len(pollutants_list)
5     cdef int num_tensors = num_cartypes * num_pollutants
6
7
8     #Conversion de listas python a arrays C
9     cdef int* c_cartypes = <int*> malloc(num_cartypes * sizeof(int))
10    cdef int i
11    for i in range(num_cartypes):
12        c_cartypes[i] = cartypes_list[i]
13
14    #Para evitar que python elimine automaticamente los nombres (al
    momento de sobreescribirlos), se crea una lista auxiliar la cual
    va a aumentar automaticamente
15    #el numero de referencia, ademas de que se eliminara
    automaticamente al finalizar la funcion
16    cdef list protected_names = []
17    cdef char** c_pollutants = <char**> malloc(num_pollutants *
    sizeof(char*))
18    for i in range(num_pollutants):
19        name = pollutants_list[i].encode('utf-8')
20        protected_names.append(name)
21        c_pollutants[i] = name
22
23
24    #Creacion de familia de tensores G
25    cdef Tensor_c* c_G = <Tensor_c*> malloc(num_tensors * sizeof(
    Tensor_c))
26    cdef int k = 0
27    cdef str key
28    for cartype in cartypes_list:

```

```

29     for pollutant in pollutants_list:
30         key = f"{cartype}_{pollutant}"
31         name = key.encode('utf-8')
32         protected_names.append(name)
33
34         c_G[k].name = name
35         c_G[k].data = <float*> &G[k, 0, 0, 0]
36         k += 1
37
38
39     #Funcion principal
40     cdef Tensor_c* c_P = get_P_whole(&sim_data.c_struct, &map_data.
41     c_struct, c_G, c_cartypes, c_pollutants)
42
43
44     #Inicializado de gestor de memoria
45     cdef TensorDeallocator owner = TensorDeallocator()
46     owner.c_P = c_P
47     owner.sim_data = &sim_data.c_struct
48
49     #Dimensiones de array np
50     cdef cnp.npy_intp shape[3]
51     shape[0] = map_data.c_struct.times + 1
52     shape[1] = map_data.c_struct.rows + 2
53     shape[2] = map_data.c_struct.cols + 2
54
55     cdef dict P_dict = {}
56     cdef cnp.ndarray P_arr
57     k = 0
58
59     for cartype in cartypes_list:
60         for pollutant in pollutants_list:
61             #Creacion de array np a partir de la direccion de c_P[k
62             ].data
63             P_arr = cnp.PyArray_SimpleNewFromData(3, shape, cnp.
64             NPY_FLOAT32, c_P[k].data)
65
66             #Incremento del numero de referencias de owner manual
67             Py_INCREF(owner)
68
69             #Establecimiento de las propiedades bases del objeto (de
70             destruccion en este caso)
71
72             #Dado que lo que hace la funcion es guardar owner como
73             un atributo de P_arr, no se incrementa el numero de referencia
74             del mismo, pudiendo hacer que se

```

```

70         #elimine antes de tiempo si se empiezan a eliminar los
arrays pese a pertenecer a otros arrays que si lo usen, de ahi
que se incremente manualmente
71         #el numero de referencia de owner
72         cnp.PyArray_SetBaseObject(P_arr, owner)
73
74         key = f"{cartype}_{pollutant}"
75
76         #Comprobacion nombres de tensores respecto a orden
esperado
77         if key != c_P[k].name.decode('utf-8'):
78             raise ValueError(f"Inconsistencia encontrada de
tensores en C respecto a orden establecido: "
79                             f"se esperaba '{key}', pero se recibio '{c_P
[k].name.decode('utf-8')}'")
80
81         P_dict[key] = P_arr
82         k += 1
83
84
85     free(c_G)
86     free(c_cartypes)
87     free(c_pollutants)
88
89     return P_dict
90

```

Código 3.24: Código de función "get_P_whole.p"

3.5. Automatización de la implementación

Para finalizar, por motivos de comodidad, se ha decidido automatizar la implementación de todo el código mediante el uso de un archivo Makefile [38]. Este consiste en un archivo utilizado para automatizar tareas, principalmente la compilación y construcción de software. Funciona junto con la herramienta make [38], leyendo reglas que definen las dependencias entre archivos, lo que permite al sistema actualizar solo los archivos modificados, ahorrando tiempo de desarrollo.

En el caso del proyecto, se han creado tres opciones posibles para el archivo Makefile: la opción "help" (o la por defecto), que muestra las posibles opciones del archivo Makefile; la opción "build", la cual compila y crea el módulo en C del simulador; la opción "build.c", la cual además de compilar el módulo del simulador, elimina los archivos temporales no relevantes; y la opción "clean", cuya finalidad es eliminar en su totalidad todo archivo referente al módulo (en caso de haber archivos temporales también los elimina).

4. Pruebas y resultados

En esta sección, se abordarán dilemas como la precisión de la implementación, junto a los resultados obtenidos en un benchmark de rendimiento en el que se comparen tanto el simulador original, como la implementación en C sin paralelizar, como la paralelizada. Esto se realizará en distintos sistemas, de manera que se pueda comparar el código en diferentes dispositivos de hardware. El código fuente de las pruebas puede ser encontrado nuevamente en "https://github.com/nical1706/sim_propagacion_contaminantes_optimizado".

4.1. Precisión de la implementación

Para que la implementación realizada sea válida, se ha de verificar que los resultados obtenidos de la misma sean lo suficientemente precisos como para ser equiparables en la práctica a los de la implementación original. Para ello, se compararán mediante un código las implementaciones de Python, C sin OMP y C con OMP; los mismos parámetros iniciales para todas las versiones.

Sin embargo, para realizar correctamente comparaciones entre simuladores, primero hay que asegurar que la implementación original cumpla con un nivel de precisión aceptable.

4.1.1. Imprecisión de la implementación original

En Python, el tipo de valor "float" no es de 32 bits (como ocurre por defecto en lenguajes como C), sino que hace uso de la implementación del estándar IEEE 754 [39] de doble precisión (64 bits o "double" en otros lenguajes).

Si bien a primera vista esta mayor cantidad de bits parece mejorar la precisión del simulador, esto sólo se cumple si los datos se operan estrictamente con variables de su misma jerarquía. Al observar el código 4.1, se aprecia que varios parámetros iniciales se han establecidos como valores decimales genéricos (equivalentes al tipo "float64" de Python) y otros usan el tipo entero. Por otro lado, al inicializar los tensores de datos "G" y "P", se establece explícitamente que la información almacenada es de tipo "float32", de forma que al interactuar los operandos de 64 bits o enteros nativos con los tensores de 32 bits, se produce un escalado en los datos conocido como "upcast" para permitir la realización de la operación correspondiente.

El problema reside en que, tras realizar la operación, se intenta asignar el resultado final de vuelta a las estructuras de 32 bits. Esto fuerza un truncamiento de los datos que introduce leves redondeos a nivel de bit. Pero debido a que el simulador utiliza datos de temporalidades anteriores para obtener los siguientes estados, se genera un efecto cascada por el cual, tras varias iteraciones, el error acumulado se vuelve perceptible a simple vista en las visualizaciones.

```

1 # Parameters as in the traffic simulator. Change to do experiments.
2 width = 10
3 height = 10
4 carTypes = [0]#[0, 1, 2] # 0 = EV, 1 = Petrol, 2 = Diesel
5 pollutants = ['CO2']#['CO2', 'NOx', 'VOC', 'PMexhaust', 'PMexhaustprueba',
6   'PMnonexhaust25', 'PMnonexhaust10']
7 times = 10
8 gamma = 0.01
9 delta = 0.1
10 corner_factor = 1
11 WN = 0.3
12 WE = 0.5
13 # Assume as given a family of tensors G of the form
14 G = {f"{cartype}_{pollutant}": np.zeros((width+2, height+2, times+1),
15   dtype=np.float32)
16   for cartype in carTypes
17   for pollutant in pollutants}
18 # and an accessibility matrix
19 acc = np.ones((width+2,height+2))

```

Código 4.1: Extracto de inicio de implementación original.

Para solventar esto, se ha realizado una implementación más exacta (código 4.2), en la que se evitan los problemas descritos por medio del encapsulado explícito de los tipos de datos, asegurando que en todo momento los datos sean del mismo tipo. Puesto que en el código original se especificaba que los datos de entrada "G" eran de tipo "float32", se ha fijado este tipo como el estándar para todas las operaciones del código, cancelando por completo el "upcast" implícito de Python.

```

1 #Para evitar cambios de tipos por parte de Python de int a float64, se
   han fijado las siguientes variables
2 CERO = np.float32(0.0)
3 UNO = np.float32(1.0)
4 CUATRO = np.float32(4.0)
5
6 def get_wind_prec(acc):
7     #Inicializado de matrices en 32 bits
8     displ_N = np.zeros((width+2, height+2), dtype=np.float32)
9     displ_S = np.zeros_like(displ_N)
10    displ_E = np.zeros_like(displ_N)
11    displ_W = np.zeros_like(displ_N)
12    displ_NW = np.zeros_like(displ_N)
13    displ_NE = np.zeros_like(displ_N)
14    displ_SW = np.zeros_like(displ_N)
15    displ_SE = np.zeros_like(displ_N)
16
17    sign_WN = int(np.sign(WN))
18    sign_WE = int(np.sign(WE))

```

```

19
20 for p in range(1, width+1):
21     for q in range(1, height+1):
22         #Casting explicito de las funciones maximo y absoluto
23         max_WN = np.maximum(WN, CERO, dtype=np.float32)
24         max_m_WN = np.maximum(-WN, CERO, dtype=np.float32)
25         max_WE = np.maximum(WE, CERO, dtype=np.float32)
26         max_m_WE = np.maximum(-WE, CERO, dtype=np.float32)
27
28         abs_WE = np.abs(WE, dtype=np.float32)
29         abs_WN = np.abs(WN, dtype=np.float32)
30
31         #Ecuaciones separadas para evitar upcast de Python
32         term_N = UNO - np.maximum(acc[p + sign_WE, q - 1], acc[p +
33         sign_WE, q], dtype=np.float32) * abs_WE
34         displ_N[p,q] = (acc[p,q] * max_WN * term_N * acc[p, q - 1]).
35         astype(np.float32)
36
37         term_S = UNO - np.maximum(acc[p + sign_WE, q + 1], acc[p +
38         sign_WE, q], dtype=np.float32) * abs_WE
39         displ_S[p,q] = (acc[p,q] * max_m_WN * term_S * acc[p, q + 1])
40         .astype(np.float32)
41
42         term_E = UNO - np.maximum(acc[p + 1, q - sign_WN], acc[p, q -
43         sign_WN], dtype=np.float32) * abs_WN
44         displ_E[p,q] = (acc[p,q] * max_WE * term_E * acc[p + 1, q]).
45         astype(np.float32)
46
47         term_W = UNO - np.maximum(acc[p - 1, q - sign_WN], acc[p, q -
48         sign_WN], dtype=np.float32) * abs_WN
49         displ_W[p,q] = (acc[p,q] * max_m_WE * term_W * acc[p - 1, q])
50         .astype(np.float32)
51
52         displ_NE[p,q] = (acc[p,q] * max_WN * max_WE * acc[p + 1, q -
53         1]).astype(np.float32)
54         displ_NW[p,q] = (acc[p,q] * max_WN * max_m_WE * acc[p - 1, q
55         - 1]).astype(np.float32)
56         displ_SE[p,q] = (acc[p,q] * max_m_WN * max_WE * acc[p + 1, q
57         + 1]).astype(np.float32)
58         displ_SW[p,q] = (acc[p,q] * max_m_WN * max_m_WE * acc[p - 1,
59         q + 1]).astype(np.float32)
60
61         #Suma estricta en 32 bits
62         sum_wind = (displ_N + displ_S + displ_E + displ_W + displ_NE +
63         displ_NW + displ_SE + displ_SW).astype(np.float32)
64         stays = (UNO - sum_wind).astype(np.float32)
65
66

```

```

53     wind = (displ_N[1:-1, 2:], displ_S[1:-1, :-2], displ_E[:-2, 1:-1],
54             displ_W[2:, 1:-1], displ_NE[:-2, 2:], displ_NW[2:, 2:], displ_SE[:-2,
55             :-2], displ_SW[2:, :-2], stays[1:-1, 1:-1])
56     return wind
57
58 def get_difMatrix_prec(acc):
59     acc_neig_edge = (acc[0:-2, 1:-1] + acc[2:, 1:-1] + acc[1:-1, 0:-2] +
60     acc[1:-1, 2:]).astype(np.float32)
61     acc_neig_corner = (acc[0:-2, 0:-2] + acc[2:, 2:] + acc[2:, 0:-2] +
62     acc[0:-2, 2:]).astype(np.float32)
63
64     #Extraccion de calculo respecto a original para forzar facilmente
65     tipado
66     f_factor = (delta / (CUATRO + CUATRO * corner_factor)).astype(np.
67     float32)
68     dif_matrix = (UNO - (acc_neig_edge + acc_neig_corner * corner_factor)
69     .astype(np.float32) * f_factor).astype(np.float32)
70     return dif_matrix
71
72 def get_P_whole_prec(acc):
73     P = {f"{cartype}_{pollutant}": np.zeros((width+2, height+2, times+1),
74     dtype=np.float32)
75         for cartype in car_types
76         for pollutant in pollutants}
77     wind = get_wind_prec(acc)
78     dif_matrix = get_difMatrix_prec(acc)
79
80     #Cambio de variable respecto a codigo original para evitar
81     sobreescribir acc de entrada
82     acc_m = acc[1:-1, 1:-1]
83     for i in range(times+1):
84         for key in P:
85             diffusion_edge = (P[key][0:-2, 1:-1, i] + P[key][2:, 1:-1, i]
86             + P[key][1:-1, 0:-2, i] + P[key][1:-1, 2:, i]).astype(np.float32)
87             diffusion_corner = (P[key][0:-2, 0:-2, i] + P[key][2:, 2:, i]
88             + P[key][2:, 0:-2, i] + P[key][0:-2, 2:, i]).astype(np.float32)
89
90             f_factor = (delta / (CUATRO + CUATRO * corner_factor)).astype
91             (np.float32)
92             dif_P = (dif_matrix * P[key][1:-1, 1:-1, i]).astype(np.float3
93             2)
94             P[key][1:-1, 1:-1, i] = (acc_m * (dif_P + (diffusion_edge +
95             diffusion_corner * corner_factor).astype(np.float32) * f_factor).
96             astype(np.float32)).astype(np.float32)
97
98             if i < times:
99                 m_gamma = (UNO - gamma).astype(np.float32)

```

```

85         # Wind, sources and loss to atmosphere
86         P[key][1:-1, 1:-1, i+1] = (m_gamma * acc_m * (wind[0]*P[
key][1:-1, 2:, i] + wind[1]*P[key][1:-1, :-2, i] + wind[2]*P[key][:-2,
1:-1, i] + wind[3]*P[key][2:, 1:-1, i] + wind[4]*P[key][:-2, 2:, i] +
wind[5]*P[key][2:, 2:, i] + wind[6]*P[key][:-2, :-2, i] + wind[7]*P[
key][2:, :-2, i] + wind[8]*P[key][1:-1, 1:-1, i] + G[key][1:-1, 1:-1,
i]).astype(np.float32)).astype(np.float32)
87
88     return P

```

Código 4.2: Inicio código de prueba de precisión, en la que se encuentra la implementación con datos igualados.

4.1.2. Código de pruebas de precisión

Con lo anterior explicado, se procede a realizar un benchmark de precisión para verificar que no haya diferencias entre implementación en Python (modificada para evitar "upcast") y C, y ya de paso mostrar el inicializado y uso del módulo en un código Python externo.

Para el inicializado de los valores, se ha creado un código Python externo llamado "utils.py" (código 4.3), en el cual se han creado las funciones de inicializado de datos para cada implementación del simulador. Para el inicializado de los datos, primero se establece la matriz de accesibilidad, la cual está compuesta por estructuras simétricas simulando la distribución de una ciudad; posteriormente se generan datos aleatorios de emisiones para G, de manera que cada estado temporal tiene datos diferentes.

```

1 def init_data_python(width, height, times, car_types, pollutants):
2     #acc contiguo en memoria (con orden C) para acelerar tiempo
transmision
3     acc = np.zeros((width+2, height+2), dtype=np.float32, order='C')
4
5     #Plantilla para aplicacion de patrones de edificios
6     template = np.zeros(100, dtype=np.float32)
7
8     template[0:4] = 1
9     template[96:100] = 1
10    template[48:52] = 1
11
12    template[25:27] = 1
13    template[73:75] = 1
14
15    #Aplicacion del patron
16    #np.ceil asegura copias del template, slicing [:width] corta al
tamano exacto
17    pattern_w = np.tile(template, int(np.ceil(width / 100)))[ :width]
18    pattern_h = np.tile(template, int(np.ceil(height / 100)))[ :height]
19    row_mask = np.zeros(width + 2, dtype=bool)

```

```

20 row_mask[1:-1] = (pattern_w == 1)
21 col_mask = np.zeros(height + 2, dtype=bool)
22 col_mask[1:-1] = (pattern_h == 1)
23
24 acc[row_mask == 1, :] = 1
25 acc[:, col_mask == 1] = 1
26
27
28 #Inicializado tensor de emisiones de entrada G
29 G = {f"{cartype}_{pollutant}": np.zeros((width+2, height+2, times+1),
30 dtype=np.float32)
31         for cartype in car_types
32         for pollutant in pollutants}
33
34 #Umbral estadístico para generación de datos aleatorios
35 u = 0.1
36 for key in G:
37     #Peso (arbitrario) de emisiones según el tipo de origen
38     if 'electric' in key:
39         emission = 0.0
40     elif 'diesel' in key:
41         emission = 5000.0
42     else: # gasoline
43         emission = 3000.0
44     if emission > 0:
45         for t in range(times + 1):
46             rand = np.random.rand(width+2, height+2)
47             #Filtrado de ruido según umbral y acc
48             f_rand = (rand < u) * acc
49             pollution = f_rand * emission
50             G[key][:, :, t] = pollution
51 return (acc, G)
52
53 def init_data_no_omp(width, height, times, car_types, pollutants):
54     #acc contiguo en memoria (con orden C) para acelerar tiempo
55     #transmision
56     acc = np.zeros((width+2, height+2), dtype=np.float32, order='C')
57
58     #Plantilla para aplicación de patrones de edificios
59     template = np.zeros(100, dtype=np.float32)
60
61     template[0:4] = 1
62     template[96:100] = 1
63     template[48:52] = 1
64
65     template[25:27] = 1

```

```

65     template[73:75] = 1
66
67     #Aplicacion del patron
68     #np.ceil asegura copias del template, slicing [:width] corta al
tamano exacto
69     pattern_w = np.tile(template, int(np.ceil(width / 100)))[:width]
70     pattern_h = np.tile(template, int(np.ceil(height / 100)))[:height]
71     row_mask = np.zeros(width + 2, dtype=bool)
72     row_mask[1:-1] = (pattern_w == 1)
73     col_mask = np.zeros(height + 2, dtype=bool)
74     col_mask[1:-1] = (pattern_h == 1)
75
76     acc[row_mask == 1, :] = 1
77     acc[:, col_mask == 1] = 1
78
79
80     map_data = simulator_core_no_omp.MapData(
81         rows=width,
82         cols=height,
83         times=times,
84         acc=acc
85     )
86
87
88     #Inicializado tensor de emisiones de entrada G
89     G = {f"{cartype}_{pollutant}": np.zeros((width+2, height+2, times+1),
dtype=np.float32)
90         for cartype in car_types
91         for pollutant in pollutants}
92
93     #Umbral estadistico para generacion de datos aleatorios
94     u = 0.1
95     for key in G:
96         #Peso (arbitrario) de emisiones segun el tipo de origen
97         if 'electric' in key:
98             emission = 0.0
99         elif 'diesel' in key:
100             emission = 5000.0
101         else: # gasoline
102             emission = 3000.0
103         if emission > 0:
104             for t in range(times + 1):
105                 rand = np.random.rand(width+2, height+2)
106                 #Filtrado de ruido segun umbral y acc
107                 f_rand = (rand < u) * acc
108                 pollution = f_rand * emission
109

```

```

110         G[key][:, :, t] = pollution
111
112
113     G_numpy_para_C = simulator_core_no_omp.dict_to_contiguous_4d_tensor(G
114     , map_data, car_types, pollutants)
115     return (acc, map_data, G, G_numpy_para_C)
116
117 def init_data_omp(width, height, times, car_types, pollutants):
118     #acc contiguo en memoria (con orden C) para acelerar tiempo
119     #transmision
120     acc = np.zeros((width+2, height+2), dtype=np.float32, order='C')
121
122     #Plantilla para aplicacion de patrones de edificios
123     template = np.zeros(100, dtype=np.float32)
124
125     template[0:4] = 1
126     template[96:100] = 1
127     template[48:52] = 1
128
129     template[25:27] = 1
130     template[73:75] = 1
131
132     #Aplicacion del patron
133     #np.ceil asegura copias del template, slicing [:width] corta al
134     #tamano exacto
135     pattern_w = np.tile(template, int(np.ceil(width / 100)))[:width]
136     pattern_h = np.tile(template, int(np.ceil(height / 100)))[:height]
137     row_mask = np.zeros(width + 2, dtype=bool)
138     row_mask[1:-1] = (pattern_w == 1)
139     col_mask = np.zeros(height + 2, dtype=bool)
140     col_mask[1:-1] = (pattern_h == 1)
141
142     acc[row_mask == 1, :] = 1
143     acc[:, col_mask == 1] = 1
144
145     map_data = simulator_core_omp.MapData(
146         rows=width,
147         cols=height,
148         times=times,
149         acc=acc
150     )
151
152     #Inicializado tensor de emisiones de entrada G
153     G = {f"{cartype}_{pollutant}": np.zeros((width+2, height+2, times+1),
154         dtype=np.float32)}

```

```

153         for cartype in car_types
154         for pollutant in pollutants}
155
156     #Umbral estadístico para generación de datos aleatorios
157     u = 0.1
158     for key in G:
159         #Peso (arbitrario) de emisiones según el tipo de origen
160         if 'electric' in key:
161             emission = 0.0
162         elif 'diesel' in key:
163             emission = 5000.0
164         else: # gasoline
165             emission = 3000.0
166         if emission > 0:
167             for t in range(times + 1):
168                 rand = np.random.rand(width+2, height+2)
169                 #Filtrado de ruido según umbral y acc
170                 f_rand = (rand < u) * acc
171                 pollution = f_rand * emission
172
173                 G[key][:, :, t] = pollution
174
175
176     G_numpy_para_C = simulator_core_omp.dict_to_contiguous_4d_tensor(G,
177     map_data, car_types, pollutants)
178     return (acc, map_data, G, G_numpy_para_C)

```

Código 4.3: Código auxiliar para pruebas.

A partir de esto, se ha implementado el código "precision.py" (código 4.4), en el que se inicializan los datos y se obtienen los resultados de cada simulador (tanto la implementación original en Python, como la modificada, como la versión en C con y sin OMP). Tras eso, se han comparado los resultados de las implementaciones de Python para determinar cuánta diferencia causa el reescalado de los datos, y los resultados de la versión modificada en Python con ambas versiones en C. Destacar el fijado de semilla antes de cada inicializado, ya que en caso contrario todos los datos se generarían de manera distinta para cada implementación.

```

1 from utils import *
2
3 import contaminationSeparated
4 import simulator_core_no_omp
5 import simulator_core_omp
6
7 car_types = [0, 1, 2] # 0 = EV, 1 = Petrol, 2 = Diesel
8 pollutants = ['CO2']#[ 'CO2', 'NOx', 'VOC', 'PMexhaust', 'PMexhaustprueba',
9 #No necesario (ni posible) pasar valores dimensionales y temporales a
10 float32 ya que se usan únicamente en indexación

```

```

10 width, height, times = 100, 100, 1000
11 num_car_types, num_pollutants = len(car_types), len(pollutants)
12 num_tensors = num_car_types * num_pollutants
13 #Para no cometer el error de comparar precisiones diferentes, se
    establece float32 como la precision general de todos los datos de tipo
    decimal (ya que
14 #es la que se usa en todo el codigo numpy)
15 gamma = np.float32(0.01)
16 delta = np.float32(0.1)
17 corner_factor = np.float32(1.0)
18 WN = np.float32(-0.2)
19 WE = np.float32(0.4)
20
21 #Inicializado datos sim_data implementacion no OMP
22 sim_data_n = simulator_core_no_omp.SimulationData(
23     delta=delta,
24     corner_factor=corner_factor,
25     gamma=gamma,
26     num_cartypes=num_car_types,
27     num_pollutants=num_pollutants,
28     WN=WN,
29     WE=WE
30 )
31 #Inicializado datos sim_data implementacion OMP
32 sim_data = simulator_core_omp.SimulationData(
33     delta=delta,
34     corner_factor=corner_factor,
35     gamma=gamma,
36     num_cartypes=num_car_types,
37     num_pollutants=num_pollutants,
38     WN=WN,
39     WE=WE
40 )
41
42
43 #####
44 ##          CODIGO PYTHON MODIFICADO          ##
45 #####
46
47 (...)
48
49
50 #####
51 ##          COMPARACION DE PRECISION          ##
52 #####
53
54 print("Calculando version Python modificada")

```

```

55 #Fijado de semilla para evitar datos de prueba distintos
56 np.random.seed(42)
57 acc, G = init_data_python(width, height, times, car_types, pollutants)
58 #Cambio de tipo para no usar int
59 acc = acc.astype(np.float32)
60 P_python = get_P_whole_prec(acc)
61
62
63 print("Calculando version Python original")
64 contaminationSeparated.width = width
65 contaminationSeparated.height = height
66 contaminationSeparated.times = times
67 contaminationSeparated.carTypes = car_types
68 contaminationSeparated.pollutants = pollutants
69 contaminationSeparated.WN = WN
70 contaminationSeparated.WE = WE
71 contaminationSeparated.gamma = gamma
72 contaminationSeparated.delta = delta
73 contaminationSeparated.corner_factor = corner_factor
74 contaminationSeparated.acc = acc
75 contaminationSeparated.G = G
76
77 P_python_o = contaminationSeparated.get_P_whole(acc)
78
79
80 print("Calculando version no OMP")
81 #Fijado de semilla para evitar datos de prueba distintos
82 np.random.seed(42)
83 _, map_data, _, G_numpy_para_C = init_data_no_omp(width, height, times,
84     car_types, pollutants)
85 P_n_omp = simulator_core_no_omp.get_P_whole_p(sim_data_n, map_data,
86     car_types, pollutants, G_numpy_para_C)
87
88
89 print("Calculando version OMP")
90 #Fijado de semilla para evitar datos de prueba distintos
91 np.random.seed(42)
92 _, map_data, _, G_numpy_para_C = init_data_omp(width, height, times,
93     car_types, pollutants)
94 P_omp = simulator_core_omp.get_P_whole_p(sim_data, map_data, car_types,
95     pollutants, G_numpy_para_C)
96
97
98 atol = 1e-6
99
100 print("\nComprobacion de diferencia entre codigo original y modificado")

```

```

98 for key in P_python:
99     tensor_py = P_python[key]
100     tensor_o = P_python_o[key]
101
102     diferencia_absoluta = np.abs(tensor_py - tensor_o)
103     max_diff = np.max(diferencia_absoluta)
104
105     print(f"Tensor {key}:")
106     print(f"  Diferencia maxima absoluta: {max_diff:.8e}")
107
108     if np.allclose(tensor_py, tensor_o, atol=atol, rtol=0.0):
109         print(f"  Los resultados superan la precision: {atol}")
110     else:
111         print(f"  Los resultados difieren con la precision: {atol}")
112         indices_error = np.unravel_index(np.argmax(diferencia_absoluta),
diferencia_absoluta.shape)
113         print(f"      Mayor discrepancia en coordenada {indices_error}")
114         print(f"      Valor Python modificado: {tensor_py[indices_error]}")
115     )
116     print(f"      Valor Python original:      {tensor_o[indices_error
117     ]}")
118
119 print("\nComprobacion de diferencia entre codigo Python y C sin OMP")
120 for key in P_python:
121     tensor_py = P_python[key]
122     tensor_c = P_n_omp[key]
123
124     #Transposicion de tensor de Python de (X, Y, Tiempo) a (Tiempo, X, Y)
para realizar comparacion
125     tensor_py_transpuesto = np.transpose(tensor_py, (2, 0, 1))
126
127     diferencia_absoluta = np.abs(tensor_py_transpuesto - tensor_c)
128     max_diff = np.max(diferencia_absoluta)
129
130     print(f"Tensor {key}:")
131     print(f"  Diferencia maxima absoluta: {max_diff:.8e}")
132
133     if np.allclose(tensor_py_transpuesto, tensor_c, atol=atol, rtol=0.0):
134         print(f"  Los resultados superan la precision: {atol}")
135     else:
136         print(f"  Los resultados difieren con la precision: {atol}")
137         indices_error = np.unravel_index(np.argmax(diferencia_absoluta),
diferencia_absoluta.shape)
138         print(f"      Mayor discrepancia en coordenada {indices_error}")
139         print(f"      Valor Python modificado: {tensor_py_transpuesto[
indices_error]})")

```

```

139         print(f"        Valor C sin OMP:        {tensor_c[indices_error]}")
140
141     print("\nComprobacion de diferencia entre codigo Python y C con OMP")
142     for key in P_python:
143         tensor_py = P_python[key]
144         tensor_c = P_omp[key]
145
146         #Transposicion de tensor de Python de (X, Y, Tiempo) a (Tiempo, X, Y)
147         #para realizar comparacion
148         tensor_py_transpuesto = np.transpose(tensor_py, (2, 0, 1))
149
150         diferencia_absoluta = np.abs(tensor_py_transpuesto - tensor_c)
151         max_diff = np.max(diferencia_absoluta)
152
153         print(f"Tensor {key}:")
154         print(f"    Diferencia maxima absoluta: {max_diff:.8e}")
155
156         if np.allclose(tensor_py_transpuesto, tensor_c, atol=atol, rtol=0.0):
157             print(f"    Los resultados superan la precision: {atol}")
158         else:
159             print(f"    Los resultados difieren con la precision: {atol}")
160             indices_error = np.unravel_index(np.argmax(diferencia_absoluta),
161             diferencia_absoluta.shape)
162             print(f"        Mayor discrepancia en coordenada {indices_error}")
163             print(f"        Valor Python modificado: {tensor_py_transpuesto[
164             indices_error]}")
165             print(f"        Valor C con OMP:        {tensor_c[indices_error]}")

```

Código 4.4: Código de benchmark de precisión.

4.1.3. Resultados de pruebas de precisión

Tras ejecutar el código detallado, se han obtenido los datos explicados a continuación.

Comparación entre implementación original en Python y modificada

Respecto a la imprecisión generada por lo explicado anteriormente en el código original, se han comparado los datos de salida de cada tensor de cada simulación y se han obtenido los resultados de la gráfica 4.1 (Resultados numéricos expuestos en la tabla 4.1).

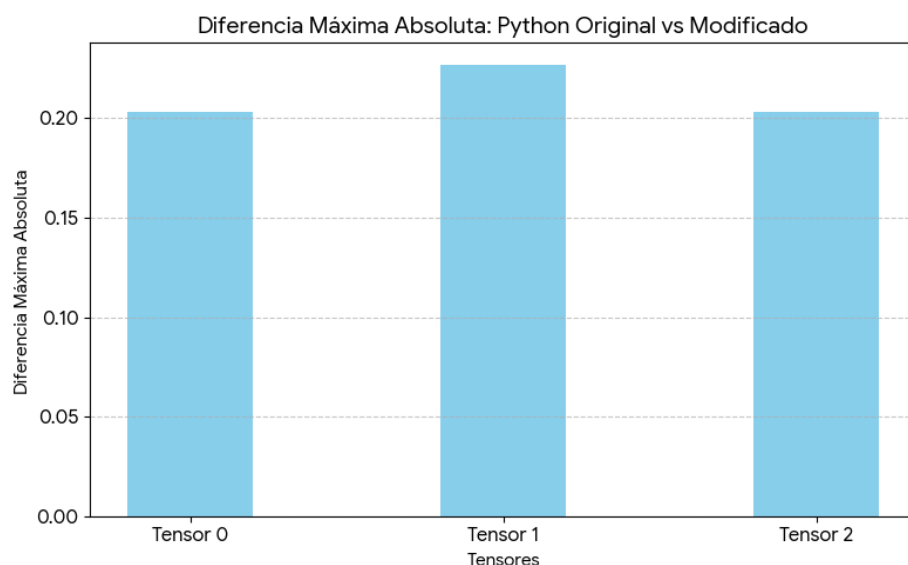


Figura 4.1: Diferencias máximas entre Python original y modificado, separado por tensores de simulación.

Comparando la implementación en Python original con la modificada para evitar redondeos indeseados (específicamente, se comparan los tensores del simulador por separado, dado que para cada par de tipo de contaminante y tipo de coche se generan entradas aleatorias diferentes), se tiene una diferencia numérica máxima en el segundo tensor de 0.227. De esta forma, se evidencia la divergencia mencionada por los redondeos automatizados del sistema, confirmando que la nueva versión soluciona los problemas mencionados.

Comparación entre implementación modificada en Python y versiones de C

A continuación, se muestra en la imagen 4.2 los casos de máxima imprecisión entre Python y C con OMP, y Python y C sin OMP (resultados numéricos expuestos en la tabla 4.1).

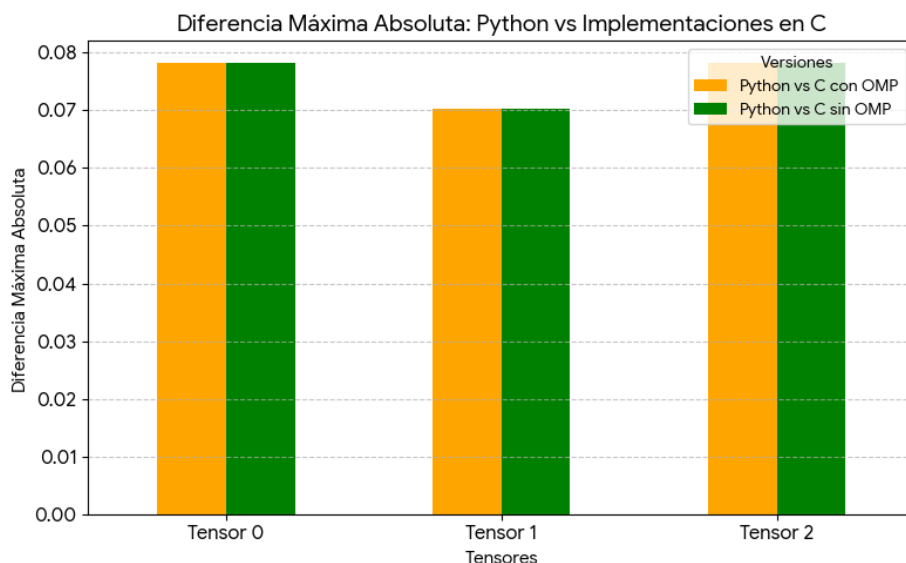


Figura 4.2: Diferencias máximas entre Python modificado y versiones de C, separado por tensores de simulación.

Como se ve en la gráfica, aparte de que las versiones en C con y sin OMP son equivalentes en cuanto a resultado (lo cual es coherente con lo que se ha estado explicando); se tiene un error máximo de 0.078.

Si bien esto puede hacer parecer que la implementación en C es errónea, la causa del error viene arraigada en el uso de valores de tipo coma flotante en el algoritmo. Esto es debido a que la propiedad asociativa en operaciones de punto flotante no se cumple, de forma que si una operación equivalente en resultado se realiza en distinto orden, se obtienen resultados con diferencias a nivel de bit. Dado que el algoritmo es iterativo respecto a estados anteriores, este error se acumula, llegando al estado final como un error medible.

Dado que la implementación realiza optimizaciones mediante el reordenado de las operaciones aritméticas (tanto manual como automático por el compilador), la única solución completa a esto sería realizar los cálculos de manera exacta a Python, lo cual degradaría considerablemente el rendimiento actual. Otra forma de minimizar el problema sería pasar los datos a tipos con precisiones más altas ("float64", "float128", etcétera), pero tendría como contraparte que, además de que los datos ocuparían mucho más espacio en memoria, se reduciría el rendimiento notoriamente al consistir en operaciones más costosas.

Debido a esto, se ha determinado que la implementación es válida, ya que cualquier otra solución implicaría una pérdida severa del rendimiento del sistema.

Comparación	Tensor	Coords	Dif. Máxima
código original y modificado	Tensor 0	(52,88,579)	2.03125000e-01
código original y modificado	Tensor 1	(100,72,568)	2.26562500e-01
código original y modificado	Tensor 2	(100,71,721)	2.03125000e-01
código Python y C sin OMP	Tensor 0	(996,52,40)	7.81250000e-02
código Python y C sin OMP	Tensor 1	(300,100,94)	7.03125000e-02
código Python y C sin OMP	Tensor 2	(649,52,44)	7.81250000e-02
código Python y C con OMP	Tensor 0	(996,52,40)	7.81250000e-02
código Python y C con OMP	Tensor 1	(300,100,94)	7.03125000e-02
código Python y C con OMP	Tensor 2	(649,52,44)	7.81250000e-02

Tabla 4.1: Tablas de diferencias máximas absolutas.

4.1.4. Imprecisión de IEEE 754

Algo que destaca de los resultados obtenidos es el hecho de que tanto el tensor 0 como el tensor 2 tienen los mismos resultados, lo cual parece contradecir lo expuesto con anterioridad, pero si se miran las coordenadas recogidas en 4.1 para cada diferencial máximo, se aprecia que este varía según el tensor. Esto sugiere que, en realidad, se está sufriendo nuevamente una imprecisión generada por el estándar IEEE 754 [39].

Si se observa cómo se implementa matemáticamente el tipo "float32" en el estándar IEEE 754, se obtiene la siguiente fórmula:

$$(-1)^{\text{signo}} \times 1.\text{mantisa} \times 2^{(\text{exponente}-127)}$$

Esta se compone de 3 componentes:

- **Signo:** signo de valor numérico, ocupa 1 bit.
- **Mantisa:** dígitos significativos del número. Ocupa 23 bits, lo que ocasiona que sólo pueda representar 2^{23} (8388608) valores fraccionarios únicos.
- **Exponente:** multiplicador de escala en base 2, ocupa 8 bits. Acota el valor en un rango delimitado por potencias de 2 consecutivas.

A raíz de esto, y teniendo en cuenta que los datos generados están por encima de 65536 (2^{16}) y por debajo de 131072 (2^{17}), el exponente usado para la comparación de resultados es 16, de manera que el tamaño total del intervalo de valores posibles en esta sección es de 65536. Dado que, según el estándar IEEE 754, todas las combinaciones permitidas por la mantisa deben repartirse equitativamente a lo largo de este intervalo, se obtiene que ha de haber una diferencia mínima entre valor y valor (conocida como "ULP") de 0.0078125 (ya que es lo que se obtiene a partir de $65536/8388608$).

Si se analiza, por ejemplo, el error máximo entre Python y C en los tensores 0 y 2 ($7,8125 \times 10^{-2}$), se aprecia que es un múltiplo del ULP obtenido ($0,078125/0,0078125 = 10$). De esta forma, se demuestra que los resultados se

encuentran afectados por esta limitante estandarizada, propagándose a todos los resultados y haciendo que, en realidad, todas las diferencias se encuentren en un intervalo de imprecisión de 0.0078125.

4.2. Rendimiento de la implementación

Tras haber demostrado que la implementación en C es válida, se procede a la prueba de rendimiento del mismo. Para ello, se comparará la versión original de Python con las versiones de C con y sin OMP, además de realizar una prueba comparando el rendimiento de C con OMP con distinta cantidad de hilos, para evaluar la escalabilidad del algoritmo.

Sumado a eso, cada prueba se realizará dos sistemas diferentes: un ordenador doméstico y de uso común, y el clúster HPC de I3US [23].

4.2.1. Entornos de prueba

Para la realización de pruebas de rendimiento, se tienen dos entornos de hardware para poder comparar el rendimiento del simulador en diferentes situaciones. Específicamente, se tienen:

- **Ordenador doméstico:** se ha hecho uso de un ordenador doméstico para probar el simulador en un sistema reducido en cuanto a computación. Específicamente, el sistema cuenta con un procesador AMD Ryzen 7 7700X con 8 núcleos, y 32GB de memoria RAM DDR5 (más detalles en la tabla 4.2).
- **Clúster HPC I3US:** se ha conseguido acceso al clúster de alta prestaciones del Instituto Universitario de Investigación de Ingeniería Informática (I3US) y la Unidad de Excelencia SCORE (Smart Computer Systems Research and Engineering Lab) de la Universidad de Sevilla. Este cuenta con múltiples módulos de computación (CPU, GPU, FPGA), pero para computación en CPU, se cuentan con 10 nodos de computación, 4 de ellos con 56 núcleos AMD EPYC 7453 y los otros 6 con 128 núcleos AMD EPYC 7713 (los que se usarán). Específicamente, se trata de un Lenovo ThinkSystem SR645, el cual cuenta con dos procesadores físicos de 64 núcleos cada uno, sumando un total de 128 núcleos basados en la arquitectura Zen 3 (más detalles en la tabla 4.2).

Algo crítico a considerar es que, al contrario que un sistema doméstico convencional, este sistema presenta una arquitectura "dual-socket" con topología NUMA ("Non-Uniform Memory Access"). Esto implica que los 2TB de memoria RAM DDR4 no están centralizados, sino que se segmenta físicamente según el procesador, de manera que cada CPU tiene su sección de memoria específica (conectándose entre sí mediante el bus de interconexión "Infinity Fabric", el cual comunica los distintos procesadores). Debido a esto, aunque a nivel de software se trata la memoria como si estuviera unificada, el tiempo de acceso de un núcleo a su memoria local es significativamente menor que el acceso a la memoria conectada al otro procesador.

Característica	Ordenador Doméstico	Clúster HPC I3US
Modelo CPU	AMD Ryzen 7 7700X	2x AMD EPYC 7713
Arquitectura	Zen 4 (Soporte AVX-512)	Zen 3 (Soporte AVX2)
Núcleos / Hilos	8 / 16	128 / 256
Memoria RAM	32 GB DDR5	2 TiB DDR4 (NUMA)
Velocidad Memoria	~ 5200 MT/s (83 GB/s)	2933 MT/s
Topología	Monolítica (<i>Single-Socket</i>)	Distribuida (<i>Dual-Socket</i>)

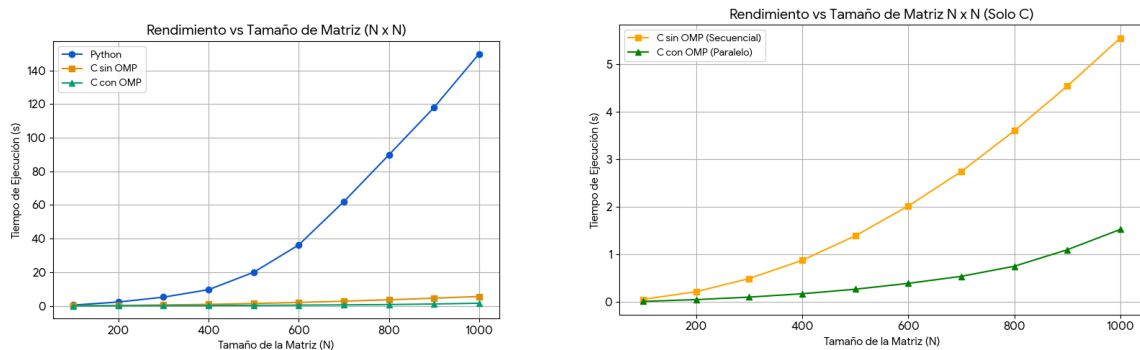
Tabla 4.2: Especificaciones técnicas de los entornos de prueba.

4.2.2. Rendimiento comparativo entre implementaciones

Para la evaluación del rendimiento comparando las implementaciones del simulador, se han realizado dos pruebas: el aumento espacial de los datos iniciales y el aumento temporal.

Sistema doméstico

Para la prueba de rendimiento con aumento espacial, se han obtenido los resultados proporcionados en las gráficas 4.3 y en la tabla 4.3.



(a) Resultados con implementación Python. (b) Resultados sin implementación Python.

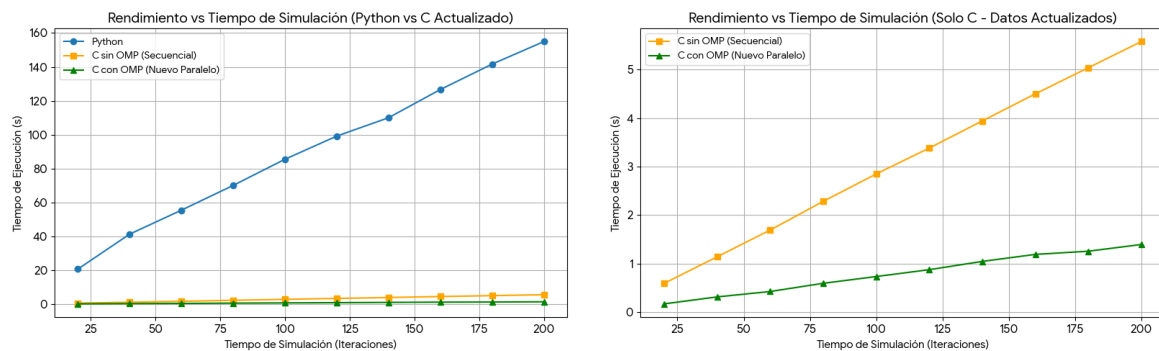
Figura 4.3: Resultados de prueba de crecimiento espacial en sistema doméstico.

El tiempo de ejecución crece cuadráticamente respecto al tamaño en todos los sistemas, lo cual es coherente con el hecho de que se está aumentando el tamaño espacial en 2 dimensiones.

Tamaño ($N \times N$)	Python (s)	C sin OMP (s)	C con OMP (8 hilos) (s)
100	0.45	0.06	0.01
200	2.26	0.22	0.05
300	5.15	0.49	0.10
400	9.62	0.88	0.17
500	19.94	1.39	0.27
600	36.24	2.02	0.39
700	61.98	2.74	0.54
800	89.81	3.60	0.75
900	117.95	4.54	1.10
1000	149.90	5.55	1.53

Tabla 4.3: Tiempos de ejecución según el tamaño de la matriz en sistema doméstico.

Pasando a la prueba de rendimiento con aumento temporal, se han obtenido los resultados proporcionados en las gráficas 4.4 y en la tabla 4.4.



(a) Resultados con implementación Python. **(b)** Resultados sin implementación Python.

Figura 4.4: Resultados de prueba de crecimiento temporal en sistema doméstico.

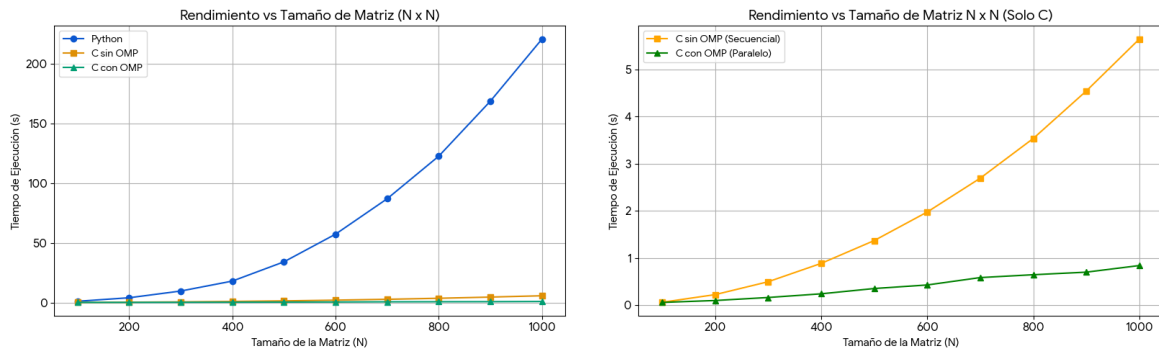
El tiempo de ejecución crece linealmente respecto a tiempo de entrada de simulación en todos los sistemas, lo cual es coherente con el hecho de que se está aumentando únicamente una dimensión de los datos.

Tiempo Sim.	Python (s)	C sin OMP (s)	C con OMP (8 hilos) (s)
20	20.70	0.59	0.17
40	41.32	1.14	0.32
60	55.47	1.69	0.43
80	70.09	2.28	0.60
100	85.53	2.85	0.74
120	99.19	3.38	0.88
140	110.02	3.94	1.05
160	126.67	4.50	1.19
180	141.65	5.04	1.26
200	154.96	5.58	1.40

Tabla 4.4: Tiempos de ejecución según el tiempo de simulación en sistema doméstico.

Clúster de altas prestaciones

Para la prueba de rendimiento con aumento espacial, se han obtenido los resultados proporcionados en las gráficas 4.5 y en la tabla 4.5.



(a) Resultados con implementación Python. (b) Resultados sin implementación Python.

Figura 4.5: Resultados de prueba de crecimiento espacial en clúster.

El tiempo de ejecución crece cuadráticamente respecto al tamaño en Python y C sin OMP. Sin embargo, al observar la implementación OMP no se ve un crecimiento parecido, sino uno lineal. Esto se debe a que el cuello de botella de la ejecución se encuentra en el tiempo de cabecera de control ("overhead") de los hilos del programa. Para visualizarlo, se plantea la siguiente ecuación como representación del tiempo de ejecución de un programa (derivada de la ley de Amdahl):

$$T_{\text{total}}(t) = (1 - p) + \frac{p}{t}$$

Siendo p el tiempo de ejecución de la parte de código paralelizable y t el número de hilos. Se procede a decomponer el tiempo de ejecución de la sección de código no paralelizable ($1 - p$) en lo siguiente:

$$T_{\text{total}}(t) = T_{\text{sec}} + T_{\text{control}} + \frac{p}{t}$$

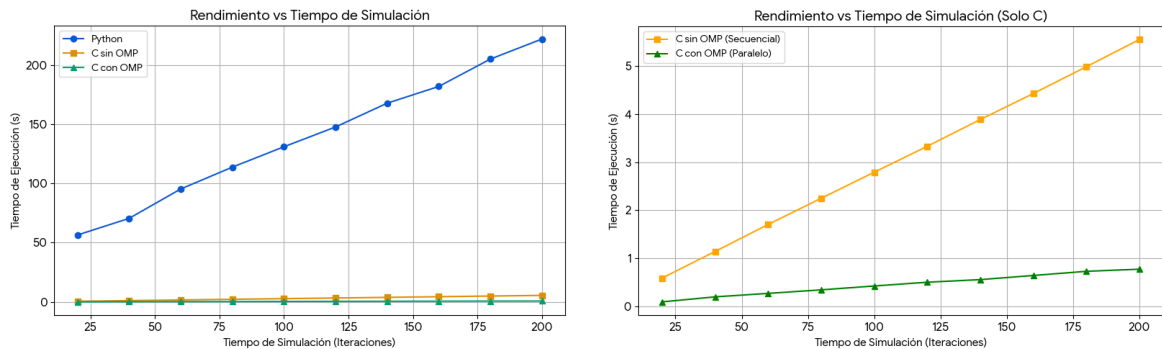
Siendo T_{control} el tiempo de overhead del programa y T_{sec} el resto del código secuencial.

De esta forma, lo que ocurre es que el tiempo de overhead (cuyo crecimiento es lineal dado que no se ve afectado por el crecimiento cuadrático sino por el crecimiento lineal de hilos usados, que en cuyo caso es nulo porque se ha dejado fijo a 24) es notoriamente superior al del tiempo paralelizable. La justificación técnica de tal diferencia reside en la sincronización entre núcleos, la cual tiene una latencia de varios ordenes de magnitud superior al tiempo de calculo en esas escalas de datos [40].

Tamaño ($N \times N$)	Python (s)	C sin OMP (s)	C con OMP (24 hilos) (s)
100	1.026	0.060	0.057
200	3.964	0.223	0.098
300	9.596	0.496	0.162
400	17.924	0.885	0.240
500	34.026	1.368	0.352
600	57.183	1.973	0.427
700	86.805	2.690	0.584
800	122.458	3.532	0.644
900	168.484	4.539	0.699
1000	220.454	5.639	0.839

Tabla 4.5: Tiempos de ejecución según el tamaño de la matriz en clúster.

Pasando a la prueba de rendimiento con aumento temporal, se han obtenido los resultados proporcionados en las gráficas 4.6 y en la tabla 4.6.



(a) Resultados con implementación Python. (b) Resultados sin implementación Python.

Figura 4.6: Resultados de prueba de crecimiento temporal en clúster.

El tiempo de ejecución crece linealmente respecto a tiempo de entrada de simulación en todos los sistemas, lo cual es coherente con el hecho de que se está aumentando únicamente una dimensión de los datos.

Tiempo Sim.	Python (s)	C sin OMP (s)	C con OMP (24 hilos) (s)
20	56.501	0.590	0.094
40	70.511	1.144	0.198
60	95.309	1.705	0.270
80	113.745	2.250	0.344
100	130.926	2.794	0.425
120	147.634	3.332	0.503
140	167.681	3.890	0.557
160	181.822	4.430	0.644
180	204.917	4.989	0.730
200	221.838	5.554	0.775

Tabla 4.6: Tiempos de ejecución según el tiempo de simulación en clúster.

4.2.3. Rendimiento comparativo entre número de hilos

Para evaluar la escalabilidad del programa, se han hecho múltiples pruebas de rendimiento en ambos entornos de pruebas variando el número de hilos, de manera de se pueda determinar cuál el punto óptimo del sistema en cuanto a paralelización mediante hilos.

Sistema doméstico

Para el sistema doméstico, se han obtenido los resultados proporcionados en la gráfica 4.7 y en la tabla 4.7.

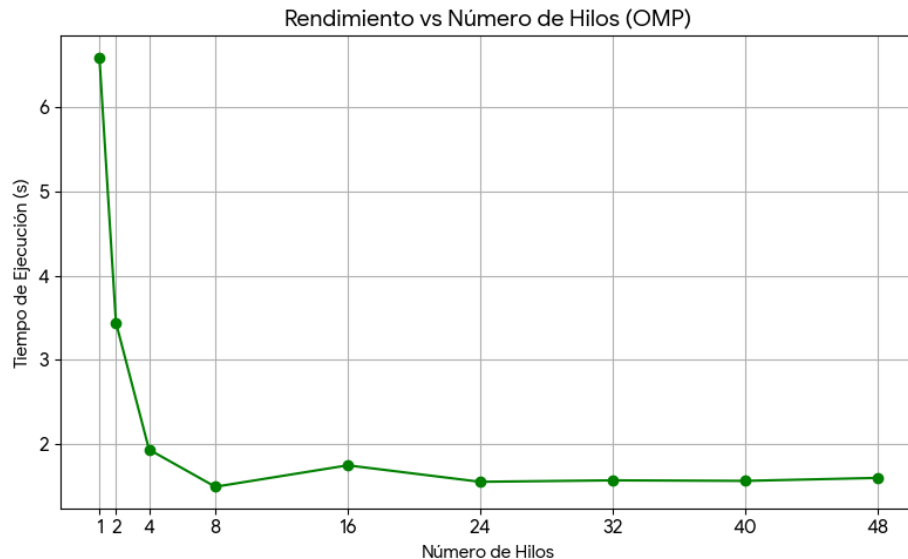


Figura 4.7: Tiempo de ejecución de programa con diferentes cantidades de hilos en el sistema doméstico.

Como se observa en la imagen 4.7, el punto óptimo en cuanto a la escalabilidad del sistema está en 8 hilos, lo cual concuerda con el hecho de que se tienen 8 núcleos

físicos. Sin embargo, se puede apreciar un pico en el tiempo de ejecución al momento de usar 16 hilos.

La razón de esto es debido a que el procesador del sistema doméstico (AMD Ryzen 7 7700X) cuenta con 8 núcleos físicos y 16 hilos lógicos mediante la tecnología SMT ("Simultaneous Multithreading") [41]. Al ejecutar el programa con 8 hilos, cada hilo cuenta con un núcleo físico dedicado, de manera que cada proceso tiene su propia memoria caché y unidad de ejecución. Sin embargo, al usar 16 hilos, el sistema operativo asigna dos hilos a cada núcleo físico por la tecnología SMT, lo cual está pensado para aprovechar los tiempos de espera que se puedan generar en un núcleo con un único hilo, pero como el programa paralelizado consiste en un proceso de computación intensiva, nunca hay tiempos de espera. Además de eso, dado que se hay el doble de llamadas a la memoria por el hilo extra en el núcleo, se generan cuellos de botella en la misma que ralentizan aún más el programa. Para valores superiores a 16 hilos, el tiempo se estabiliza con pequeñas variaciones debidas a la sobrecarga ("overhead") adicional que genera el cambio de contexto del sistema operativo entre los hilos.

Nº Hilos	1	2	4	8	16	24	32	40	48
C con OMP (s)	6.59	3.44	1.94	1.49	1.75	1.55	1.57	1.56	1.60

Tabla 4.7: Tiempos de ejecución según la cantidad de hilos de OpenMP en el sistema doméstico.

Clúster de altas prestaciones

Para el clúster, se han obtenido los resultados proporcionados en la gráfica 4.8 y en la tabla 4.8.

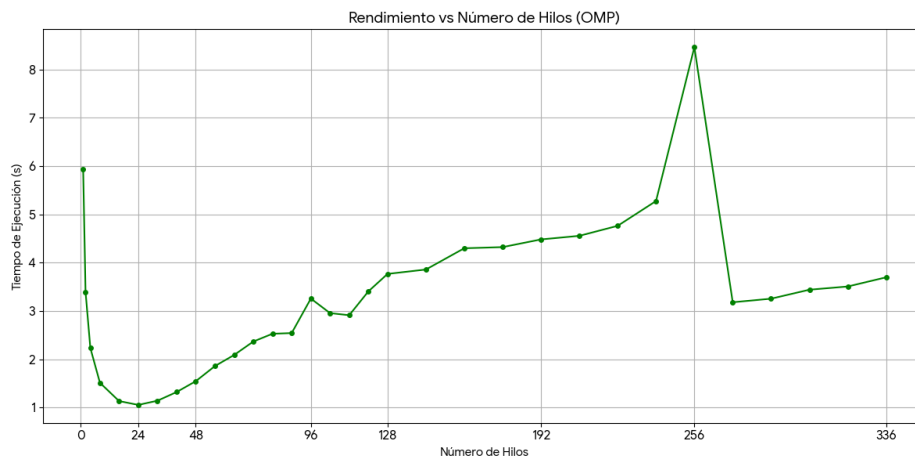


Figura 4.8: Tiempo de ejecución de programa con diferentes cantidades de hilos en el clúster.

Como se observa en la imagen 4.7, el punto óptimo en cuanto a la escalabilidad del sistema está en 24. Sin embargo, los resultados muestran no sólo un

degradamiento acentuado del rendimiento a medida que se emplean más hilos, sino que se observa un pico en 256 hilos como el peor de la prueba de rendimiento el cual deriva posteriormente en una mejora temporal del rendimiento.

Como ya se mencionó, la memoria posee una arquitectura NUMA, de manera que cada CPU tiene su propia sección de la memoria RAM. Esto implica que, al momento de asignar memoria, se sigue una política "first touch", en la que el bloque de memoria se asigna únicamente para una de las secciones de la memoria (de manera que sólo una de las CPUs tiene un acceso directo a la misma). Con esto en mente, es necesario recordar que la implementación realizada, debido a que está pensada para no hacer un uso excesivo de la capacidad de la memoria, el espacio de memoria dedicada a los datos de entrada en Python es compartida físicamente al código en C. Y dado que Python es mononúcleo, este espacio de memoria se encuentra únicamente en una de las dos secciones de la RAM, haciendo por tanto que sólo sea accesible de manera directa para una de las CPUs. Si la otra CPU requiere en algún momento de los datos de entrada se verá forzada a usar el bus "Infinity Fabric", añadiendo latencia a la operación de acceso a los datos y degradando considerablemente la ejecución del algoritmo.

Además de eso, la arquitectura del procesador AMD EPYC 7713 incorpora una agrupación de núcleos llamada CCD o "Core Compute Die" (no estando incorporado en el AMD Ryzen 7 7700X). Esto consiste en que los núcleos del procesador se agrupan en bloques de 8 elementos, de forma que cada CCD tiene su segmento de memoria caché L3. Dado que los CCD se conectan entre sí mediante un bus interno de tipo "Infinity Fabric", si un núcleo requiere acceder a un dato que está en una memoria caché externa al CCD, se añadirá una latencia grave a la operación realizada.

En otras palabras, la CPU incorpora una arquitectura NUMA interna para la gestión y comunicación de los hilos por medio de una política "first touch". Esto genera que la implementación se vea aún más limitada en cuanto a eficiencia, dado que la gestión de memoria interna del simulador en C está planteada para que se reserven los recursos de la misma de forma externa a los hilos de ejecución, con el objetivo de que esta sea compartida y utilizada por todos los hilos, optimizando así el espacio de memoria requerida para la simulación. De esta forma, sólo habrá un CCD que tenga acceso a todos los datos de la simulación, mientras que el resto deberán usar el bus "Infinity Fabric", afectando gravemente al rendimiento final.

Respecto a la degradación del tiempo con 256 hilos, esta se explica de forma similar a la del tiempo de 16 hilos en el sistema doméstico (es decir, por el SMT), pero a esto se le añade que, dada la cantidad de hilos ejecutándose a la vez, se da un fenómeno conocido como "false sharing", en el que se da el caso en el que los hilos escriben continuamente en direcciones contiguas alojadas en la misma línea de caché, forzando al hardware a invalidar y recargar esos bloques continuamente. Además, OMP tiene una política de espera activa por defecto, de manera que todos los hilos finalizados se congelan esperando al resto mientras saturan las comunicaciones internas de la CPU y el bus entre CPUs ("Infinity Fabric") con peticiones de sincronización en cada barrera implícita. Esto es aplicable a la mayoría de casos de medición de hilos de la prueba de rendimiento, pero como en

este punto se está forzando SMT, el problema se acentúa considerablemente.

Finalmente, cuando el sistema sobrepasa la capacidad física del procesado en cuanto al manejo de hilos (caso conocido como "oversubscription"), el sistema operativo desactiva el SMT y empieza a virtualizar los hilos (gestionando mediante software la activación y desactivación de los mismos y su entorno), liberando carga de los procesadores y optimizando las comunicaciones.

Nº Hilos	1	2	4	8	16	24	32	40
C con OMP (s)	5.94	3.39	2.23	1.51	1.13	1.06	1.14	1.32
Nº Hilos	48	56	64	72	80	88	96	104
C con OMP (s)	1.55	1.86	2.09	2.37	2.53	2.54	3.26	2.96
Nº Hilos	112	120	128	144	160	176	192	208
C con OMP (s)	2.91	3.41	3.77	3.86	4.30	4.32	4.48	4.56
Nº Hilos	224	240	256	272	288	304	320	336
C con OMP (s)	4.76	5.28	8.46	3.18	3.25	3.44	3.51	3.70

Tabla 4.8: Tiempos de ejecución según la cantidad de hilos de OpenMP en el clúster.

4.3. Resultados de aceleración

Con todos los datos obtenidos, se puede determinar que pese a que el sistema acelera de forma sustancial la implementación de Python original, no se está aprovechando la totalidad del hardware en cuanto a capacidad computacional pura, viéndose limitado por las capacidades de la memoria (sobre todo en el clúster).

Si se compara la aceleración de las implementaciones respecto a la original en Python, para cada entorno por separado (tomando como base la prueba de escalado máximo espacial de $1000 \times 1000 \times 200$), se obtiene la siguiente tabla:

Entorno de prueba	Speedup C secuencial	Speedup C con OMP
Ordenador Doméstico	27,01	97,97 (8 hilos)
Clúster HPC I3US	39,09	262,76 (24 hilos)

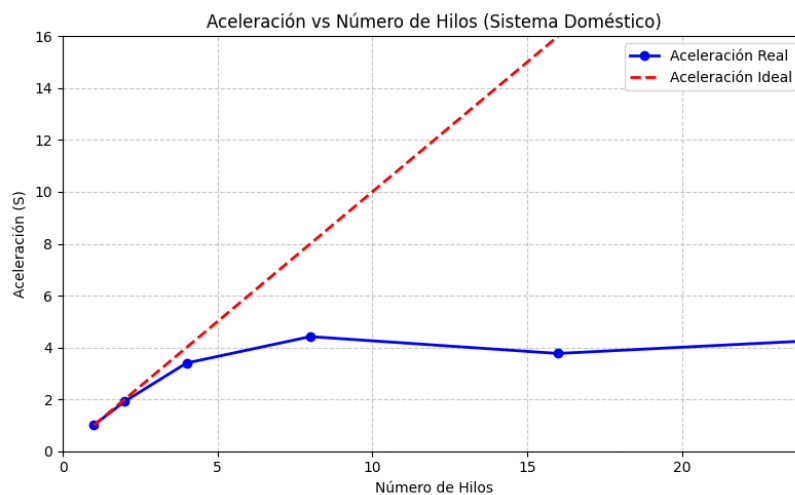
Tabla 4.9: Factores de aceleración respecto a la versión en Python para una matriz de tamaño 1000x1000.

Los datos revela que la migración del algoritmo de Python a C supone el mayor salto de rendimiento inicial, logrando aceleraciones casi 40 veces superiores (en el clúster) gracias a la eliminación del sobrecoste del intérprete y a la gestión contigua de la memoria. Además, la implementación de OpenMP mejora tal rendimiento,

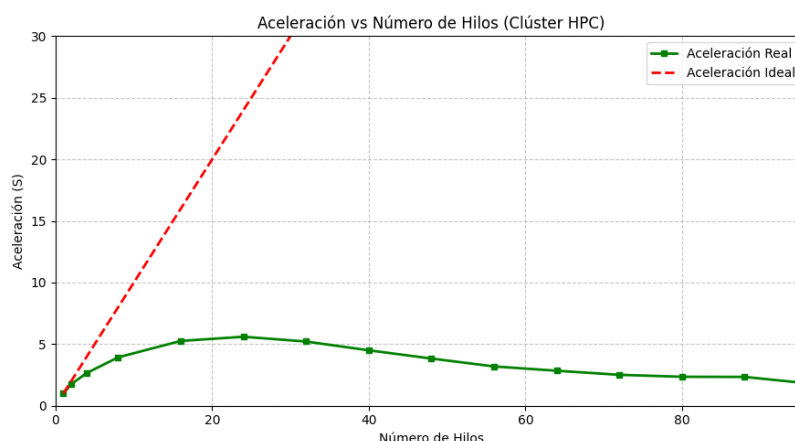
multiplicando casi por 100 y 260 la velocidad en el ordenador doméstico y en el clúster respectivamente.

Se ha de aclarar que, la razón por la que la mejora en rendimiento es superior en el clúster es debido a que la capacidad de cómputo en mononúcleo está notoriamente por debajo de la del entorno doméstico, al tener una arquitectura de núcleo más antigua y operar en una frecuencia notoriamente inferior (el Ryzen 7 7700X tiene una frecuencia estándar de 4.5GHz, mientras que el EPYC 7713 tiene una de 2GHz). De esta manera, dado de Python es completamente mononúcleo, en cuando se paraleliza el código se aprovechan las verdaderas capacidades del sistema que son el ancho de banda de transferencia y la cantidad de núcleos paralelos disponible.

Respecto a la aceleración generada por los hilos, se puede representar mediante las gráficas 4.9 (con datos en tablas 4.10 y 4.11), comparando los resultados con la aceleración ideal en cada caso:



(a) Resultados de aceleración en sistema doméstico.



(b) Resultados de aceleración en clúster.

Figura 4.9: Aceleración de hilos respecto la ideal.

Como es de esperar, conforme el número de hilos aumenta, la aceleración del

sistema se aleja de la aceleración ideal. Específicamente, en el sistema doméstico, se alcanza una aceleración con 8 hilos de 4.42 (es decir, una mejora del 342 %) respecto a la implementación sin OMP, mientras que en el clúster se obtiene con 24 hilos un 5.60 (o sea, una mejora del 460 %).

Con esto se puede determinar que la escalabilidad del programa llega a su punto máximo exactamente en el límite físico en el sistema doméstico (descartando la implementación con SMT por lo explicado anteriormente); mientras que en el clúster esta se queda por debajo del límite físico de los procesadores, debido a los limitantes de la arquitectura explicados (overhead, comunicación entre procesadores, saturación de comunicación de la caché, etcétera).

Nº Hilos	1	2	4	8	16	24	32	40	48
Aceleración Real	1.00	1.92	3.40	4.42	3.77	4.25	4.20	4.22	4.12
Aceleración Ideal	1	2	4	8	16	24	32	40	48

Tabla 4.10: Comparativa de aceleración real frente a ideal en el sistema doméstico.

Nº Hilos	1	2	4	8	16	24	32	40
Acel. Real	1.00	1.75	2.66	3.93	5.26	5.60	5.21	4.50
Acel. Ideal	1	2	4	8	16	24	32	40

Nº Hilos	48	56	64	72	80	88	96	104
Acel. Real	3.83	3.19	2.84	2.51	2.35	2.34	1.82	2.01
Acel. Ideal	48	56	64	72	80	88	96	104

Nº Hilos	112	120	128	144	160	176	192	208
Acel. Real	2.04	1.74	1.58	1.54	1.38	1.38	1.33	1.30
Acel. Ideal	112	120	128	144	160	176	192	208

Nº Hilos	224	240	256	272	288	304	320	336
Acel. Real	1.25	1.13	0.70	1.87	1.83	1.73	1.69	1.61
Acel. Ideal	224	240	256	272	288	304	320	336

Tabla 4.11: Comparativa de aceleración real frente a ideal en el clúster de altas prestaciones.

Finalmente, comparando los sistemas entre sí, si se toman las 2 mejores implementaciones (OMP) en el mismo caso de entrada ($1000 \times 1000 \times 200$), se obtiene una aceleración de:

$$S_{\text{sistemas}} = \frac{T_{\text{Doméstico OMP}}}{T_{\text{Clúster OMP}}} = \frac{1,53 \text{ s}}{0,839 \text{ s}} \approx \times 1,82 \quad (4.1)$$

La comparativa directa entre sistemas indica que el clúster, a pesar de poseer 128 núcleos físicos, sólo ofrece una mejora relativa del 82 % frente al ordenador doméstico de 8 núcleos bajo el tamaño de prueba estudiado. Este comportamiento subraya los cuellos de botella encontrados en el clúster explicados anteriormente.

5. Planificación

A continuación, se desarrolla en profundidad la planificación específica para el proyecto. Esta se establece en un marco temporal de, aproximadamente, 250 horas totales de trabajo.

5.1. Planificación temporal

La planificación temporal define cómo se desglosa el trabajo y el tiempo asignado a cada tarea estructural del documento para llegar desde el código original en Python hasta el entorno paralelizado en CPU de alto rendimiento.

5.1.1. Gestión de paquetes de trabajo (WBS)

Como se muestra en el esquema [5.1](#), el proyecto se divide en 6 fases troncales, segmentadas en Paquetes de Trabajo (PT) más específicos:

- **Análisis y Estudio Previo:** abarca tanto el estudio en profundidad del algoritmo original (30 horas) como el análisis las tecnologías implementadas (15 horas).
- **Diseño de la Solución:** diseño de la estructuras de datos (15 horas) y del puente de comunicación interlenguaje (10 horas).
- **Implementación del Código Base en C:** implementación completa del simulador en C (45 horas).
- **Paralelización e Integración:** paralelización del código (20 horas) e integración de intercomunicación de lenguajes mediante Cython (40 horas).
- **Pruebas y Validación:** comprobación de precisiones entre implementaciones (15 horas) y pruebas de rendimiento y eficiencia (20 horas).
- **Documentación Final:** redacción y documentación del proyecto en forma de memoria académica (40 horas).

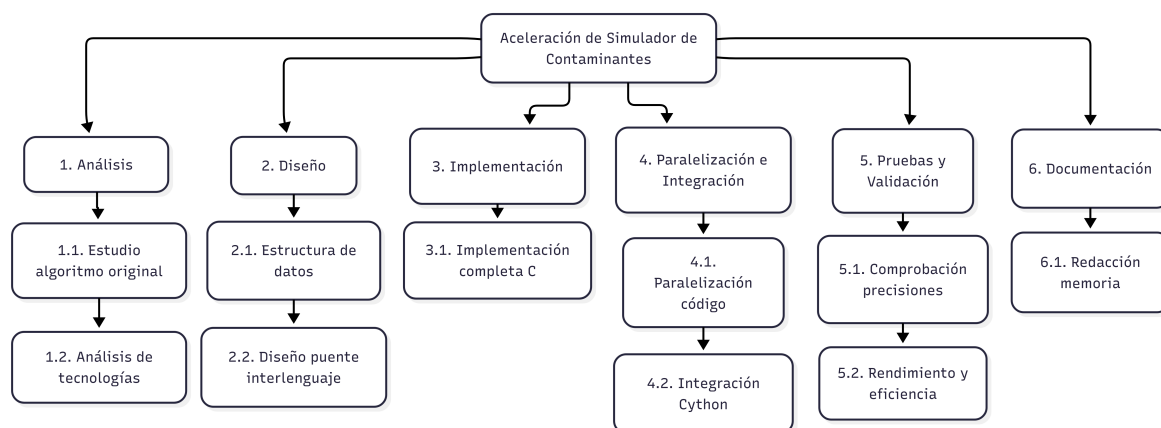


Figura 5.1: WBS del proyecto.

5.1.2. Diagrama de Gantt

La orquestación de estos paquetes de trabajo genera un calendario consolidado que ilustra la progresión cronológica de todas las tareas a lo largo del tiempo de vida del TFG. Se introduce a continuación en la gráfica 5.2 el diagrama.

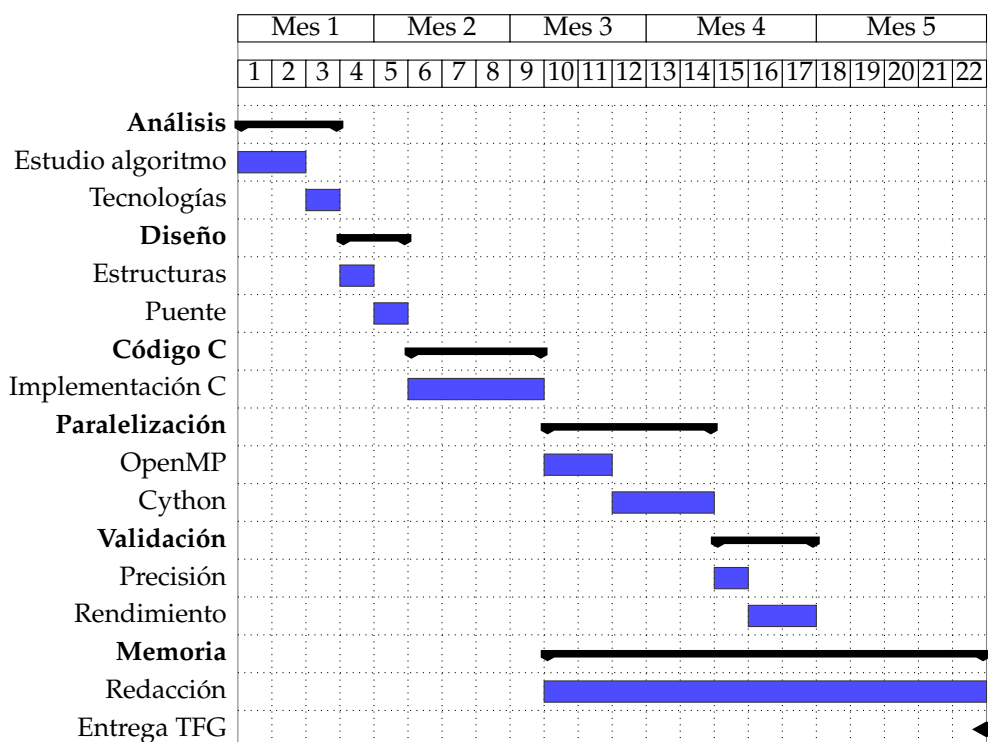


Figura 5.2: Diagrama de Gantt del proyecto.

5.2. Planificación económica

En la tabla 5.1, para dotar al proyecto de un prisma profesional, se han de cuantificar los recursos como si este desarrollo de aceleración de software formase

parte de un contrato de ingeniería para el proyecto SAINEVRA. Los costes asociados a las herramientas software son cero, puesto que se han empleado lenguajes y compiladores Open Source o libres de licencia académica (Python, GCC, Cython, OpenMP). También se han anulado costos relacionados con el clúster, dado que el proyecto tiene acceso al mismo de forma gratuita.

Tabla 5.1: Presupuesto detallado del proyecto

Concepto	Unidades / Tiempo	Coste Unit.	Subtotal
<i>1. Recursos Humanos</i>			
Ingeniero Junior	250 horas	15,00 €/h	3.750,00 €
<i>2. Hardware e Infraestructura</i>			
PC Personal	1	800,00 €	800,00 €
Uso de Clúster HPC (I3US)	1	0,00 €	0,00 €
TOTAL ESTIMADO			4550,00 €

6. Conclusiones y trabajo a futuro

6.1. Conclusiones

Tras el desarrollo y evaluación del proyecto, el proyecto ha logrado abordar de todos los objetivos que se plantearon al inicio, obteniendo principalmente las siguientes conclusiones:

- **Equivalencia funcional:** se ha logrado proporcionar una implementación equivalente al original desarrollado en Python, pudiéndose integrar en un entorno de Python vanilla (es decir, sin librerías externas ni modificaciones respecto a la implementación CPython) dado el enfoque realizado de integración con el lenguaje.
- **Mejora sustancial del rendimiento:** se ha demostrado que la nueva implementación es muy superior a la original en cuanto a su eficiencia. Esta aceleración se ha obtenido gracias a la aplicación de múltiples técnicas de optimización, tanto a nivel de funcionamiento del algoritmo, como a nivel de instrucciones (mediante SIMD), como a nivel de paralelización de hilos en CPU (mediante OpenMP); con resultados de mejora del 9697 % ($\times 97,97$) en el sistema doméstico usado y del 26176 % ($\times 262,76$) en el clúster.
- **Limitaciones por funcionamiento de la memoria:** pese a los buenos resultados obtenidos, se ha demostrado la existencia de un cuello de botella generado por la disparidad entre la alta velocidad de computación de la CPU y la latencia y ancho de banda de la memoria. Como consecuencia, actualmente no se están logrando aprovechar al máximo las capacidades reales de cómputo de los sistemas.
- **Arquitectura HPC:** aunque el clúster posee unas prestaciones teóricas que son muy superiores a las de un ordenador convencional, su compleja arquitectura y funcionamiento exige aplicar un refinamiento específico para el sistema en cuanto a las optimizaciones de gestión de datos para no penalizar el rendimiento del programa.

6.2. Trabajo a futuro

Dado que las conclusiones han revelado el peso que tiene la memoria en el rendimiento final del simulador, el futuro de esta herramienta debe orientarse hacia tecnologías que mitiguen dicho impacto y ofrezcan mayor paralelismo masivo. Por ello, se proponen los siguientes trabajos a realizar en un futuro:

- **Implementación en GPU:** las GPUs ofrecen un ancho de banda de memoria significativamente mayor y están diseñadas para ejecutar de forma paralela miles de hilos simultáneos, lo cual encaja a la perfección con la naturaleza del algoritmo de difusión de contaminantes. Además, se conoce que el clúster

I3US cuenta ya con gráficas NVIDIA A100, por lo que no se requeriría de una excesiva inversión para la realización de la implementación. Sin embargo, el rendimiento efectivo de esta implementación dependerá de comprobar que los hilos realizan operaciones homogéneas y acceden a memoria de forma suficientemente regular, ya que estas condiciones reducen la divergencia de control y permiten aprovechar el paralelismo masivo que proporciona la GPU.

- **Optimización de uso de memoria para clúster:** será necesario modificar y replantear el código para aprovechar mejor a la arquitectura específica del clúster, implementando técnicas avanzadas de bloqueo de caché o partición de datos para minimizar las transferencias entre la RAM y las memorias caché de los procesadores.

Bibliografía

- [1] TIOBE Software. Tiobe index for Python and C, 2024. URL <https://www.tiobe.com/tiobe-index/>.
- [2] Amir Gholami, Zhewei Yao, Sehoon Kim, Coleman Hooper, Michael W. Mahoney, and Kurt Keutzer. Ai and memory wall. *IEEE Micro*, 44(3):33–39, 2024. doi: 10.1109/MM.2024.3373763.
- [3] Luis Miguel Trinidad Salvador and Maria José Morón Fernández. Análisis y optimización microarquitectónica del algoritmo de ruido perlin 2d en amd zen 3. In *Actas de las Jornadas de SARTECO*, 2026. Aceptado (En prensa).
- [4] J Craig Venter et al. The sequence of the human genome. *Science*, 291(5507):1304–1351, 2001.
- [5] Peter Bauer, Alan Thorpe, and Gilbert Brunet. The quiet revolution of numerical weather prediction. *Nature*, 525(7567):47–55, 2015.
- [6] David E Shaw et al. Anton 2: raising the bar for performance and programmability in a special-purpose molecular dynamics supercomputer. *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 41–53, 2014.
- [7] Philippe R Spalart. Strategies for turbulence modelling and simulations. *International Journal of Heat and Fluid Flow*, 21(3):252–263, 2000.
- [8] Veronika Eyring et al. Overview of the coupled model intercomparison project phase 6 (cmip6) experimental design and organization. *Geoscientific Model Development*, 9(5):1937–1958, 2016.
- [9] Andreas Horni, Kai Nagel, and Kay W Axhausen. *The multi-agent transport simulation MATSim*. Ubiquity Press, 2016.
- [10] Pablo Alvarez Lopez et al. Microscopic traffic simulation using sumo. In *21st International Conference on Intelligent Transportation Systems (ITSC)*, pages 2575–2582. IEEE, 2018.
- [11] InfiniBand Trade Association. Infiniband architecture specification, 2000. URL <https://www.infinibandta.org/>.
- [12] Proyecto de investigación SAINEVRA. Enfoque holístico de simulación e ia para la determinación del despliegue nacional de estaciones de recarga para resolver la ansiedad de autonomía en vehículos eléctricos. <https://grupo.us.es/sainevra/>, 2026. PID2023-151065OB-I00, financiado por el Ministerio de Ciencia, Innovación y Universidades / Agencia Estatal de Investigación, MICIU/AEI/10.13039/501100011033/. Universidad de Sevilla. Accessed: 2026-05-13.

- [13] Proyecto de investigación SANEVEC. Un enfoque basado en simulación para determinar el despliegue de una red urbana de estaciones de recarga de vehículos eléctricos con beneficios medioambientales y sociales. <https://grupo.us.es/sanevec/>, 2023. TED2021-130825B-I00, financiado por la Agencia Estatal de Investigación, del Ministerio de Ciencia e Innovación, con fondos NextGenerationEU de la Unión Europea, MICIU/AEI/10.13039/501100011033. Universidad de Sevilla. Accessed: 2026-05-13.
- [14] Amaro García Suárez. Simtravel: Simulador de tráfico urbano para la optimización de una red de estaciones de recarga de vehículos eléctricos. Trabajo fin de grado, Universidad de Sevilla, 2020. URL <https://github.com/amarogs/simtravel>.
- [15] Guido Van Rossum and Fred L Drake Jr. *Python tutorial*. Centrum voor Wiskunde en Informatica Amsterdam, The Netherlands, 1995.
- [16] Charles R. Harris et al. Array programming with numpy. *Nature*, 585(7825): 357–362, 2020.
- [17] Mark Lutz. *Learning Python: Powerful Object-Oriented Programming*. .O'Reilly Media, Inc.", 2013.
- [18] Rui Pereira, Marco Couto, Francisco Ribeiro, Rui Rua, Jácome Cunha, João Paulo Fernandes, and João Saraiva. Ranking programming languages by energy efficiency. *Science of Computer Programming*, 205:102609, 2021. ISSN 0167-6423. doi: 10.1016/j.scico.2021.102609. URL <https://doi.org/10.1016/j.scico.2021.102609>.
- [19] David Beazley. *Understanding the Python GIL*. PyCON, 2010.
- [20] Brian W Kernighan and Dennis M Ritchie. *The C programming language*. Pearson Educación, 1988.
- [21] Richard M Stallman and Developer Community. *Using the GNU Compiler Collection (GCC)*. Free Software Foundation, 2001. URL <https://gcc.gnu.org/onlinedocs/>.
- [22] Leonardo Dagum and Ramesh Menon. Openmp: an industry standard api for shared-memory programming. *IEEE computational science and engineering*, 5(1): 46–55, 1998.
- [23] Instituto de Investigación en Ingeniería Informática (I3US). Infraestructura y clúster hpc de computación avanzada, 2024. URL <https://i3us.us.es>.
- [24] Wm. A. Wulf and Sally A. McKee. Hitting the memory wall: implications of the obvious. *SIGARCH Comput. Archit. News*, 23(1):20–24, mar 1995. ISSN 0163-5964. doi: 10.1145/216585.216588. URL <https://doi.org/10.1145/216585.216588>.
- [25] Python Software Foundation. *ctypes — A foreign function library for Python*, 2026. URL <https://docs.python.org/3/library/ctypes.html>.

- [26] Python Software Foundation. *Python/C API Reference Manual*, 2026. URL <https://docs.python.org/3/c-api/index.html>.
- [27] Armin Rigo and CFFI Developers. *Cffi: Goals and platforms*. <https://cffi.readthedocs.io/en/stable/goals.html>, 2024.
- [28] PyPI. Cython project on PyPI. <https://pypi.org/project/Cython/>, 2026.
- [29] Abraham Silberschatz, Peter Baer Galvin, and Greg Gagne. *Operating System Concepts*. Wiley, Hoboken, NJ, 10th edition, 2018. ISBN 978-1-119-45633-9.
- [30] Microsoft. *Macros (C/C++) - MSVC*. <https://learn.microsoft.com/es-es/cpp/preprocessor/macros-c-cpp?view=msvc-170>, 2023.
- [31] Agner Fog. *Instruction tables: Lists of instruction latencies, throughputs and micro-operation breakdowns for Intel, AMD, and VIA CPUs*. Technical University of Denmark, 2025. URL https://www.agner.org/optimize/instruction_tables.pdf.
- [32] Microsoft. *_restrict - C++ Language Reference (MSVC)*. <https://learn.microsoft.com/es-es/cpp/cpp/extension-restrict?view=msvc-170>, 2023.
- [33] John L. Hennessy, David A. Patterson, and Christos Kozyrakis. *Computer Architecture: A Quantitative Approach*. Morgan Kaufmann, 7th edition, 2025.
- [34] P. Basanta-Val et al. *Gestión de memoria dinámica - Asignatura de Programación (UC3M)*. https://www.it.uc3m.es/pbasanta/asng/course_notes/dynamic_memory_es.html, 2023.
- [35] Python Software Foundation. *gc — Garbage Collector interface*, 2024. URL <https://docs.python.org/3/library/gc.html>. Accedido: 2024.
- [36] Python Software Foundation. *Data model*, 2024. URL <https://docs.python.org/3/reference/datamodel.html>. Accedido: 2024.
- [37] NumPy Developers. *NumPy C-API: Array API*, 2024. URL <https://numpy.org/doc/stable/reference/c-api/array.html>. Accedido: 2024.
- [38] Richard M. Stallman, Roland McGrath, and Paul D. Smith. *GNU Make: A Program for Directing Recompilation*. Free Software Foundation, 2023. URL <https://www.gnu.org/software/make/manual/make.html>.
- [39] IEEE standard for floating-point arithmetic, 2019. Revision of IEEE 754-2008.
- [40] John L Hennessy and David A Patterson. *Computer Architecture: A Quantitative Approach*. Morgan Kaufmann, Cambridge, MA, 6 edition, 2017.
- [41] Advanced Micro Devices, Inc. *AMD Ryzen 7 7700X Desktop Processors Specifications*. <https://www.amd.com/en/product/12161>, 2022.